The Lain Programming Language

Language Specification

Version 1.1

October 9, 2026

Document status:	Draft — normative
Language version:	Lain 1.1
Date:	October 9, 2026

This document is the authoritative specification of the Lain programming language. It defines the syntax, semantics, and constraints that every conforming implementation shall enforce.

Contents

Foreword **ix**

Introduction **xi**

1 Scope **1**
1.1 Compilation model . . . 2

2 Normative references **3**

3 Terms, definitions, and symbols **5**
3.1 General language terms . . . 5
3.2 Type system terms . . . 7
3.3 Ownership and memory terms . . . 8
3.4 Control flow and purity terms . . . 9
3.5 Verification terms . . . 9
3.6 Module and name resolution terms . . . 10

4 Conformance **11**
4.1 Conforming implementation . . . 11
4.2 Conforming program . . . 11
4.3 Diagnostic messages . . . 12
4.4 Implementation limits . . . 12
4.5 Safe and unsafe fragments . . . 13

5 Lexical conventions **15**
5.1 Source file encoding . . . 15
5.2 Comments . . . 15
5.3 Tokens . . . 15
5.4 Whitespace . . . 16
5.5 Keywords . . . 16
5.6 Identifiers . . . 17
5.7 Literals . . . 17
5.7.1 Integer literals . . . 17
5.7.2 Floating-point literals . . . 19
5.7.3 Boolean literals . . . 19
5.7.4 Character literals . . . 19
5.7.5 String literals . . . 20
5.7.6 Escape sequences . . . 20
5.8 Operators and punctuators . . . 21
5.8.1 Arithmetic operators . . . 21
5.8.2 Comparison operators . . . 21
5.8.3 Logical operators . . . 21
5.8.4 Bitwise operators . . . 22

5.8.5 Assignment operators . . . 22
5.8.6 Other operators and punctuators . . . 22
5.9 Token disambiguation . . . 23
5.10 Integer arithmetic and overflow . . . 23

6 Basic concepts **25**
6.1 Declarations and definitions . . . 25
6.2 Scopes . . . 25
6.3 Name resolution . . . 26
6.4 Storage duration . . . 26
6.5 Objects and values . . . 27
6.6 Lvalues and rvalues . . . 27
6.7 Alignment . . . 27

7 Types **29**
7.1 Type taxonomy . . . 29
7.2 Primitive types . . . 29
7.2.1 Integer types . . . 29
7.2.2 Hardware-aligned vs logical integer types . . . 30
7.2.3 Refinement type aliases . . . 31
7.2.4 Packed structs . . . 32
7.2.5 Integer rank and widening . . . 33
7.2.6 Integer overflow . . . 34
7.2.7 Floating-point types . . . 35
7.2.8 Boolean type . . . 36
7.2.9 The void type . . . 36
7.3 Array types . . . 36
7.4 Slice types . . . 37
7.4.1 Sized slice types . . . 38
7.5 Vector types . . . 39
7.6 Struct types . . . 41
7.6.1 Field invariants . . . 42
7.7 Enum types . . . 44
7.8 Algebraic data types (ADTs) . . . 45
7.9 Union types . . . 45
7.10 Pointer types . . . 47
7.11 Constrained types . . . 48
7.12 Type compatibility and equivalence . . . 49

8 Expressions **51**
8.1 Operator precedence and associativity . . . 51
8.2 Primary expressions . . . 51
8.3 Arithmetic operators . . . 52
8.4 Comparison operators . . . 52
8.5 Logical operators . . . 53
8.6 Bitwise operators . . . 53
8.7 The `in` operator . . . 54
8.8 Cast expression (`as`) . . . 55
8.9 Handling a union: `else` and `try` . . . 56
8.10 Move expression (`mov`) . . . 59
8.11 Borrows at call sites . . . 59
8.12 Field access . . . 59

8.13 Index expression . . . 59
8.14 Range index expression (sub-slicing) . . . 59
8.15 Function call . . . 60
8.16 Block expression . . . 61
8.17 If expression . . . 61
8.18 Match expression (`case`) . . . 61
8.19 Constructor expression . . . 61
8.20 Compound assignment . . . 61

9 Statements **63**
9.1 Blocks . . . 63
9.2 Immutable declaration . . . 63
9.3 Mutable variable declaration . . . 64
9.4 Assignment statement . . . 64
9.5 Expression statement . . . 65
9.6 If statement . . . 65
9.7 While statement . . . 66
9.7.1 Bounded while statement . . . 66
9.7.2 Measure inference . . . 67
9.8 Return statement . . . 67
9.9 Defer statement . . . 68
9.10 Unsafe block . . . 68
9.11 Assert and assume statements . . . 68
9.12 Comptime if statement . . . 69
9.13 For statement . . . 69

10 Declarations **71**
10.1 Declaration attributes . . . 71
10.2 Struct declarations . . . 72
10.3 Enum and ADT declarations . . . 72
10.4 Function declarations . . . 73
10.4.1 Parameters . . . 73
10.4.2 Return type annotation . . . 73
10.4.3 Purity constraints . . . 74
10.5 The entry point . . . 74
10.6 Extern declarations . . . 75
10.7 Module-scope constants . . . 75
10.8 Module-scope assertions . . . 77
10.9 Comptime declarations . . . 78

11 Ownership and linearity **79**
11.1 Ownership modes . . . 79
11.2 Linear states . . . 79
11.3 Diagnostics . . . 80
11.4 Mutable parameters for primitive types (`var T`) . . . 81
11.5 The read-write lock invariant . . . 81
11.6 Non-lexical lifetimes (NLL) . . . 81
11.7 Reference bindings . . . 82
11.8 Two-phase borrows . . . 83
11.9 Move semantics . . . 83
11.10 Borrowed match . . . 83
11.11 Loop and ownership . . . 84

11.12Branch consistency 84
11.13Field-level tracking 85

12 Functions and effects **87**
12.1 The effect row 87
12.2 Purity enforcement 88
12.3 Termination analysis 90
12.4 Universal Function Call Syntax (UFCS) 90
12.5 Calling conventions 91
12.6 Return values 91
12.7 Function pointers 91
12.8 The `panic` builtin 93
12.9 Function attributes 94

13 Value range constraints **95**
13.1 Overview 95
13.2 What the analysis tracks 96
13.3 Range propagation rules 96
13.4 Conditional refinement 96
13.5 In-guard semantics 97
13.6 Constraint annotations 97
13.7 Interprocedural range propagation 98
13.8 Limits and known gaps 98
13.9 Loop body affine recap 99
13.10Overflow-at-boundary check 99
13.11Sub-slice bounds 100
13.12Sized slice call-site verification 101
13.13The reference implementation (informative) 101

14 Generics and compile-time evaluation **103**
14.1 Overview 103
14.2 Type parameters 103
14.3 Compile-time value parameters 104
14.4 Type as value 104
14.5 Monomorphization 105
14.6 Comptime expression blocks 106
14.7 Comptime if 106
14.8 Comptime recursion depth 106
14.9 Mode preservation in generics 106

15 Pattern matching **107**
15.1 Overview 107
15.2 Syntax 107
15.3 Patterns 108
15.4 Exhaustiveness 108
15.5 Ownership and match 109
15.6 Case expression result type 109
15.7 String pattern matching 109

16 Module system **111**
16.1 Overview 111
16.2 Module identity 111

16.3 Import declarations . . . 111
16.4 Module loading . . . 112
16.5 Name visibility . . . 113
16.6 Standard library module paths . . . 113
16.7 Name mangling . . . 114

17 C interoperability **115**
17.1 Overview . . . 115
17.2 C headers . . . 115
17.3 Extern function declarations . . . 115
17.4 Type mapping . . . 116
17.5 ABI for function parameters . . . 117
17.6 Pointer operations . . . 117
17.7 String interop . . . 117
17.8 Opaque C types . . . 117

18 Unsafe code **119**
18.1 Purpose . . . 119
18.2 Syntax . . . 119
18.3 What unsafe enables . . . 119
18.4 What unsafe does NOT disable . . . 120
18.5 Undefined behavior in unsafe . . . 121
18.6 Borrow scope for unsafe blocks . . . 122
18.7 Nesting . . . 122

19 Memory model **123**
19.1 Overview . . . 123
19.2 Objects . . . 123
19.3 Storage duration . . . 123
19.4 Stack allocation . . . 124
19.5 Heap allocation . . . 124
19.6 Value semantics . . . 125
19.7 Object representation . . . 125
19.8 Alignment and layout . . . 125
19.9 ADT and enum layout . . . 126
19.10No garbage collector . . . 128
19.11Memory safety in the safe fragment . . . 128
19.12Compiler-internal arena (informative) . . . 128

20 Standard library **129**
20.1 Overview . . . 129
20.2 `std.c` — C standard library bindings . . . 129
20.2.1 Purpose . . . 129
20.2.2 C headers . . . 129
20.2.3 Opaque types . . . 129
20.2.4 Extern declarations . . . 130
20.3 `std.io` — Console I/O . . . 130
20.3.1 Dependencies . . . 130
20.3.2 `print` . . . 130
20.3.3 `println` . . . 130
20.4 `std.fs` — File system . . . 131
20.4.1 Dependencies . . . 131

20.4.2 Type: `File` . . . 131
20.4.3 `open_file` . . . 131
20.4.4 `close_file` . . . 131
20.4.5 `write_file` . . . 131
20.5 `std.math` — Mathematical functions . . . 132
20.5.1 Overview . . . 132
20.5.2 `min` . . . 132
20.5.3 `max` . . . 132
20.5.4 `abs` . . . 132
20.5.5 `clamp` . . . 133
20.6 Generic types (informative) . . . 133
20.6.1 `Option(T)` . . . 133
20.6.2 `Result(T, E)` . . . 133

A Grammar summary **135**

B Diagnostic codes **143**

C Language design rationale **151**
C.1 Declaring effects: the `effects` row . . . 151
C.2 Explicit `mov` at call sites . . . 152
C.3 Implicit NLL lifetimes (no annotations) . . . 152
C.4 C99 translation output . . . 152
C.5 No exception mechanism . . . 153
C.6 Explicit `unsafe` blocks . . . 153
C.7 Value Range Analysis instead of SMT . . . 153
C.8 Type parameters instead of `<T>` syntax . . . 154
C.9 Universal Function Call Syntax (UFCS) . . . 154
C.10 Bounded `while` in `func` . . . 154

D Comparison with related languages **155**
D.1 Feature matrix . . . 155
D.2 Lain versus Rust . . . 155
D.3 Lain versus C . . . 156
D.4 Lain versus Zig . . . 157
D.5 Lain versus Go . . . 157
D.6 Features unique to Lain . . . 158

Index **159**

Foreword

This document specifies the Lain programming language, Version 1.1.

Lain is a statically-typed, compiled programming language for systems programming. It guarantees memory safety, type safety, and resource safety entirely at compile time with zero runtime overhead.

This specification is structured analogously to ISO/IEC 9899 (the C standard). Clauses 1 through 4 are administrative; Clauses 5 through 20 are normative. Annexes A and B are normative; Annexes C and D are informative.

Text marked **Constraint** describes a condition whose violation requires an implementation to issue a diagnostic message. Text marked *implementation-defined*[1] denotes behavior that is valid but whose exact result is determined by the implementation and shall be documented by it. Text marked **undefined behavior**[2] denotes a situation for which this specification imposes no requirements.

> **Note**
>
> Undefined behavior in Lain can arise only inside `unsafe` blocks or through direct C interoperability. All code in the safe fragment of Lain has defined behavior.

[1] Implementation-defined behavior:
[2] Undefined behavior:

Introduction

Design philosophy

Lain is built on five foundational pillars:

P1 — **Assembly-Speed Performance** Every language feature compiles to C code that is equivalent in performance to hand-written C. There are no hidden allocations, no garbage collector, and no reference-counting overhead.

P2 — **Zero-Cost Memory Safety** Memory safety is enforced entirely at compile time through a linear type system, a borrow checker, and value range analysis. A conforming implementation that rejects all constraint violations guarantees, for safe programs, the absence of use-after-free, double-free, null dereference, and buffer overflow at runtime.

P3 — **Static Verification** The verification properties required by this specification are decidable and polynomial in program size. No SMT solver or unbounded fixpoint iteration is required.

P4 — **Determinism** Pure functions (`func`) are referentially transparent and guaranteed to terminate. The same inputs always produce the same outputs.

P5 — **Syntactic Simplicity** Lain has no implicit copies, no hidden conversions (beyond the defined integer-widening rules), and no magic methods. Ownership is always explicit.

Relationship to C

Lain compiles to C99. The C code generated by a conforming Lain implementation is the *translation output*; this specification does not prescribe the form of that output beyond the observable behavior of the resulting program.

How to read this specification

Each normative clause is structured as follows: a prose description of the syntax (with a reference to the formal grammar in Annex A), followed by the semantics, followed by **Constraint** blocks listing every condition that requires a diagnostic message. Non-normative **Note** and **Example** environments appear throughout to aid understanding.

Chapter 1

Scope

1 This document specifies the Lain programming language. It defines the syntax, the semantic rules, and the constraints that every conforming Lain implementation shall enforce. It serves as the single authoritative reference for language users and implementors alike.

2 Lain is a statically-typed, compiled programming language designed for systems programming, embedded systems, and safety-critical software. It achieves memory safety, type safety, and resource safety entirely at compile time with zero runtime overhead.

3 This specification covers:

- lexical structure and grammar (§ 5);
- type system and type inference (§ 7);
- variable declarations, scoping, and initialization (§ 6, § 10);
- expressions, operators, and evaluation order (§ 8);
- statements and control flow (§ 9);
- functions and their effect rows (§ 12);
- ownership, borrowing, and the linear type system (§ 11);
- value range constraints and static verification (§ 13);
- compile-time evaluation and generics (§ 14);
- pattern matching and exhaustiveness checking (§ 15);
- module system (§ 16);
- C interoperability (§ 17);
- unsafe code boundaries (§ 18);
- memory model (§ 19);
- diagnostic messages (Annex B);
- standard library (§ 20).

4 This specification does *not* cover:

- the internal architecture of any particular compiler implementation;
- build system integration or tooling;
- IDE or editor support;
- operating system interfaces beyond what the standard library provides.

1.1 Compilation model

1 Lain uses a source-to-C transpilation model. A Lain source file (extension `.ln`) is processed by a conforming implementation to produce C99 source code, which is subsequently compiled by a C99 compiler to produce an executable. The observable behavior of the program is defined by this specification; the form of the intermediate C99 code is *implementation-defined*[1].

2 A conforming implementation shall perform the following translation phases in the following order:

1. **Lexical analysis** — source text is converted to a token stream as described in § 5.
2. **Parsing** — the token stream is reduced to an abstract syntax tree according to the grammar in Annex A.
3. **Name resolution** — identifiers are resolved to declarations as described in § 6.
4. **Type checking** — types are inferred and verified as described in § 7 through § 10.
5. **Exhaustiveness checking** — pattern matching completeness is verified as described in § 15.
6. **Use analysis (NLL)** — last-use points are computed for all variables.
7. **Linearity and borrow checking** — ownership and borrowing rules are verified as described in § 11.
8. **Value range analysis** — bounds safety and numeric constraints are verified as described in § 13.
9. **Code emission** — well-formed programs are translated to C99.

3 A program that completes all phases without a diagnostic message is **well-formed** and its translation output is defined.

> **Note**
>
> Phases 3–8 collectively form *semantic analysis*. An implementation may interleave these phases as long as the observable behavior is equivalent to executing them in the order listed.

[1] Implementation-defined behavior: the generated C99 code need not be human-readable

Chapter 2

Normative references

1 The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies.

ISO/IEC 9899:2018 (C17)
Information technology — Programming languages — C.
International Organization for Standardization, Geneva, 2018.
Referenced for: C99/C17 calling conventions (§ 17), standard integer types and their representations (§ 7), and undefined behavior categories (§ 18).

IEC 60559:2020 (IEEE 754-2019)
Floating-point arithmetic.
International Electrotechnical Commission, Geneva, 2020.
Referenced for: semantics of `f32` and `f64` arithmetic (§ 7).

2 The following documents are informative references. They are not required for conformance but provide useful background.

ISO/IEC 14882:2020 (C++20)
Programming languages — C++.
Referenced for: vocabulary and structure conventions used in this document.

The Rust Reference
The Rust Reference, `https://doc.rust-lang.org/reference/`.
Referenced for: linear type and ownership terminology.

Chapter 3

Terms, definitions, and symbols

1 For the purposes of this document, the following terms and definitions apply. Terms defined here are used with these precise meanings throughout all normative clauses. Where a term is used in its ordinary English sense rather than as a defined term, it appears without emphasis.

> **Note**
>
> Cross-references of the form *(see § N)* point to the clause where the concept is treated normatively.

3.1 General language terms

abstract syntax tree (AST)

The in-memory tree representation of a Lain program produced by the parser and consumed by semantic analysis.

behavior

External appearance or action of a program as observed by a user or another program. See also: *defined behavior*, *implementation-defined behavior*, *unspecified behavior*, *undefined behavior*.

conforming implementation

An implementation that correctly processes all programs that satisfy the rules of this specification and issues a diagnostic message for every program that violates a **Constraint** (see § 4).

conforming program

A program that uses only the syntax and semantics defined in this specification, does not rely on undefined behavior, and does not depend on implementation-defined extensions.

constraint

A requirement imposed on the source text. If a program violates a constraint, the implementation *shall* emit a diagnostic message. Constraints appear in **Constraint** environments throughout this specification.

defined behavior

Behavior that is fully and precisely specified by this document for every conforming program. Contrast with *undefined behavior* and *implementation-defined behavior*.

diagnostic message

A human-readable message issued by the implementation to report a constraint violation. Every diagnostic shall include a source location (file name, line number) and an error code of the form `[EXXX]` or `[VRA]` (see Annex B).

ill-formed program

A program that violates one or more of the syntactic or semantic rules of this specification. An ill-formed program is not a conforming program and a conforming implementation shall reject it with a diagnostic.

implementation

The system (compiler, linker, runtime) that translates a Lain source program into an executable. The reference implementation is the *lain* compiler.

implementation-defined behavior

Behavior that is valid but not fully specified by this document; the implementation shall determine the exact behavior and document it.

program

A collection of one or more Lain source files (`.ln`) that together constitute a translation unit or a set of modules.

source file

A UTF-8 encoded text file with the extension `.ln` containing Lain source text.

translation output

The C99 source code emitted by a conforming implementation for a well-formed Lain program.

undefined behavior

Behavior for which this specification imposes no requirements. A program that exhibits undefined behavior is erroneous. In the safe fragment of Lain, no operation produces undefined behavior. Undefined behavior may arise only inside `unsafe` blocks or through direct C interoperability.

unspecified behavior

Behavior for which this specification provides two or more possibilities without prescribing which shall occur. A conforming program shall not depend on unspecified behavior.

well-formed program

A program that satisfies all syntactic and semantic rules of this specification and is therefore accepted by a conforming implementation without diagnostic.

3.2 Type system terms

algebraic data type (ADT)

A sum type in which at least one variant carries fields (see § 7.8); a sum whose variants carry none is an enumeration.

constrained type

A type annotated with a value range constraint of the form *T op bound*, e.g. `int >= 0` (see § 7).

integer rank

A total ordering on integer types that determines the result type of mixed-type arithmetic. The rank system is defined in § 7.

integer widening

An implicit conversion from an integer type of lower rank to one of higher rank, permitted in arithmetic and assignment contexts (see § 7).

linear type

A type whose values must be consumed exactly once: a struct with a `mov` field, or one that holds a value of a linear type. Every other type, the primitive types included, is unrestricted (see § 11).

nominal type

A type identified by its declared name rather than its structure.

ownership mode

The kind of access a parameter or binding has: shared (the default, read-only), `var` (read-write), or owned (`mov`: the holder shall consume it) (see § 11).

primitive type

One of the built-in types: the integers `u`*N* and `i`*N* for *N* from 1 to 64, `usize`, `isize` and `int` (an alias of `i32`); `f32`, `f64` and `float` (an alias of `f32`); and `bool`. Primitive types are unrestricted (non-linear).

structural type equivalence

Two types are structurally equivalent if they have the same structure. Lain uses nominal equivalence for user-defined types; the name is part of the type identity.

unrestricted type

A type whose values may be used any number of times without explicit consumption. All primitive types are unrestricted. Contrast with *linear type*.

3.3 Ownership and memory terms

active borrow

A borrow that is currently in the ACTIVE phase; its subject cannot be moved or mutated while the borrow is active (see § 11).

arena

A memory allocation strategy in which all allocations are made from a contiguous block; the entire block is freed at once. Lain's compiler allocates all AST nodes from arenas.

borrow

A temporary access to a value without taking ownership. A shared borrow grants read-only access; a mutable borrow grants read-write access (see § 11).

borrow phase

Either RESERVED (the borrow has been registered but is not yet active, during argument evaluation) or ACTIVE (the borrow is in force) (see § 11).

consumed variable

A linear variable whose ownership has been transferred (via `mov`). Reading or transferring a consumed variable is a constraint violation.

linear state

The tracking state of a linear variable: one of *Uninit*, *Unconsumed*, or *Consumed* (see § 11).

move

The transfer of ownership of a value from one variable or storage location to another, performed by the `mov` operator. After a move, the source is in the Consumed state.

non-lexical lifetime (NLL)

A lifetime that ends at the variable's *last use* rather than at the end of its lexical scope. Lain uses NLL to minimize the duration of borrows (see § 11).

read-write lock invariant

The aliasing invariant: at any program point, for any owned value, there exist either N shared borrows ($N \geq 0$) and no mutable borrow, or exactly one mutable borrow and no shared borrows (see § 11).

resource

A value that represents an external entity (file handle, socket, etc.) whose lifetime must be explicitly managed. Resources are modelled as linear types.

two-phase borrow

A borrow that begins in the RESERVED phase during argument evaluation and is promoted to ACTIVE after the call expression is complete (see § 11).

3.4 Control flow and purity terms

decreasing measure

An expression that the implementation shall verify is non-negative and strictly decreases on every iteration of a loop, or at every recursive call. It is written with `decreasing`, or inferred (see § 9.7.2 and § 12.3).

`func`

A function. Its effects are those its **effect row** names, and the row's default is empty — pure and terminating. A function shall not have an effect its row does not name (see § 12).

purity

The property of a function with an empty row: its result depends only on its inputs and it has no observable side effects on any state outside its own stack frame.

effect row

The set of effects a function may have, drawn from `io`, `diverge`, `raises` and `alloc`. Inferred from the body, empty by default, widened by an `effects` clause, and a complete upper bound in either case (see § 12.1).

`proc`

Removed. An introducer that granted effects rather than stating them; the effect row replaced it. The keyword is still reserved and diagnoses the replacement (see § 10.5).

termination

The property that a function always returns, in a finite number of steps. Every function whose effect row does not name `diverge` shall be verified to terminate by the implementation (see § 12).

3.5 Verification terms

bounds check

The obligation that an index lies within `[0, N-1]`. Lain discharges it at compile time and emits no run-time test: an index the analysis cannot prove in range is *ill-formed* (`[E085]`), except inside `unsafe`, which waives it.

comptime

Compile-time evaluation. The value of a module-scope constant is computed by the compiler (§ 10.7), and `comptime if` selects code at compile time; a `comptime { ... }` expression is planned (see § 14).

exhaustiveness

The property of a `case` expression in which every possible value of the scrutinee type is covered by at least one arm (see § 15).

in guard

A use of the `in` operator as a loop or conditional guard. An in guard of the form `idx in 0..arr.len` proves to the VRA that `idx` $\in$ `[0, arr.len-1]` in the guarded body (see § 8, § 13).

interval

A pair $[\ell, u]$ representing the set of integers $\{x \mid \ell \leq x \leq u\}$. VRA represents value ranges as intervals with $-\infty$ and $+\infty$ sentinels.

monomorphization

The process by which a generic function is instantiated for each concrete set of type arguments, producing a separate, non-generic translation for each instantiation (see § 14).

return constraint

A value range constraint on the return value of a function, declared in the function signature as *T op bound* (see § 13).

value range analysis (VRA)

A static analysis that tracks integer value ranges through the program using interval arithmetic, used to eliminate bounds checks and prove constraint satisfaction (see § 13).

3.6 Module and name resolution terms

import

A directive (`import module.path`) that makes the names defined in another module available in the current module (see § 16).

module

The unit of compilation in Lain. Each source file defines one module.

qualified name

A name of the form *module.identifier* that refers to a declaration in a specific module (see § 6).

Universal Function Call Syntax (UFCS)

A syntactic sugar by which `x.f(args)` is desugared to `f(x, args)` before name resolution. UFCS allows methods to be defined as free functions (see § 12).

Chapter 4

Conformance

4.1 Conforming implementation

1 A **conforming implementation** shall:

1. accept, without diagnostic, every *conforming program*;
2. reject, with at least one diagnostic message, every program that violates any constraint stated in this specification;
3. translate accepted programs such that their observable behavior conforms to the semantic rules stated in this specification;
4. implement all features described as normative in Clauses 5 through 20;
5. document every behavior described in this specification as *implementation-defined*[1].

2 A conforming implementation may:

1. issue additional diagnostic messages (warnings) beyond those required by this specification, provided they do not cause a conforming program to be rejected;
2. accept programs through implementation-defined extensions, provided those extensions are documented and do not alter the semantics of any conforming program;
3. emit no run-time check for an obligation it has discharged at compile time (§ 13); the reference implementation emits none.

> **Note**
>
> The reference implementation of Lain (*lain* compiler, version corresponding to this specification) is intended to be a conforming implementation. Where it is known not to conform, the clause concerned says so in a note. Where this specification marks a behavior as implementation-defined, the reference implementation's documented behavior takes precedence for programs compiled by that implementation.

4.2 Conforming program

1 A **conforming program** is a program that:

1. consists only of syntax and semantics defined in this specification;

[1] Implementation-defined behavior: the exact value or behavior is implementation-defined

2. does not rely on undefined behavior;
3. does not rely on unspecified behavior to produce a specific outcome;
4. does not depend on any implementation-defined extension.

2 A **strictly conforming program** additionally:

1. does not use any behavior documented as implementation-defined, even when that behavior is deterministic on a given implementation.

4.3 Diagnostic messages

1 A diagnostic message issued for a constraint violation shall include:

1. the source file name;
2. the line number of the offending construct;
3. the diagnostic code in the form `[EXXX]` (see Annex B);
4. a human-readable description of the violation.

2 A conforming implementation may issue additional diagnostic information (column number, source context, suggested fix) beyond the minimum required by §4.3.

> **Note**
>
> The reference implementation additionally prints the source line and a caret (`^`) pointing to the column of the offending token.

4.4 Implementation limits

1 A conforming implementation shall support at least the following minimum translation limits. Programs that require more than these limits invoke *implementation-defined*[2].

Resource	Minimum	Clause
Identifier length (bytes)	255	§5
Block nesting depth	64	§9
Function parameters	127	§10
Array elements	$2^{31} - 1$	§7
Module import depth	32	§16
`case` arms per match expression	256	§15
Struct fields	256	§7
Enum variants	256	§7
Comptime evaluation depth	256	§14
Type parameters of a generic type or function	8	§14.2
Scope nesting depth	64	§6

[2]Implementation-defined behavior: the behavior when a translation limit is exceeded

> **Note**
>
> The reference implementation accepts at most 64 value parameters in a *generic* function, fewer than the minimum for function parameters above, which it meets for a function that is not generic. It refuses a generic function with more at the declaration (`[E124]`), as it does a declaration with more than eight type parameters; both are known limitations.

4.5 Safe and unsafe fragments

1 The **safe fragment** of a Lain program consists of all code that does not appear inside an `unsafe` block and does not use raw pointer operations.

2 A conforming implementation shall guarantee, for any well-formed program in the safe fragment, that execution produces no:

- use of a variable after its ownership has been transferred (use-after-move);
- double transfer of ownership of the same value;
- access to an uninitialized variable;
- violation of the read-write lock invariant (§3.3);
- buffer overrun that has been statically proven safe by value range analysis.

3 The **unsafe fragment** comprises all code inside `unsafe` blocks. Within the unsafe fragment, the programmer is responsible for upholding the memory safety invariants listed in §18; the implementation shall not be required to verify them.

> **Constraint**
>
> The constraints of the ownership and linearity system (§11) are *not* suspended inside `unsafe` blocks. Only raw pointer operations, direct ADT field access, and the bounds-check requirement are relaxed. An implementation shall continue to enforce the ownership and linearity diagnostics (`[E001]`–`[E008]` and `[E016]`) inside `unsafe` blocks.

Chapter 5

Lexical conventions

1 A Lain source file is translated in two conceptual steps: (1) the source text is decomposed into tokens as described in this clause; (2) the token stream is parsed into an abstract syntax tree according to the grammar. Implementation may combine these steps.

5.1 Source file encoding

1 A Lain source file shall be a sequence of bytes encoded in UTF-8 (Unicode Standard, Annex D). The byte order mark (U+FEFF) shall not appear in a source file. If it does, behavior is *implementation-defined*[1].

2 Multi-byte UTF-8 sequences are permitted inside string literals and comments but shall not appear in identifiers. Identifiers shall consist solely of ASCII characters (see §5.6).

3 Line endings are normalized: a carriage return (U+000D) followed immediately by a line feed (U+000A) is treated as a single line feed. A lone carriage return is treated as a line feed.

5.2 Comments

1 A *line comment* begins with `//` and extends to the first following line feed character. The line feed is not part of the comment.

2 A *block comment* begins with `/*` and ends with the nearest following `*/`. Block comments may be nested: each opening `/*` inside a block comment opens a new nesting level requiring its own closing `*/`.

3 Comments are stripped during lexical analysis and have no effect on the semantics of the enclosing program.

> **Constraint**
>
> An unterminated block comment (one for which no matching `*/` is found before the end of the source file) is *ill-formed* (`[E100]`).

5.3 Tokens

1 The result of lexical analysis is a sequence of tokens. Whitespace and comments serve solely to separate tokens. The grammar production for *token* is given in Annex A (§5.3).

2 Where the token stream is ambiguous, the lexer shall apply the *maximal munch* rule: at each position, the longest sequence of characters that forms a valid token is consumed.

> **Example**
>
> ```
> // SYNOPSIS: tokens, not a program
> ```

[1] Implementation-defined behavior: whether a leading BOM is silently skipped or causes a diagnostic

```
..=     // one token: range-inclusive, NOT '.' followed by '.='
>=      // one token: greater-than-or-equal, NOT '>' followed by '='
```

3 Keywords (§ 5.5) take priority over identifiers: the character sequence `func` at a point where an identifier is expected is always the keyword **`func`**.

5.4 Whitespace

1 The whitespace characters are: space (U+0020) and horizontal tab (U+0009). They serve only to separate adjacent tokens.

2 Line feeds (U+000A), after normalization (§ 5.1), are significant: they act as statement terminators in contexts where a statement may follow (see § 9). Multiple consecutive line feeds are treated as a single statement boundary.

3 A semicolon (`;`) may be used as a statement separator, equivalent to a line feed. Its purpose is to place more than one statement or declaration on a single line; a newline is the normal terminator, and no semicolon is needed at the end of a line. Two statements on one line without a separator are ill-formed (**`[E100]`**).

> **Rationale**
>
> Newlines as the primary terminator eliminate the class of bugs caused by *missing* terminators. A semicolon is available only when the programmer deliberately wants several statements on one line (e.g. `a = 1; b = 2`), making a multi-statement line an explicit, opt-in choice rather than something that can happen by accident.

5.5 Keywords

1 The following identifiers are reserved as keywords. They shall not be used as user-defined identifiers. The grammar production is *keyword* (Annex A).

Keyword	Purpose	Clause
`and`	Logical AND operator	§ 8
`as`	Type cast	§ 8
`assert`	Proof obligation; module-scope assertion	§ 9.11
`assume`	A fact taken without proof (**`unsafe`** only)	§ 9.11
`break`	Loop break	§ 9
`c_include`	*reserved*; refused, see **`[E100]`**	§ 17.2
`case`	Pattern matching	§ 15
`comptime`	Compile-time evaluation	§ 14
`continue`	Loop continue	§ 9
`decreasing`	Bounded-loop termination measure	§ 9
`defer`	Deferred execution	§ 9
`else`	Alternative branch	§ 9
`extern`	Foreign function declaration	§ 17
`false`	Boolean literal	§ 5.7
`for`	Range-based iteration	§ 9
`func`	Function declaration	§ 12

(continued)

Keyword	Purpose	Clause
`if`	Conditional	§ 9
`import`	Module import	§ 16
`in`	Range iteration / bounds guard	§ 8
`mov`	Ownership transfer	§ 11
`nil`	*reserved*; removed: absence is the marker `none`	—
`or`	Logical OR operator	§ 8
`proc`	*reserved*; removed, see `[E100]`	§ 10.5
`effects`	Effect row	§ 12.1
`return`	Return statement	§ 9
`true`	Boolean literal	§ 5.7
`try`	Propagate a union's marker to the caller	§ 8
`type`	Type / struct / enum declaration	§ 10
`unsafe`	Unsafe block	§ 18
`use`	Local-scope import / alias	§ 16
`var`	Mutable binding / mutable borrow	§ 10
`while`	Loop statement	§ 9

Constraint

An identifier that is identical to a keyword shall not be used in any context that requires an identifier. Such use is *ill-formed* (`[E100]`).

5.6 Identifiers

1 An identifier consists of ASCII letters, decimal digits, and underscores, and shall begin with a letter or underscore. The grammar production is *identifier* (Annex A).

2 Identifiers are case-sensitive: `foo`, `Foo`, and `FOO` are three distinct identifiers.

3 A conforming implementation shall support identifiers of at least 255 characters. The behavior when an identifier exceeds this length is *implementation-defined*[2].

Constraint

An identifier shall not be identical to a keyword (§ 5.5).

Constraint

An identifier shall begin with a letter or underscore, not a digit. A sequence of characters that begins with a digit is an integer literal (§ 5.7), not an identifier.

5.7 Literals

5.7.1 Integer literals

1 An integer literal denotes a constant integer value. Four notations are supported (grammar: *integer-literal*):

[2] Implementation-defined behavior: whether oversized identifiers are truncated or cause a diagnostic

Notation	Prefix	Example
Decimal	(none)	`42`, `1_000_000`
Hexadecimal	`0x`	`0xFF`, `0x1A_3B`
Binary	`0b`	`0b1010_0101`
Octal	`0o`	`0o755`

2 An underscore (`_`) may appear between any two digits of an integer or floating-point literal as a visual separator. It has no effect on the value: `1_000.5` is `1000.5`.

3 A hexadecimal, octal or binary literal denotes a *bit pattern* of at most 64 bits, read as a two's-complement `i64`: `0x7FFF_FFFF_FFFF_FFFF` is the largest `i64` and `0x8000_0000_0000_0000` the smallest. A pattern with the top bit set is therefore negative, and a `u64` context does not accept it implicitly; `as%` takes the same bits as the unsigned value, e.g. `0xcbf29ce484222325 as% u64`. A decimal literal denotes its value. In an expression, a decimal literal above the range of `i64` that a `u64` holds, 2^{63} to $2^{64} - 1$, is a `u64`: it is its bit pattern taken `as% u64`, as for hexadecimal. Everywhere else a literal's value is read as an `i64`: in a refinement bound, an array length and a pattern, a decimal literal shall fit in `i64`.

> **Constraint**
>
> A hexadecimal, octal or binary literal of more than 64 bits, a decimal literal above $2^{64} - 1$, or, outside an expression, a decimal literal outside the range of `i64`, is *ill-formed* (`[E086]`).

4 The range analysis holds no constant above the range of `i64`, so arithmetic that must be proven on such a value is refused even where it fits: with `m` = `10000000000000000000`, `m + 1` is `[E086]`, and `m +% 1` is accepted.

```
// VERIFY: exit 0
BIG u64 = 18446744073709551615          // above i64's range: a u64

func main() i32 {
    var u u64 = 18446744073709551615
    if u != BIG { return 1 }
    if BIG / 2 != 9223372036854775807 { return 2 }
    if u +% 1 != 0 { return 3 }
    return 0
}
```

```
func f() i64 {
    x i64 = 10000000000000000000    // ERROR [E086]: a u64 value does not fit an i64
    return x
}
```

5 The type of an integer literal is inferred from context (see § 7). In the absence of type context, its type is `i32` (whose alias is `int`) when the value fits, `i64` when it does not, and `u64` for a decimal literal above the range of `i64`.

> **Constraint**
>
> An integer literal whose value does not fit in the type determined by context is *ill-formed* (`[E086]`) — it is diagnosed by the same range mechanism that rejects arithmetic overflow (§ 13.10).

Example

```
var a = 255          // i32, value 255
var b u8 = 255       // u8, value 255 -- OK
var c u8 = 256       // [E086]: 256 does not fit in u8
var d = 0xFF         // i32, value 255
var e = 0b1111_0000  // i32, value 240
var f = 1_000_000    // i32, value 1000000
var g = 5000000000   // i64: too large for an i32
```

5.7.2 Floating-point literals

A floating-point literal consists of a decimal integer part, a decimal point, and a fractional part, optionally followed by an exponent part `e`*N* or `E`*N*; or of a decimal integer part followed by an exponent part, with no point (`1e9`, `5E-3`, `2e+2`). The exponent's sign is optional and it has at least one digit, so `2e` is the integer `2` followed by the identifier `e` (grammar: *float-literal*).

The default type of a floating-point literal is `f64`.

```
// VERIFY: exit 0
func main() i32 {
    big f64 = 1e9
    small f64 = 5E-3
    if big != 1000000000.0 { return 1 }
    if small != 0.005 { return 2 }
    return 0
}
```

Example

```
var x = 3.14       // f64
var y f32 = 1.0    // f32
var z = 2.5e3      // f64, value 2500.0
```

5.7.3 Boolean literals

The boolean literals are **`true`** and **`false`** (grammar: *bool-literal*). Both have type `bool`.

5.7.4 Character literals

A character literal is a single byte value enclosed in single quotes. It has type `u8`. Escape sequences (§ 5.7.6) are permitted.

Example

```
var a = 'A'     // u8, value 65
var n = '\n'    // u8, value 10
var z = '\0'    // u8, value 0
```

Constraint

A character literal shall contain exactly one character or one escape sequence. An empty or multi-character character literal is *ill-formed* (**`[E100]`**).

Constraint

A character literal whose closing single quote is not found before the end of the source file is *ill-formed* (`[E100]`). The diagnostic is at its opening quote.

5.7.5 String literals

10 A string literal is a sequence of zero or more characters enclosed in double quotes. Escape sequences (§5.7.6) are permitted (grammar: *string-literal*).

Constraint

A string literal whose closing double quote is not found before the end of the source file is *ill-formed* (`[E100]`). The diagnostic is at its opening quote.

11 The type of a string literal of N visible characters is `Fixed_u8_`N`+1` (an array of $N+1$ bytes, null-terminated). This type is implicitly coercible to `u8[]` (a byte slice).

12 The `.len` property of a string literal value equals N (excluding the null terminator).

Example

```
var s = "hello"    // type: Fixed_u8_6 (5 chars + null)
                   // s.len == 5
```

13 String literals are stored in the read-only data segment of the translation output. Modifying a string literal through a pointer is **undefined behavior**[3].

5.7.6 Escape sequences

14 The following escape sequences are recognized in string and character literals:

Escape	Byte value	Description
`\n`	0x0A	Line feed (newline)
`\t`	0x09	Horizontal tab
`\r`	0x0D	Carriage return
`\0`	0x00	Null byte
`\\`	0x5C	Backslash
`\"`	0x22	Double quote
`\'`	0x27	Single quote
`\x`*HH*	0x*HH*	The byte whose value is the two hex digits *HH*

Constraint

A backslash not followed by one of the characters in the table above is an unrecognized escape sequence and is *ill-formed* (`[E100]`). A `\x` not followed by exactly two hex digits is *ill-formed* (`[E100]`); a third digit is an ordinary character (`"\x414"` is the two bytes `A` and `4`).

[3] Undefined behavior: the result of writing through a pointer derived from a string literal

```
// VERIFY: exit 0
func main() i32 {
    var s = "\x414"          // \x41 is 'A'; the 4 after it is an
        ordinary character
    if s.len != 2 { return 1 }
    if s[0] != 65 { return 2 }
    if s[1] != 52 { return 3 }
    return 0
}
```

5.8 Operators and punctuators

The complete set of operator and punctuator tokens is listed below. The grammar productions *operator* and *punctuator* are in Annex A.

5.8.1 Arithmetic operators

Token	Meaning
`+`	Addition (there is no unary `+`)
`-`	Subtraction; unary negation
`*`	Multiplication
`/`	Division
`%`	Modulo (remainder)
`+? -? *?`	Checked add/sub/mul (`panic` on overflow)
`+% -% *% /%`	Wrapping add/sub/mul/divide (modular)
`+| -| *| /|`	Saturating add/sub/mul/divide (clamp)

The plain `+`, `-`, `*` auto-widen their result so they cannot overflow (§7.2.5); the `?`/`%`/`|` variants keep the operand type and select an explicit overflow policy (§7.2.6).

5.8.2 Comparison operators

Token	Meaning
`==`	Equal
`!=`	Not equal
`<`	Less than
`>`	Greater than
`<=`	Less than or equal
`>=`	Greater than or equal

5.8.3 Logical operators

Token	Meaning
`and`	Logical AND (short-circuit)
`or`	Logical OR (short-circuit)
`!`	Logical NOT (prefix)

5.8.4 Bitwise operators

Token	Meaning
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`«`	Left shift
`»`	Right shift
`«%`	Left shift that discards the bits shifted out
`~`	Bitwise complement (prefix)

5.8.5 Assignment operators

Token	Meaning
`=`	Assignment
`+=`	Add and assign
`-=`	Subtract and assign
`*=`	Multiply and assign
`/=`	Divide and assign
`%=`	Modulo and assign
`&=`	Bitwise AND and assign
`|=`	Bitwise OR and assign
`^=`	Bitwise XOR and assign
`«=`	Left shift and assign
`»=`	Right shift and assign

5.8.6 Other operators and punctuators

Token	Meaning
`.`	Member access; module path separator
`..`	Exclusive range (half-open interval)
`..=`	Inclusive range
`...`	Variadic parameters (`extern` only)
`(`	Open parenthesis

Token	Meaning
`)`	Close parenthesis
`{`	Open brace
`}`	Close brace
`[`	Open bracket
`]`	Close bracket
`,`	List separator
`:`	Case-arm separator; the sentinel of a slice type (`u8[:0]`)
`;`	Statement separator, equivalent to a newline
`@`	Prefix of a builtin (`@sizeof`, `@load`)
`?`	After `as`, a checked cast (`as?`)

3 Attributes use bracket syntax `[name]` or `[name(args)]` before a declaration (§ 10). The brackets `[` and `]` listed above serve both as array/slice subscripts and as attribute delimiters; the context (declaration position vs. expression position) determines the role.

5.9 Token disambiguation

1 Several token sequences have context-dependent meaning that is resolved by the parser, not the lexer. The lexer produces the same token regardless of context:

Symbol	Context 1	Context 2
`*`	Multiplication (binary)	Pointer dereference (unary prefix, only in `unsafe`)
`-`	Subtraction (binary)	Negation (unary prefix)
`.`	Member access	Module path separator
`&`	Bitwise AND (binary)	Address-of (unary prefix); after `case`, a borrowed match
`|`	Bitwise OR (binary)	In a type, the union separator (`T | none`); after `as`, a saturating cast
`%`	Remainder (binary)	After `as`, a wrapping cast

5.10 Integer arithmetic and overflow

1 Integer arithmetic in Lain uses two's complement representation for all signed integer types.
2 A program in which an integer operation can overflow is *ill-formed*. The plain operators `+`, `-` and `*` carry a proof obligation: the result shall be shown to lie within its type. Where the implementation cannot show it, the program is *ill-formed* (`[E086]`). Overflow is therefore a diagnosable constraint violation, not a runtime behaviour.
3 `+`, `-` and `*` *widen*: the result of an operation on two operands of width $N < 64$ has a type wide enough to represent every result, so the operation itself cannot overflow. The obligation attaches instead to the *narrowing* — the point at which that result is stored into, passed as, or returned as a narrower type. At width 64 no wider type exists and the obligation attaches to the operation.
4 Overflow that is intended shall be written as such. Each form is a distinct operator, and each is well-formed by construction:

- **`+%`**, **`-%`**, **`*%`** — *modular*: the result wraps modulo 2^N.
- **`+|`**, **`-|`**, **`*|`** — *saturating*: the result is clamped to the bounds of its type.
- **`+?`**, **`-?`**, **`*?`**, **`as?`** — *checked*: the operation shall be handled by an **`else`** arm supplying a fallback value, propagating with **`return`**, or **`panic`**ing (**`[E067]`**).

5 Within an **`unsafe`** block the obligation is waived for every operation lexically inside it, including the narrowing. Responsibility passes to the program.

6 A conforming implementation that emits C99 shall emit translation output in which no operation it has accepted is undefined behaviour. This is a requirement on the *translation output*, not a licence for the source language. The reference implementation meets it without any compiler flag: a wrapping operation is computed in the unsigned type of its width and converted back, `(int32_t)((uint32_t)a + (uint32_t)b)`. The unsigned operation is defined by ISO C; the conversion back is implementation-defined there, and modular on every C compiler the reference implementation targets.

> **Note**
>
> Unsigned arithmetic in ISO C is modular and signed overflow is undefined. Lain admits neither as a source-language behaviour for the plain operators: a program that would rely on either is rejected, and a program that wants one says which. See § 13.10.

Chapter 6

Basic concepts

6.1 Declarations and definitions

1 A *declaration* introduces a name into a scope and associates it with an entity: a variable, a function, a type, a constant, or a module. A name declared at module scope may be used anywhere in its module, above its declaration as well as below it; a local name only after its declaration (`[E106]`, §6.3).

2 A *definition* is a declaration that also allocates storage (for variables) or provides a body (for functions). In Lain, every declaration at the local level is also a definition.

Constraint

Redeclaring a name already visible in the current or an enclosing scope — a duplicate parameter name, or a local that repeats or shadows an outer binding — is *ill-formed* (`[E013]`). Lain forbids shadowing (§6.2).

Constraint

A module-scope name is defined once: two functions, two types, or a constant and a function of one name are *ill-formed* (`[E013]`), in one module or across the modules a program imports, which share one namespace. Two `extern` declarations of one function are allowed when they agree: the same parameter and return types, variadicity and effect row; two that disagree are *ill-formed* (`[E013]`).

Constraint

Use of an undeclared identifier is *ill-formed* (`[E106]`). The reference implementation issues this diagnostic when the translation output is generated — after name resolution, monomorphization, and UFCS desugaring are complete — so that a name still unbound at that point is unambiguously undeclared.

Example

```
func f() {
    var x = 1
    var x = 2   // [E013]: redeclaration of 'x' is forbidden
}
```

6.2 Scopes

1 A *scope* is a region of program text within which a declared name can be used. Lain has the following scope levels:

Module scope Names declared at the top level of a source file are in module scope. Module-scope names are accessible throughout the module and, by default, accessible from other modules that import this module.

Function/procedure scope Parameter names are in the scope of the function or procedure body.

Block scope Names declared inside a block (`{ }`) are in the scope of that block. They are not accessible outside the enclosing block.

2 Scopes are *lexically nested*. Lain *forbids shadowing* entirely: a declaration whose name is already visible in the current scope or in any enclosing scope — a function parameter shadowed by a local, or a name redeclared in a nested block — is *ill-formed* (`[E013]`).

> **Rationale**
>
> Disallowing shadowing removes a class of bugs in which an inner binding silently captures a name the reader expects to denote an outer one. Every identifier in a function body therefore resolves to exactly one declaration.

3 A conforming implementation shall support at least 64 levels of nested block scope (§ 4.4).

6.3 Name resolution

1 Name resolution maps an identifier in source text to its declaration. The lookup algorithm is:

1. Search the innermost block scope.
2. If not found, search progressively wider enclosing block scopes.
3. If not found, search the function/procedure scope (parameters).
4. If not found, search the module scope.
5. If not found, search imported module scopes (in import order).
6. If not found, attempt UFCS resolution (§ 12).
7. If still not found, the identifier is undeclared and the program is *ill-formed* (`[E106]`).

2 Because shadowing is forbidden (§ 6.2), a name resolves to at most one declaration along the scope chain: a lookup that would find a second binding of the same name is rejected at the point of the redeclaration (`[E013]`), not silently resolved by an innermost-wins rule.

6.4 Storage duration

1 Every object has a *storage duration* that determines the lifetime of its storage.

Automatic storage duration Variables declared inside a block or function body. Storage is allocated on entry to the block and deallocated on exit.

Static storage duration Module-scope constants (§ 10.7). Each is computed at compile time and is immutable, and its storage persists for the lifetime of the program. There are no module-scope variables.

Heap duration Objects allocated through C interoperability; their lifetime is the program's to manage, and the ownership rules track it (§ 19.3).

6.5 Objects and values

1 An *object* is a region of storage interpreted according to a type. Every declared variable corresponds to an object.
2 Every variable has:

- a *declared type* (§ 7);
- an *ownership mode*: owned (default for linear types), `shared` (read-only reference), or `var` (mutable reference) (§ 11);
- a *linear state*: Uninit, Unconsumed, or Consumed (for linear types only) (§ 11).

6.6 Lvalues and rvalues

1 An *lvalue* is an expression that designates an object with a persistent storage location: a variable identifier, a field access on an lvalue, or an array index expression on an lvalue.
2 An *rvalue* is an expression whose value has no persistent storage location: literals, the result of arithmetic operations, or the result of a function call.
3 Assignment requires the left-hand side to be an lvalue.

> **Constraint**
>
> An assignment whose left-hand side is an rvalue is *ill-formed* (`[E100]`).

6.7 Alignment

1 Every type has a required alignment. The alignment of a type is *implementation-defined*[1]. Struct layout follows C99 struct layout rules, including padding, over the fields in the storage order of § 7.6.
2 Stack-allocated variables are aligned to their type's required alignment. The implementation shall not insert additional padding between local variables unless required by the hardware.

[1] Implementation-defined behavior: the same as the alignment of the corresponding C99 type

Chapter 7

Types

1 Lain is statically typed. Every variable, expression, and parameter has a type determined at compile time. There is no runtime type information (RTTI). The grammar productions for types are in Annex A (*type*).

7.1 Type taxonomy

1 The types of Lain are organized as follows:

- *Primitive types*: integer, floating-point, boolean, void (§ 7.2);
- *Array types*: fixed-size, compile-time-sized (§ 7.3);
- *Slice types*: dynamically-sized views into contiguous memory, including sized slice types with static length constraints (§ 7.4);
- *Vector types*: a fixed number of lanes, operated on lane by lane (§ 7.5);
- *Struct types*: named product types (§ 7.6);
- *Enum types*: simple enumerations (§ 7.7);
- *ADT types*: sum types with associated data (§ 7.8);
- *Pointer types*: raw memory addresses, unsafe only (§ 7.10);
- *Constrained types*: types with value range annotations (§ 7.11).

2 Types are classified as *linear* or *unrestricted*. Linear types must be consumed exactly once (see § 11). Unrestricted types may be used any number of times.

3 All primitive types are unrestricted. A struct type is linear if and only if at least one of its fields is of a linear type. This property is transitive.

7.2 Primitive types

7.2.1 Integer types

1 The integer types are listed below. Sizes marked *(platform)* are *implementation-defined*[1].

Type	Signedness	Bit width	Range	C99 type
`u8`	unsigned	8	$[0, 2^8 - 1]$	`uint8_t`

[1] Implementation-defined behavior: determined by the target platform as in the C99 standard

(continued)

Type	Signedness	Bit width	Range	C99 type
`u16`	unsigned	16	$[0, 2^{16} - 1]$	`uint16_t`
`u32`	unsigned	32	$[0, 2^{32} - 1]$	`uint32_t`
`u64`	unsigned	64	$[0, 2^{64} - 1]$	`uint64_t`
`usize`	unsigned	(platform)	$[0, 2^{N} - 1]$	`size_t`
`i8`	signed	8	$[-2^{7}, 2^{7} - 1]$	`int8_t`
`i16`	signed	16	$[-2^{15}, 2^{15} - 1]$	`int16_t`
`i32`	signed	32	$[-2^{31}, 2^{31} - 1]$	`int32_t`
`i64`	signed	64	$[-2^{63}, 2^{63} - 1]$	`int64_t`
`isize`	signed	(platform)	$[-2^{N-1}, 2^{N-1} - 1]$	`ptrdiff_t`
`int`	signed	32	$[-2^{31}, 2^{31} - 1]$	`int32_t`

2 `int` is a documented alias of `i32`, identical in every semantic respect (boundary checks, niche optimization, codegen). Use either spelling — `int` when bit-width is incidental, `i32` when explicit width matters.

3 Generalized integer types `iN`/`uN` are admitted for any $N \in [1, 64]$. Hardware-aligned widths ($N \in \{1, 8, 16, 32, 64\}$) map to standard C99 stdint types. Logical widths (other N) occupy the smallest hardware-aligned container that fits N bits when standalone; in a `[packed]` struct they occupy exactly N bits.

4 `usize` and `isize` are the platform-native unsigned and signed integer types whose width equals the pointer width. Their exact width is *implementation-defined*[2].

7.2.2 Hardware-aligned vs logical integer types

5 The `iN`/`uN` types fall into two semantic categories:

Hardware-aligned ($N \in \{1, 2, 4, 8, 16, 32, 64\}$, with the common subset being $\{8, 16, 32, 64\}$): these correspond to fundamental C99 stdint widths (`uint8_t`, `int32_t`, etc.). They are the canonical choice for C interop and for any context where exact storage size matters at the byte boundary.

Logical (other N): these are *bit-width quantities*. Their semantics are context-dependent:

- **Standalone**: storage uses the smallest hardware-aligned container fitting N bits (e.g., `u12` maps to `uint16_t`). The natural range is $[-2^{N-1}, 2^{N-1} - 1]$ signed or $[0, 2^{N} - 1]$ unsigned.
- **Inside a packed struct** (future feature, § 7.2.4): the type occupies exactly N bits of storage with bit-exact layout.

6 **Distinction from refinement.** Logical `iN`/`uN` types differ from a refinement constraint on `int`:

- `int >= 0 and <= 7` declares an `int` with a range constraint. Storage is the `int` default (`int32_t`); bit-width is not specified.
- `u3` declares a 3-bit quantity. Standalone its range is $[0, 7]$ and its storage is `uint8_t`; in a `[packed]` struct (§ 7.2.4) it occupies exactly 3 bits.

The two forms are interchangeable when only the range matters; they differ when bit-exact storage layout matters.

[2] Implementation-defined behavior: the pointer width on the target platform

Example

```
// A refinement constraint on int, named: range [0, 4095],
    container = int32_t.
// (A constraint is written on a parameter, a field or an alias
    like this one,
// not on a local's declaration.)
type Twelve = int >= 0 and <= 4095
type Pressure = int >= 0 and <= 1000

func main() int {
    var x u12 = 4095        // logical bit-width: a 12-bit
        quantity, container = uint16_t
    var y Twelve = 4095     // the same range as an int
        refinement
    var p Pressure = 500
    return 0
}
```

7.2.3 Refinement type aliases

7 A type alias may carry a refinement constraint that narrows the underlying type's value range:

```
type Percent = int >= 0 and <= 100
type Pressure = int >= 0 and <= 1000
type NonZero = int != 0
```

8 Variables declared with a refinement type alias inherit its constraint. Assignments whose source value range violates the alias constraint are *ill-formed* (**[E086]**).

Example

```
type Percent = int >= 0 and <= 100

func main() int {
    var ok Percent = 75              // accepted: 75 fits [0,
        100]
    // var bad Percent = 150         // refused, E086: 150 > 100
    return 0
}
```

9 A refinement on an element type is part of the type. An array, a slice or a pointer converts only to one whose element type is refined the same way (**[E012]**): every reader of an element assumes its type's range, and a write through a `var` reference must keep it. So an `i32[2]` does not convert to a `Pos[2]`, a `*i32` does not convert to a `*Pos`, and a `var Pos[]` does not convert to a `var i32[]`. An array literal is a fresh value, so it lands element by element, and each element is checked against the refinement (**[E086]**).

```
type Pos = i32 >= 1 and <= 9

func f(a Pos[2]) i32 { return 100 / a[0] }

func main() i32 {
    var z i32[2] = [0, 0]
    return f(z)     // ERROR [E012]
}
```

```
// VERIFY: exit 25
type Pos = i32 >= 1 and <= 9

func f(a Pos[2]) i32 { return 100 / a[0] }

func main() i32 {
    if f([4, 5]) != 25 { return 1 }
    return 25
}
```

7.2.4 Packed structs

The **[packed]** attribute (§ 10.1) on a struct declaration enables bit-exact memory layout. Each `iN`/`uN` field occupies exactly N bits within a single scalar container.

```
[packed]
type GpioModer {
    pin0 u2      // bit 0-1
    pin1 u2      // bit 2-3
    pin2 u2      // bit 4-5
    pin3 u2      // bit 6-7
}
// total: 8 bits -> uint8_t container, bit-exact layout
```

Layout rules:

- Fields are laid out in declaration order, starting at bit 0.
- Each field occupies exactly its declared bit width.
- Total bit width must be ≤ 64; container is the smallest of `uint8_t`, `uint16_t`, `uint32_t`, or `uint64_t` that fits the total.

Access semantics:

- Construction: `GpioModer(1, 2, 3, 0)` packs the four 2-bit fields into the container.
- Read: `r.pin1` extracts bits 2-3 via shift-and-mask; an `iN` field is sign-extended.
- Write: `r.pin1 = v` updates bits 2-3, leaving other fields unchanged.
- The whole struct is the container integer: it is copied, passed and returned as one.

> **Constraint**
>
> Inside a **[packed]** struct, fields shall be `iN`/`uN` types only. Nested structs, pointers, and slices are not permitted. Violation is *ill-formed* (**[E121]**).

> **Constraint**
>
> The total bit width of fields in a **[packed]** struct shall not exceed 64. Otherwise *ill-formed* (**[E121]**).

> **Constraint**
>
> A field of a **[packed]** struct has no address. A `var` reference to one (`f(var r.pin1)`, `var x = var r.pin1`) or its address (`&r.pin1`) is *ill-formed* (**[E121]**), inside `unsafe` as well:

this is a property of the representation, not a safety policy. A `var` reference to the whole struct is well-formed.

Note

This feature targets embedded use cases: hardware registers, packed network protocols, codec implementations. The reference implementation emits the container as a scalar typedef and every access inline: shift-and-mask reads, read-modify-write stores, and construction by OR-ing the shifted fields. The layout can be asserted where the type is declared: `assert @sizeof(GpioModer)== 1` (§ 10.8).

7.2.5 Integer rank and widening

13 The integer types are partially ordered by *rank*:

Rank	Types	Bit width
1	`u8`, `i8`	8
2	`u16`, `i16`	16
3	`u32`, `i32`, `int`	32
4	`u64`, `i64`	64
5	`usize`, `isize`	platform

14 **Implicit widening**: a value of a lower-rank integer type may be used where a higher-rank integer type is expected, without an explicit cast.

15 **Result type of arithmetic (auto-widening).** The result type of a plain `+`, `-`, or `*` is *wide enough to hold every representable result*, so the operation itself cannot overflow: `+`/`-` widen by one bit (with a sign bit when an unsigned operand feeds a signed result) and `*` sums the operand widths — e.g. `u8 + u8` yields `u9`, `u8 - u8` yields `i10`, `i32 + i32` yields `i33`, and `i32 * i32` yields `i64`. When the required width would exceed 64 bits (the `i128` ceiling), the result type is the higher-rank operand and the operation is subject to the overflow diagnostic below. A *signed* `/` widens by one bit — `i32 / i32` yields `i33` — because exactly one quotient does not fit its operands' type, TYPE_MIN/−1; an unsigned `/` and every `%` keep the higher-rank operand type (a remainder is smaller in magnitude than its divisor). The other binary operators (`&`, `|`, `^`, `<<`, `>>`) yield the higher-rank operand type. Overflow therefore becomes possible only at a *narrowing* (§ 7.2.6).

Example

```
var a u8  = 42
var b int = a           // u8 -> int: implicit widening, OK
var c = a + a           // u8 + u8 -> u9 (holds 0..510; cannot
   overflow)
var d = c as u8         // narrowing back to u8 is checked (see
   below)
```

Constraint

An implicit conversion from an integer type to one that cannot represent every value of the source — a narrowing, or a signedness change — is *ill-formed* unless value-range analysis

proves it fits (§13). Such a conversion otherwise requires an explicit cast tier (`as`, `as?`, `as%`, or `as|`; §8.8).

7.2.6 Integer overflow

16 Because `+`, `-`, and `*` auto-widen (§7.2.5), overflow cannot occur at the operation. It can occur only when a value flows into a type that cannot hold it — a *narrowing* at an assignment, return, argument, or explicit cast. A narrowing whose value range is not proven to fit the target is *ill-formed* (`[E086]`); the programmer resolves it by proving the range, widening the target, or choosing an explicit overflow policy:

Policy	Form
proven (default)	`x as T` (narrows only if proven, else `[E086]`)
checked (`else`)	`x as? T`, `a +? b`, `a -? b`, `a *? b`
wrapping (modular)	`x as% T`, `a +% b`, `a -% b`, `a *% b`, `a /% b`
saturating (clamp)	`x as| T`, `a +| b`, `a -| b`, `a *| b`, `a /| b`

17 A checked form performs a run-time test and, on overflow, evaluates its `else` arm — a fallback value, a `return`, or `panic(...)`, which terminates the program; a checked form without `else` is *ill-formed* (`[E067]`). A wrapping form uses two's-complement modular arithmetic. A saturating form clamps to the target type's minimum or maximum. Every form is defined at every integer width, odd widths included: a wrapping result is reduced modulo 2^N for the type's own N, not for the width of the machine type that holds it. The checked, wrapping and saturating *operators* compute in one integer type, chosen from their operands and never wider than the wider stated one:

- an integer literal, negated or not, and plain arithmetic (`+`, `-`, `*`, `/`, whose type the compiler widened so that it cannot overflow) take the other operand's type, on either side, and the policy applies to their exact value: `100 +% x` is `x +% 100`, and `s +% x * y` over `i32`s wraps in `i32`. The exact value is not the same as the value reduced to the chosen type first, and the difference is observable wherever a literal does not fit it: on a `u8`, `x +| 300` saturates the exact 310 and yields 255, where reducing 300 to a `u8` first would yield 54. For `+%` the two readings coincide — both yield 54 — so a reader who assumes the reduction is wrong only about saturation, and only sometimes;
- two literals, negated or not, state no type between them: the operation computes in the type of the place its result goes, so `x u8 = 250 +% 10` is 4, and where the result goes nowhere typed (`var y = 250 +% 10`, a variadic argument) in the literals' own type, `i32`, or `i64` for a literal outside it;
- of two stated types that differ, the one that holds every value of the other, in either order: `a +| b` and `b +| a` with a `u8` and a `u32` are both a `u32` saturating add;
- two stated types neither of which holds every value of the other (`u32` and `i32`, `i32` and `u64`) are *ill-formed* (`[E012]`): convert one.

```
// VERIFY: exit 0
func sat(x u8) u8 { return x +| 300 }      // the exact 310, clamped
func wrap(x u8) u8 { return x +% 300 }     // 310 mod 256

func main() i32 {
```

```
    if sat(10) as i32 != 255 { return 1 }
    if wrap(10) as i32 != 54 { return 2 }
    return 0
}
```

The amount of a wrapping shift `x <<% k` is a count, not an operand, and may have any integer type. For division the only overflowing case is TYPE_MIN/−1: `/%` yields TYPE_MIN and `/|` yields TYPE_MAX; every other quotient is the ordinary one.

Constraint

Where no wider type exists — a signed `/` or `%` on 64-bit operands — the operation itself must be proven not to divide TYPE_MIN by −1: the dividend's range excludes TYPE_MIN or the divisor's excludes −1. Otherwise the program is *ill-formed* (**[E086]**). There is no implicit run-time check; `/%` and `/|` state what the programmer means.

Constraint

An implicit conversion between integer types and floating-point types is *ill-formed*. Such conversions require an explicit `as` cast.

Constraint

An implicit conversion between integer types and `bool` is *ill-formed*. Such conversions require an explicit `as` cast.

7.2.7 Floating-point types

`f32` and `f64` are 32-bit and 64-bit floating-point types conforming to IEC 60559 (IEEE 754). They correspond to C's `float` and `double` respectively. `float` is a documented alias of `f32`. The default type of a floating-point literal is `f64` (§5.7.2). Arithmetic on `f32` and `f64` follows IEEE 754 semantics: overflow produces ±∞; the result of 0.0/0.0 and similar operations is NaN. These are defined behaviors in Lain. An `f32` converts to `f64` implicitly, because every `f32` is exactly an `f64`. The other way rounds, so it is written: `d as f32` gives the nearest `f32` (IEEE 754 rounding), ±∞ beyond its range, and a NaN for a NaN, and it owes no proof. A float literal, negated or not, takes the type of an `f32` destination; in arithmetic a float literal takes the other operand's type, and `f32` with `f64` computes in `f64`. So `x f32 = 0.5 * 3.0`, two literals, computes in `f64` and needs `as f32`.

Constraint

An `f64` value used where an `f32` is expected, without `as f32`, is *ill-formed* (**[E012]**). `as|`, `as?` and `as%` say what happens at an integer type's ends and do not take a float target (**[E012]**).

```
// VERIFY: exit 0
func main() i32 {
    d f64 = 0.1
    s f32 = d as f32          // rounds: written
    back f64 = s              // exact: implicit
    h f32 = 1.5               // a literal takes the f32
        destination
    if back == d { return 1 } // 0.1 has no exact f32
    if h * 2.0 != 3.0 { return 2 }
    return 0
}
```

```
func narrow(d f64) f32 {
    x f32 = d                    // ERROR [E012]: rounds the value, write `as f32`
    return x
}
```

7.2.8 Boolean type

22 `bool` has exactly two values: `true` and `false`. Every boolean the language makes has this type: the literals, the comparisons, `in`, and `and`, `or` and `!` (§ 8.5); none of them is an integer. `bool` is stored as a single byte; its exact representation is *implementation-defined*[3].

> **Constraint**
>
> There are no implicit **value** conversions between `bool` and any integer type for general arithmetic or assignment.

23 The condition expression of `if` and `while` accepts either a value of type `bool` or any integer type. When an integer value is used in condition position, zero is treated as `false` and any non-zero value is treated as `true`. This relaxation is ergonomic only and does not extend to other contexts.

> **Example**
>
> ```
> func f(n i32) i32 {
>     if n { return 1 }        // OK: integer in condition, non-zero is true
>     var b bool = n           // ERROR [E012]: not a value conversion; write n != 0
>     return 0
> }
> ```

7.2.9 The void type

24 `void` represents the absence of a value. It is used exclusively in pointer types (`*void`) for C interoperability (see § 17).

> **Constraint**
>
> A variable, parameter or field of type `void` is *ill-formed* (`[E012]`).

7.3 Array types

1 An array type `T[N]` denotes a contiguous sequence of N values of type `T`. N shall be a compile-time constant integer greater than zero (grammar: *array-type*).

2 The `.len` property of an array type yields N as a compile-time constant of type `int`.

3 In a local declaration, `N` may be any constant expression, a module constant (`var a u8[K]`) as well as a literal; the local is then a fixed array of `N` elements, and an initializer shall have exactly that length (`[E012]`). A length that is not a constant expression is *ill-formed* in a declaration with an initializer (`[E137]`): without one, the local is a VLA (the note below). A type alias may name an array type, `type Buf4 = u8[K]`, and is that array type.

[3] Implementation-defined behavior: whether `true` is 1 or any non-zero value

> **Note**
>
> **Runtime-length local arrays (VLAs).** A *local* declaration `var a T[n]` whose length `n` is a runtime value and which has no initializer is an owned, runtime-sized stack array (emitted as a C variable-length array `T a[n]`), with `a.len == n` and native bounds-checked indexing. It is distinct from a sized-slice *view* of the same spelling: a parameter `f(a T[n])`, or an *initialized* local `var s = arr[i..j]`, is a borrowed `{len, data}` slice, not owned storage. Indexing a VLA under a proven guard (`i < n`) needs no `unsafe`; an index VRA cannot bound is `[E085]` as for any array.

4 Array elements are laid out contiguously in memory in increasing index order, with element 0 at the base address. The stride between elements is *implementation-defined*[4].

> **Constraint**
>
> An array size $N \leq 0$ is *ill-formed* (`[E100]`).

> **Constraint**
>
> Every array index expression shall satisfy $0 \leq idx < arr.len$. An access for which value range analysis cannot prove this is an out-of-bounds violation (`[E085]`).

> **Example**

```
var arr int[5] = [1, 2, 3, 4, 5]
var x = arr[2]          // OK: 2 is statically in [0, 4]
// var y = arr[5]       // [E085]: 5 >= arr.len
```

7.4 Slice types

1 A slice type `T[]` is a reference to a contiguous sequence of values of type `T` whose length is determined at runtime. Its runtime representation is a *fat* pair, a data pointer and a length (`struct { T* data; size_t len; }`); as a parameter it is passed as two arguments, the length first (§17.5).

> **Constraint**
>
> **The `*` rule.** A `*` prefix denotes a *raw pointer*, and only that. Arrays and slices are always passed by reference, so they never carry a `*`: a slice is `T[]`, a sized slice `T[n]`, a fixed array `T[N]`, a sentinel slice `T[:0]`. A `*` is permitted only before a non-array, non-slice type — `*T`, `**T`, `*void`, and function-pointer types (§17). A `*` before any bracket form is *ill-formed* (`[E100]`).

2 The reference/pointer type family and its C mapping (parameter position):

Lain	What it is	C parameter	Cost
`u8`	scalar	`uint8_t` (by value)	1 word
`T[N]`	inline fixed array	`T*`, its first element	1 word
`T[]`	fat slice	`size_t, T*`	2 words
`T[n]`	sized slice	`size_t, T*`	2 words
`T[:0]`	slice followed by a 0	`size_t, T*`, as `T[]`	2 words
`*T`	raw pointer	`T*`	1 word
`*void`	opaque C pointer	`void*`	1 word

[4] Implementation-defined behavior: equal to `sizeof(T)` in C99, including any intra-element padding

> **Note**
>
> A `*` in the emitted C has *two* independent sources: (1) a `*` written in the Lain type, which is a raw pointer; and (2) the implicit by-reference passing of an *aggregate* (array, slice, struct) parameter — a hidden `T*` that is part of the calling convention, not the type. The same aggregate is a plain value as a local (`var p Point` emits `Point p`) but a pointer as a parameter (`Point*`). The sentinel of `T[:0]` is not in its representation either: it is a promise that a 0 follows the last element, which a string literal's C string keeps. This is why an aggregate type carries no `*` even though it is passed as a pointer: the `*` names raw pointers, not the borrow.

3 A slice has two properties accessible via dot notation:

- `.len`: the number of elements, type `usize`;
- `.data`: a pointer to the first element, type `*T`.

4 A string literal of N characters has static type `+1Fixed_u8_N` (a null-terminated fixed-size array of $N+1$ bytes) and is implicitly coercible to `u8[]`.

7.4.1 Sized slice types

5 A *sized slice type* is a slice type annotated with a size constraint on its length. The constraint is part of the parameter or variable declaration, not of the nominal type itself. The syntax is:

Syntax	Meaning
`T[]`	unsized dynamic slice (no length constraint)
`T[n]`	length exactly `n`; n is a size expression
`T[>= n]`	length $\geq$ `n`; n is a size expression
`T[> n]`	length $>$ `n`; n is a size expression

6 A *size expression* (*size_expr*) may be any of:

- an integer literal (e.g., `3`, `0`);
- an identifier that names a numeric variable in scope (e.g., `n`);
- a member expression of the form `p.len`, where `p` is a slice or array parameter in scope (e.g., `a.len`, `src.len`);
- a binary arithmetic expression over the above (e.g., `a.len + b.len`, `src.len - 1`).

> **Example**

```
// Exact-length constraint: out must have length == a.len +
    b.len
// (the bounds keep a.len + b.len from overflowing a usize)
func concat(a i32[<= 1000000], b i32[<= 1000000], out i32[a.len
    + b.len]) {
    for j in 0..a.len { out[j] = a[j] }
    for j in 0..b.len { out[j + a.len] = b[j] }
}

// Lower-bound constraint: arr must have at least one element
func first(arr i32[> 0]) i32 { return arr[0] }

```

```
// Derived constraint using member .len (src is not empty, so
    src.len - 1 cannot underflow)
func slide(src i32[>= 1], out i32[src.len - 1]) {
    for i in 0..out.len { out[i] = src[i + 1] -% src[i] }
}
```

Constraint semantics.

- `T[expr]`: the value range analysis (VRA, §13) records the constraint `len == expr` and uses it to prove subsequent bounds checks inside the function body.
- `T[>= expr]`: VRA records `len >= expr`.
- `T[> expr]`: VRA records `len > expr`, i.e. `len >= expr + 1`.

Call-site verification. At every call site, the compiler verifies that the argument satisfies the formal parameter's size constraint (`[E087]`); see §13.12.

C ABI decomposition (Fase 7). When a function parameter has a sized or unsized dynamic slice type, the compiler decomposes it into a pair of C parameters:

- a length parameter `size_t`;
- a data pointer `T* restrict`.

Whether the callee may write through the pointer is stated by the GCC attribute `access(read_only, p, n)` or `access(read_write, p, n)`, where p and n are the positions of the pointer and its length. A slice whose length is an expression over other parameters' lengths (`out i32[a.len + b.len]`) is passed the same way: its length is an ordinary parameter, checked against the expression at every call (§13.12).

Example

```
// Source:
func vadd(var out i32[], a i32[out.len], b i32[out.len]) {
    for i in 0..out.len { out[i] = a[i] +% b[i] }
}

// C emission (the name is prefixed with its module's):
// __attribute__((access(read_write, 2, 1)))
    __attribute__((access(read_only, 4, 3)))
// __attribute__((access(read_only, 6, 5)))
// void vadd(size_t, int32_t* restrict, size_t, int32_t*
    restrict,
//           size_t, int32_t* restrict);
```

7.5 Vector types

`Vec(N, T)` is a value of `N` *lanes* of type `T`, held and passed whole. `TxN` is the same type: `u8x16` is `Vec(16, u8)`, `i32x4` is `Vec(4, i32)`. A vector may be a local, a parameter, a return value or a struct field.

Constraint

`N` shall be a power of two from 1 to 64, and `T` an 8, 16, 32 or 64-bit integer type, or `f32` or `f64` (`[E100]`).

2 A vector is built from an array literal of exactly `N` elements where a vector is expected: a declaration, an argument, a field, a return, or an operand beside a vector (`a - [1, 1, 1, 1]`). `@splat(V, x)` is the vector of type `V` with `x` in every lane.

3 The arithmetic, bitwise and shift operators and the comparisons apply lane by lane, and a scalar operand applies to every lane. Integer lanes *wrap*: a lane's result is taken modulo 2^w for a w-bit lane, including unary `-`, so vector arithmetic carries no overflow obligation. Two obligations remain, as on scalars:

- a division or remainder's divisor shall be provably non-zero (**[E015]**); a scalar divisor is judged like a scalar's, while a vector divisor cannot be proven, because the analysis holds no per-lane facts;
- a shift's amount shall be a scalar provably in $[0, w-1]$ (**[E086]**); an amount given per lane is refused for the same reason.

4 A comparison yields a *mask*: every bit of a lane is set where the comparison holds and clear where it does not (255 and 0 for `u8` lanes). For integer lanes the mask has the operands' vector type; for `f32` and `f64` lanes it has `i32` and `i64` lanes, so a lane that held is -1 (a float with every bit set would be a NaN, not a mask).

5 `v[k]` is lane `k` of `v`, read or written in place. `k` shall be provably less than `N` (**[E085]**).

6 `@load(V, b, i)` reads the `N` elements of `b` starting at element `i` into a vector of type `V`, and `@store(b, i, v)` writes `v`'s lanes there. For an array or slice `b` the obligation is `i + N <= b.len` (**[E085]**). A raw pointer has no length, so a load or store through one is allowed only inside `unsafe`. `b` shall be an array, a slice or a raw pointer (**[E100]**): a struct's bytes are in an order the implementation chooses (§ 7.6).

7 `@movemask(v)` is the `u32` whose bit `k` is the top bit of lane `k`: of a mask, the lanes where the comparison held.

> **Constraint**
>
> The operand of `@movemask` shall be a vector of at most 32 lanes of 8 bits (**[E100]**).

8 `@popcount(x)`, `@ctz(x)` and `@clz(x)` take an integer and give a `u32`: the number of bits of `x` that are set, the number of zero bits below its lowest set bit, and the number above its highest, counted in `x`'s width. With `@movemask` they scan a mask: `@ctz` of a mask is its first matching lane.

> **Constraint**
>
> The argument of `@ctz` and of `@clz` shall be provably non-zero, as a divisor shall: from its type, its range or a guard (**[E015]**). Neither count is defined for 0, which is the mask of a comparison that matched no lane.

```
func first_match(bits u32) u32 {
    if bits == 0 { return 32 }      // no lane matched
    return @ctz(bits)
}
```

9 `@shuffle(t, idx)` is a table lookup: lane `k` of the result is `t[idx[k]]` when `idx[k] < N`, and 0 otherwise. It is total: an index out of range reads 0, never another lane and never an undefined value, so it states no obligation. This is the behaviour of ARM NEON's `tbl` and of Rust's `swizzle_dyn`; on x86 with SSSE3 it is `pshufb`. The classifier idiom `@shuffle(tbl, (x >> 4)& 15)` indexes a 16-entry table by each byte's high nibble.

> **Constraint**
>
> `t` shall be a vector of integer lanes and `idx` a vector of as many lanes of the unsigned integer type of the same width: `u8x16` for a `u8x16` or `i8x16` table, `u32x4` for a `u32x4` table (**`[E100]`**).

10 The language names no instruction set. A vector is translated to a C `vector_size` type; the C compiler's target flags choose the instructions, and the same program builds for x86-64 and AArch64. `@movemask` uses the SSE2 or AVX2 intrinsic where the target defines it and a portable loop elsewhere; `@shuffle` uses `__builtin_shuffle` under GCC and a lane loop elsewhere.

7.6 Struct types

1 A struct type groups named fields into a single named product type. The grammar production is *struct-declaration* (Annex A).

2 A struct's fields are constructed and accessed by name; the order in which they are *stored* is the implementation's. It stores them by decreasing alignment, fields of equal alignment in declaration order, so that the padding between them is the least the alignments allow. The memory layout is then *implementation-defined*[5].

```
type TokenKind { Ident, Number, Punct }
type Token {
    kind TokenKind      // one byte, stored last
    pos  u32
    len  u16
}
assert @sizeof(Token) == 8      // 12 in declaration order
```

3 A struct whose layout is read outside the program keeps its declaration order:

- one declared **`[ordered]`** (§10.1) — a file format, a wire header;
- one declared **`[packed]`** (§7.2.4), whose bit order is the declaration;
- one reachable from the signature of an **`extern`** function — by value, or through a pointer, an array, a slice, a vector, a function pointer or a field of such a struct — since the foreign code was written against the declaration.

A tagged sum's payloads (§7.8) are stored the same way, each by decreasing alignment, unless the sum is declared **`[ordered]`** or is reachable from an **`extern`** signature; either keeps every payload in declaration order. Inside a program the storage order is not observable: there are no stored references, field addresses cannot be compared or converted to integers, `@load` and `@store` take an array, a slice or a raw pointer, never a struct (§7.5), and a cast that would view a reorderable struct's bytes is refused (below). Only the size is observable, through `@sizeof`.

> **Constraint**
>
> A cast between a pointer to a struct whose storage order the compiler chooses — one not declared **`[ordered]`** or **`[packed]`**, not reachable from an **`extern`** signature, and of more than one field — and any other type (another pointer, an integer) is *ill-formed* (**`[E012]`**), in either direction and inside **`unsafe`** as well: **`unsafe`** permits the cast, not an unspecified layout. So is such a cast over an array of that struct, or over a struct that holds it by value. Viewing a struct's bytes needs a declared layout: **`[ordered]`**. A cast between pointers to

[5] Implementation-defined behavior: the C99 layout of the fields in that order, including compiler-inserted padding for alignment

the same struct type is not a view. The same holds for a tagged sum with a payload of more than one field whose order the compiler chooses: viewing its bytes needs `[ordered]` on the sum. A sum whose payloads have at most one field each has no order to expose.

4 A struct type S is linear if at least one field of S has a linear type (transitively).

Constraint

Q-008. Every field of a struct or enum variant whose declared type is linear shall be annotated `mov`. Omitting `mov` on a linear field is *ill-formed* (`[E083]`). This requirement makes linearity locally visible in the declaration: a reader of a struct definition does not need to inspect the field's type definition to know whether the struct is linear.

Example

```
type Handle {
    mov ptr *int              // explicit `mov` on linear field
}

type Session {
    mov h Handle              // OK: Handle is linear, mov
        required
    name u8[]                 // unrestricted field, no mov needed
}

// Ill-formed: missing mov on a linear field
type Bad {
    h Handle                  // [E083]
}
```

5 **Struct construction**: all fields shall be provided in declaration order. The type of each argument shall be compatible with the declared field type (§ 7.12).

Constraint

A struct constructor expression with a number of arguments different from the number of fields is *ill-formed* (`[E100]`).

Constraint

Writing to a field of a struct variable requires the variable to be declared as `var` (mutable). Writing to an immutable binding's field is *ill-formed*.

Constraint

Using `==` or `!=` on a struct type is *ill-formed*. Field-by-field comparison must be performed explicitly.

7.6.1 Field invariants

6 A field declaration may state a fact about the field's value for the whole life of every value of the struct type. Three forms are provided:

- `pos usize in 0..text.len` — an *index* invariant: `pos` is a valid index into the slice or array field `text`, always below its length. A cursor that may rest one past the last element is `in 0..=text.len`;

- `pos usize <= src.len` — a *position* invariant: `pos` compared with the length of the slice or array field `src`, by `<`, `<=`, `>` or `>=`;
- `len usize <= cap` — a relation between two integer fields, by the same four comparisons;
- `src u8[<= 4294967295]` — a *length bound* on a slice field: its length compared with an integer constant, written inside the brackets as for a parameter (`a i32[>= 2]`), by `<`, `<=`, `>`, `>=` or `==`. The constant may be a literal, a module constant (`src u8[<= MAXU32]`) or an expression over them (`src u8[<= (1 << 32)- 1]`), evaluated as a module-scope `assert` is (§10.8); it is enforced exactly as the literal it evaluates to. A fixed-length field may be sized the same way: `data u8[BUF]` is `data u8[64]` when `BUF` is 64.

A refinement against a literal (`pct i32 <= 100`) is a plain field refinement (§7.11) and is not a field invariant.

7 A length bound is what a *narrow* position needs: `pos u32 <= src.len` does not make `pos + 1` fit a `u32` when `src.len` is a `usize`, since a longer source admits `pos` $= 2^{32} - 1$; bounding `src` to at most $2^{32} - 1$ bytes does. A slice field's length changes only when the field (or the whole struct) is written — a `var` reference cannot rebind a slice (`[E009]`) and an element write keeps the length — so those writes are where it is checked.

8 An index invariant cannot hold for an empty container. A position invariant can: `pos usize <= src.len` admits `pos == 0` over an empty source and `pos == src.len` at its end, which is the state a scanner is in at end of input.

Constraint

A field invariant shall hold wherever a value of the struct type is built, and after every write to either field it relates. A construction or a field write at which it is not proven is *ill-formed* (`[E121]`).

Constraint

A `var` reference to a field that a field invariant relates — a `var` argument (`f(var s.pos)`) or a reference binding (`var r = var s.pos`) — is *ill-formed* (`[E121]`): a write through it would not be checked against the invariant. Pass the whole struct as `var` instead; writes to its fields are checked.

Constraint

A field refinement naming another field in any other shape — another comparison, a field of another kind, an expression — is *ill-formed* (`[E132]`). So is a field whose length is bounded or sized by anything but a constant (other than the struct's own generic parameter, `type Buf(N usize){ data u8[N] }`), and a refinement written after a slice type (`src u8[] <= 4096`), which bounds nothing a slice has.

9 Because every write that could break a field invariant is checked, the invariant is a fact at every read of the field: `c.text[c.pos]` needs no bounds check under `pos usize in 0..text.len`, and `b.cap - b.len` needs no underflow check under `len usize <= cap`.

Example

```
type Lexer {
    src u8[]
    pos usize <= src.len          // position: 0 <= pos <=
        src.len
}
func lexer_new(text u8[:0]) Lexer { return Lexer(text, 0) }
    // OK even for ""
```

```
func advance(l var Lexer) {
    if l.pos < l.src.len { l.pos = l.pos + 1 }
        // keeps the invariant
}
```

```
type Lexer {
    src u8[]
    pos usize <= src.len
}
func bad(l var Lexer) { l.pos = l.pos + 1 }   // ERROR: E121 (and E086), unguarded
```

7.7 Enum types

An enum type is a type with a finite, named set of values. Each variant carries no associated data. The grammar production is *enum-declaration* (Annex A).

An enum type is represented by an unsigned integer type whose values $0, 1, \ldots, n-1$ are its variants in declaration order. The integer is the smallest that holds the variants (§19.9) unless the declaration states a *backing width* between the type name and the body:

Example

```
type TokenKind u16 { Ident, Number, Punct }
assert @sizeof(TokenKind) == 2
```

A record that stores an enum has a layout that depends on the enum's width. With a backing width, adding a variant leaves that layout where it is, and a variant that no longer fits makes the declaration ill-formed instead of the enum silently wider.

Constraint

A backing width shall be `u8`, `u16`, `u32` or `u64`, and shall appear only on an enum none of whose variants carries a payload. A backing width `uN` holds at most 2^N variants. A violation is *ill-formed* (`[E100]`).

Example

```
type Sign i8 { Minus, Plus }              // ERROR: E100
```

```
type Shape u8 { Dot, Circle { r u32 } }  // ERROR: E100
```

Enum values are accessed as `TypeName.VariantName`.

Constraint

Using `==` or `!=` to compare enum values is *ill-formed*. Enum values shall be compared using `case` pattern matching (§15).

Note

The constraint on `==` for enums reflects that the underlying tag representation is implementation-defined. Pattern matching is always sound.

7.8 Algebraic data types (ADTs)

1 An ADT is a sum type in which each variant may carry associated fields. A type declaration is classified as an ADT if at least one variant has fields. The grammar production is *enum-declaration* (Annex A, using the record-variant form).

2 ADTs use a niche-based representation whenever a zero-cost encoding is available; otherwise they are a tagged union: a tag beside a union of the variant payloads (§ 19.9 gives the tag's width). A payload's fields are stored as a struct's are (§ 7.6), by decreasing alignment, unless the sum is `[ordered]` or reachable from an `extern` signature.

> **Note**
>
> **D-Niche aggressive multi-slot.** When the variant payload admits a *niche*—a bit pattern that cannot occur in any valid value of the payload type—that niche is used to encode tag-less variants. Only a sum with one payload-carrying variant, whose payload is a single field, can be encoded this way. The niche sources are the zero page for a pointer, the values outside its refinement for a constrained integer, the 254 byte patterns outside $\{0, 1\}$ for `bool`, the values above the last variant for a plain enum that declares its width (`type K u8 { A, B, C }` leaves 3 to 255), and the slots a packed sum leaves unused when it is itself the payload. A slice is not a niche source, nor is a plain enum without a declared width, whose width (and so whose spare values) would change with its variant count, nor an enum that the signature of an `extern` function reaches, since C may hand it any value of its width: a sum whose payload is one of them carries a tag. A value of an enumeration type is always one of its variants: no conversion produces one from an integer, and writing any other value into one is undefined (§ 18.5).
>
> Whenever a sum is given a tag, the compiler emits `[W120]` (a warning, never an error) with the reason. When the reason is the payload's spare values, the programmer may remove the tag by constraining the payload, by declaring a plain enum payload's width, or by choosing a payload type that has spare values; otherwise the tag is the cost of the shape. See § 19.9 for the full layout algorithm and its target dependencies.

3 ADT values are constructed as `TypeName.VariantName(args)` for variants with fields, or `TypeName.VariantName` for unit variants.

4 ADT values are destructured via `case` pattern matching (§ 15).

7.9 Union types

1 A *union type* `T | M1 | ... | Mn` holds either a value of its *value type* `T` or one of its *markers* `M1` to `Mn` (grammar: *type*). It is the one construct for a value that may be absent and for an operation that may fail: `*u8 | none` is a pointer or nothing, and `*u8 | NotFound | Denied` is a pointer or one of two errors. A marker is a name, and naming it in a union type declares it; it needs no other declaration. The marker `none` is the conventional name for absence, and is otherwise an ordinary marker.

2 A marker may carry fields: `i32 | ParseErr(line i32)` declares a marker `ParseErr` that carries an `i32`. It is written `ParseErr(42)`, and a `case` pattern `ParseErr(line)` binds its fields (§ 15).

3 A value of the value type converts implicitly to the union, wherever a value of the union is expected: returned from a function whose return type is the union, assigned to a variable of the union type, or passed to a parameter of it. A marker is written by its name, with its fields' values if it carries any.

> **Constraint**

A marker written where a union is expected shall be one of that union's markers; any other is *ill-formed* (`[E012]`).

```
// VERIFY: exit 0
func find(p *u8, ok bool) *u8 | NotFound {
    if ok { return p }              // a value converts
    return NotFound                 // a marker
}

func main() i32 {
    var r *u8 | NotFound = find("x", false)
    var got = 9
    case r {
        NotFound: got = 0
        else: got = 1
    }
    if got != 0 { return 1 }
    return 0
}
```

4 **Representation.** A union whose markers carry no fields costs nothing: it occupies exactly the storage of its value type, and each marker is stored as a bit pattern that no value of that type can have. The sources of such patterns are the ones a sum type uses (§ 7.8): the zero page of a pointer, the values a refinement excludes, the byte patterns of a `bool` other than 0 and 1, and the values above the last variant of a plain enum that declares its width. The full rules are in § 19.9.

Constraint

A union whose markers carry no fields, and whose value type has fewer spare bit patterns than the union has markers, is *ill-formed* (`[E064]`). A full-range integer has none: `i32 | none` is *ill-formed*, and a refined alias makes room.

```
// VERIFY: exit 0
type Small = u8 < 200

func main() i32 {
    if @sizeof(Small | NotFound | Denied) != 1 { return 1 }   // markers in values >= 200
    return 0
}
```

5 A union with a marker that carries fields cannot always be stored in its value type alone; it is then a tagged union, a tag beside the value or the marker's fields, and the compiler emits `[W120]` with the reason, as for any sum given a tag (§ 7.8).

6 **Testing a union.** A union value is not a value of its value type: it may hold a marker. A use of it as the value type, which reads it as a value, is permitted only where a test has excluded every marker. Two tests do so:

- `if u` in condition position is `true` exactly when `u` holds a value. Inside its true branch, `u` has the value type.
- A `case` on `u` (§ 15) has an arm for each marker, whose pattern is the marker's name (with a binding for each of its fields), and an `else:` arm, which matches the value. Inside the `else:` arm, `u` has the value type.

The operators of § 8.9 (`else` and `try`) are the third way to reach the value: each says what happens on a marker, and yields the value.

Constraint

A union value used as its value type where no test has excluded every marker is *ill-formed* (`[E063]`).

```
func first(x *u8 | none) *u8 {
    return x                        // ERROR [E063]: x may be none
}

func present(x *u8 | none, d *u8) *u8 {
    if x { return x }               // OK: here x holds a value
    return d
}
```

Constraint

The value of a union is matched only by `else:`. A `case` on a union without an `else:` arm is non-exhaustive, and *ill-formed* (`[E014]`).

```
// VERIFY: exit 0
func parse(x i32, bad bool) i32 | ParseErr(line i32) {
    if bad { return ParseErr(42) }
    return x
}

func main() i32 {
    var r i32 | ParseErr(line i32) = parse(7, true)
    case r {
        ParseErr(line):
            if line != 42 { return 1 }
            return 0
        else: return 2
    }
}
```

Note

What this implementation does. A local of a union type shall be declared with its type written out, as in the examples above: `var r = find("x", false)` is refused with `[E012]`.

7.10 Pointer types

A pointer type `*T` is a raw memory address the program may read through, and `*var T` one it may also write through. Reading or writing through a pointer requires an `unsafe` block (§ 18); making one does not.

The ownership modes for pointer types are:

Lain syntax	Mode	C99 type	Linear?
`*T`	read-only: a write through it is `[E009]`	`T*`	no
`*var T`	writable	`T*`	no
`mov *T`	owned	`T*`	yes

Constraint

Dereferencing a pointer shall occur only inside an `unsafe` block; pointer dereference in safe code is *ill-formed* (`[E060]`). A field access whose base is a raw pointer — `p.v`, `p.inner.v` — is a dereference too, though it is not spelled with `*`: outside `unsafe` a read through it is `[E060]`, and an assignment to such a field is `[E009]`. A field of a struct held by borrow is not a dereference and needs no `unsafe`. Taking the address of a variable is permitted in safe code: making a pointer is harmless, and only reading or writing through it is the danger.

7.11 Constrained types

A constrained type is a type annotated with a value range constraint:

$$T \ op \ bound$$

where *op* ∈ {`<`, `<=`, `>`, `>=`, `==`, `!=`}. What a *bound* may be depends on where the constraint is written:

Position	A bound is
parameter	a number, an integer module constant, another parameter of the function (`x i32 >= n`), or a sum or difference of these (`x i32 <= M - 1`)
return type	a number, an integer module constant, a parameter of the function (`usize <= n`), or a sum or difference of these
struct field	one number, integer module constant, or other field of the struct (`pos usize <= len`)
type alias	one number or integer module constant
variant field	one number

An integer module constant, typed or not, is folded to its value: with `MAX i32 = 100`, `i32 <= MAX` is the constraint `i32 <= 100`.

Constraint

A name in a bound that denotes none of these is *ill-formed*: `[E106]` when nothing of that name is declared, and `[E100]` when it names something else, such as a function, an array or a constant that is not an integer. In a struct field it is `[E132]`, since the bound would otherwise be silently ignored.

```
// VERIFY: exit 0
MAX i32 = 100
type Percent = i32 >= 0 and <= MAX

func main() i32 {
    var p Percent = 100              // proven: 100 <= MAX
    if p != 100 { return 1 }
    return 0
}
```

```
func f(x i32 <= LIMIT) i32 {         // ERROR [E106]: LIMIT is
    declared nowhere
    return x
}
```

3 Constrained types are used in function return types and parameter types to declare value range invariants that the value range analysis (§ 13) shall enforce.

Example

```
func nonzero_divisor() int >= 1 {
    return 42    // VRA verifies: 42 >= 1 holds
}
```

4 The constraint is part of the function signature, not the type system's nominal identity. Two constrained types with the same base type are assignment-compatible.

7.12 Type compatibility and equivalence

1 Lain uses **nominal type equivalence**: two types are the same if and only if they have the same declared name.

2 Two types T_1 and T_2 are *compatible* for an expression context if:

- $T_1 = T_2$ (same nominal type), or
- T_1 is an integer type of lower rank than T_2 (implicit widening; § 7.2.5).

Constraint

An expression of type T_1 used in a context requiring type T_2, where T_1 and T_2 are not compatible, is *ill-formed*.

3 The operators `==` and `!=` are defined for:

- primitive numeric types;
- `bool`;
- pointer types.

They are not defined for struct types, array types, or enum types.

Chapter 8

Expressions

1 An expression is a syntactic construct that evaluates to a value. Every expression has a type, determined statically at compile time. The grammar production for *expression* is in Annex A.
2 Expressions are evaluated strictly left-to-right: for any binary operator *a op b* or function call `f(a, b)`, the left operand (or earlier argument) is fully evaluated before the right operand (or later argument).

8.1 Operator precedence and associativity

1 The following table lists all binary operators in order of decreasing precedence. All operators listed on the same row have equal precedence.

Prec.	Operators	Assoc.	Result type
1 (high)	`() [] .`	left	(context-dependent)
2	unary `- ! ~` `mov` `var` `* & try`	right	operand type; `*` yields the pointee, `&` a pointer to its ope
3	`as`	left	target type
4	`* / %` and their `%/|/?` forms	left	§7.2.5
5	`+ -` and their `%/|/?` forms	left	§7.2.5
6	`<< >>`	left	left operand type
7	`&`	left	wider operand type
8	`^`	left	wider operand type
9	`|`	left	wider operand type
10	`< > <= >= in`	left	`bool`
11	`== !=`	left	`bool`
12	`and`	left	`bool`
13	`or`	left	`bool`
14 (low)	`else` (the handler, §8.9)	left	the union's value type

2 Parentheses override precedence.

8.2 Primary expressions

1 A primary expression is one of: an identifier, a literal, a parenthesized expression, an array literal or comprehension, a `case` expression, or a builtin (`@sizeof`, `@load`; *expression*). Constructing a struct or a variant is a call (§8.15). A block expression, an `if` expression and a `comptime` expression are planned, and not yet accepted (`[E100]`): their sections below say so.
2 **Identifier expression**: evaluates to the current value of the named variable.

> **Constraint**
>
> The identifier shall be declared in scope; use of an undeclared identifier is *ill-formed* (`[E106]`, §6.3).

> **Constraint**
>
> The identifier shall have been initialized before use (`[E005]`).

8.3 Arithmetic operators

1 The binary arithmetic operators are `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), and `%` (remainder). Their result types are given in §7.2.5: `+`, `-`, `*` and a signed `/` widen so the operation cannot overflow; the forms with an explicit policy (`+%`, `/|`, …, §7.2.6) keep the operand type.

2 Integer division truncates toward zero. `f32`/`f64` division follows IEEE 754.

> **Constraint**
>
> Integer division or remainder where the divisor is not provably non-zero (by value range analysis, a `!= 0` refinement, or a dominating guard; §13) is *ill-formed* (`[E015]`).

> **Constraint**
>
> `%` shall not be applied to floating-point operands.

3 The unary `-` operator negates its operand, in the operand's type. On an unsigned type only 0 has a negation, so `-x` is *ill-formed* (`[E086]`) unless `x` is proven to be 0; the wrapping negation, modulo 2^N, is written `0 -% x`.

4 The unary `*` operator reads through a pointer: `*p` is the value `p` points to, and `*p = v` writes it. The unary `&` operator yields a pointer to its operand.

> **Constraint**
>
> The operand of unary `*` shall have a pointer type; otherwise the program is *ill-formed* (`[E012]`). This holds inside `unsafe` as it does outside: `unsafe` permits reading through a pointer, not reading through a value that is not one.

```
func main() i32 {
    unsafe {
        var x = *true       // ERROR [E012]: the operand is 'bool'
    }
    return 0
}
```

8.4 Comparison operators

1 The comparison operators `==`, `!=`, `<`, `>`, `<=`, `>=` compare two values and yield `bool`.

> **Constraint**
>
> Operands of a comparison operator shall have compatible types (§7.12).

Constraint

`==` and `!=` shall not be applied to a struct, an array, an enum (§7.7), an ADT or a union, nor to a slice except against a string literal, which compares bytes and lengths (`[E012]`).

8.5 Logical operators

`and` and `or` are short-circuit operators: the right operand is not evaluated if the result is determined by the left operand alone. `!` is prefix unary negation. All yield `bool`.

Constraint

Operands of `and`, `or`, and `!` shall have type `bool`.

8.6 Bitwise operators

The bitwise operators `&`, `|`, `^`, `<<`, `>>` operate on integer operands. The result type is the wider operand type for `&`, `|`, `^`; for shift operators, the result type is the left operand type. A *literal* shifted left by a constant expression is typed by its value, as a constant sum is: the literal and the shift take the narrowest of its type, `i64` and `u64` that holds the result, so `1 << 40` is an `i64` holding 2^{40}, in a module assertion and in code alike.

Constraint

Operands of bitwise operators shall be integer types.

Constraint

The shift amount (right operand of `<<` and `>>`) shall satisfy $0 \le \mathit{shift} < N$ where N is the bit width of the left operand. A shift amount that value range analysis cannot prove to be in range is *ill-formed* (`[E086]`) — the boundary mechanism (§13.10) that also rejects arithmetic overflow.

Constraint

`x << k` is $x \cdot 2^k$, and like `*` it shall not overflow: a left shift whose result value range analysis cannot prove representable in the left operand's type is *ill-formed* (`[E086]`), for an unsigned operand as for a signed one. No bit is silently shifted out.

`x >> k` is $\lfloor x/2^k \rfloor$, rounded toward negative infinity, for every integer type. On an unsigned operand the bits shifted out are discarded; on a signed operand the sign bit is copied in, so `-7 >> 1` is -4, where `-7 / 2` is -3 because division truncates toward zero. A right shift cannot overflow, and owes nothing beyond its shift amount.

Example

```
// VERIFY: exit 0
func half_down(x i32) i32 {
    return x >> 1
}

func main() i32 {
    if half_down(-7) != -4 { return 1 }     // rounds down
    if half_down(7) != 3 { return 2 }
    if -7 / 2 != -3 { return 3 }            // division rounds
        toward zero
    return 0
}
```

> **Note**
> ISO C leaves the right shift of a negative value implementation-defined, in C11 and still in C23. The reference implementation emits C's » on the signed type and relies on GCC's documented behaviour for it, an arithmetic shift, which Clang shares.

3 The *wrapping* left shift `x <<% k` discards the bits shifted out, as `+%`, `-%` and `*%` wrap: its result is $x \cdot 2^k$ modulo 2^N, read in the left operand's type. The shift amount obeys the same constraint as for `<<`; an amount outside $0 \le k < N$ is not a wrap.

> **Example**

```
func mix(h u32, c u8) u32 {
    return (h <<% 5) +% h +% (c as u32)      // a hash step: the
        high bits are meant to go
}
```

8.7 The `in` operator

1 `in` tests **membership in a set**, and the set is written as a range or as a list of constants. `x in lo..hi` is `true` if and only if $lo \le x < hi$, and `x in lo..=hi` if and only if $lo \le x \le hi$; each bound is evaluated once. All three operands shall have integer types.

2 `x in [`$e_1, \ldots, e_n$`]` is `true` if and only if x equals some e_i. x is evaluated once and compared with each element in turn, as integers — mathematically, not after conversion to a common C type, so a `u8` holding 44 does not equal 300. Each element shall be a *closed constant*: an integer or character literal, or unary and binary arithmetic on such literals; a name is not followed. On the edge where the test is true, the analysis narrows x to the *hull* of the elements, $\min e_i \le x \le \max e_i$ — the test itself is exact, but the fact recorded is the interval. Membership in the elements of an array or slice is not an `in` expression; it is a scan.

3 The indices of an array or slice *arr* are the range `0..arr.len`, so an index test is written `idx in 0..arr.len`. The form `idx in arr`, naming the container, was retired: `arr` is a sequence of elements and not a set of indices, and the implementation refuses it with the range to write instead (`[E100]`). An index refinement on a field or a parameter is written the same way — `in 0..arr.len` for an index, `in 0..=arr.len` for a cursor that may rest at the end.

4 When `idx in 0..arr.len` appears as the condition of a `while` or `if` statement (or as the leftmost operand of an `and`-chain condition), it establishes an **in guard**: inside the body of that statement, the implementation shall accept `arr[idx]` without requiring further bounds verification.

5 In-guards propagate through `and`-chains: if the left operand of `and` establishes an in-guard, that guard is active when evaluating the right operand.

> **Example**

```
// In-guard: arr[i] is safe without further proof
func sum(arr i32[]) i32 {
    var s i32 = 0
    var i usize = 0
    while i in 0..arr.len {
        s = s +% arr[i]    // OK: in-guarded
        i = i + 1
    }
    return s
```

```
}

// And-chain propagation:
type Lexer {
    src u8[]
    pos usize
}
func skip_to_quote(l var Lexer) {
    while l.pos in 0..l.src.len and (l.src[l.pos] as int) !=
        '"' {
        l.pos = l.pos + 1
    }
}
```

Constraint

An element of a membership list `x in [...]` that is not a closed constant is *ill-formed* (**[E012]**), as is a left operand that is not an integer. An empty list is *ill-formed* (**[E100]**).

Constraint

An in-guard of the form `idx in 0..arr.len` covers only `arr[idx]` exactly. Accesses `arr[idx + k]` for any $k \neq 0$ are not covered by this guard and require separate verification or an `unsafe` block.

8.8 Cast expression (as)

1 The expression `expr as T` explicitly converts the value of *expr* to type `T`. An integer cast whose target cannot hold every value of the source — a narrowing, or a signedness change such as `u32` to `i32` or `usize` to `i64` — carries an *overflow policy* chosen by the cast tier (§ 7.2.6):

Tier	Form	On a value outside `T`
proven	`as`	narrows only if VRA proves it fits, else **[E086]**
checked	`as?`	run-time bounds check; the `else` arm on overflow
wrapping	`as%`	two's-complement modular truncation
saturating	`as|`	clamp to the minimum or maximum of `T`

2 The following casts are permitted:

From	To	Semantics
integer	integer	The same value where `T` holds it; otherwise the tier's policy
integer	float	Nearest representable value (IEEE 754)
float	integer	Truncation toward zero; `as` owes a proof that it fits (below)
float	float	`f32` to `f64` exact; `f64` to `f32` rounds to nearest, $\pm\infty$ beyond its range
pointer	pointer	Reinterpretation (`unsafe` required)
integer	pointer	Reinterpretation (`unsafe` required)
pointer	integer	Reinterpretation (`unsafe` required)

Constraint

A cast `x as T` from a floating-point type to an integer type `T` is *ill-formed* (**[E086]**) unless `x` is proven to be a *number*, not a NaN, whose integral part fits `T`: $T_{\min} - 1 < x < T_{\max} + 1$. The proof comes from a float constant, from a float local stored once whose address does not leave the function, or from ordered comparisons that guard the cast on their *true* edges. A comparison's false edge does not prove it: a NaN fails every ordered comparison, so `if x < 0.0 or x >= 256.0 { return 0 }` lets a NaN reach the cast that follows. The value is truncated toward zero, and the comparisons that guard it bound the result.

From a float, `x as| T` clamps: a NaN gives 0, a value below $T_{\min}$ gives $T_{\min}$, and one above $T_{\max}$ gives $T_{\max}$. `x as? T else e` takes its `else` arm for a NaN or a value out of range. `as%` wraps an integer and does not apply to a float operand (**[E012]**).

```
// VERIFY: exit 0
func byte_of(x f64) u8 {
    if x >= 0.0 and x < 256.0 { return x as u8 }      // proven: a number in [0, 256)
    return 0
}

func main() i32 {
    if byte_of(65.7) != 65 { return 1 }               // truncated toward zero
    if byte_of(300.0) != 0 { return 2 }
    big f64 = 300.0
    if (big as| u8) != 255 { return 3 }               // clamped
    return 0
}
```

```
func horizontal_bias(bh f64) i16 {
    return bh as i16        // ERROR [E086]: bh may be a NaN, or beyond what an i16 holds
}
```

Constraint

Casts between pointer types, between a pointer and an integer, or between any type and a pointer shall occur only inside an `unsafe` block. Such a cast outside `unsafe` is *ill-formed* (**[E012]**).

Constraint

A cast between non-numeric, non-pointer types (e.g., struct to integer) is *ill-formed*.

8.9 Handling a union: `else` and `try`

Two operators reach the value of a union (§7.9) by saying what happens on a marker. Each yields the value when the operand holds one, so its result has the union's value type.

The handler `else`. `u else h` evaluates `u`; on a value, the result is that value, and on any marker it is decided by `h`, in one of three forms:

- `u else f`: the result is `f`, which shall have the value type (**[E012]**).
- `u else return v`: the enclosing function returns `v`, converted to its return type, running its pending `defer`s as any `return` does. This is how one union's markers become another's.

- `u else panic(m)`: the program aborts with the message `m`. The `panic` gives the enclosing function the `raises` effect (§ 12).

`else` binds more loosely than every other operator: `a + b else c` is `(a + b) else c`.

```
// VERIFY: exit 0
type Small = u8 < 200

func get(ok bool) Small | NotFound {
    if ok {
        var v Small = 7
        return v
    }
    return NotFound
}

func or_zero(ok bool) Small {
    return get(ok) else 0                    // a fallback
}

func or_leave(ok bool) Small | Failed {
    v = get(ok) else return Failed           // NotFound leaves as
        Failed
    return v
}

func main() i32 {
    if or_zero(true) != 7 { return 1 }
    if or_zero(false) != 0 { return 2 }
    var a Small | Failed = or_leave(false)
    var got = 9
    case a {
        Failed: got = 0
        else: got = 3
    }
    if got != 0 { return 3 }
    return 0
}
```

3 **Propagation, `try`.** `try u` evaluates `u`; on a value, the result is that value, and on a marker the enclosing function returns that marker, with its fields, running its pending `defer`s. The enclosing function's return type shall therefore be a union that lists every marker of `u`; it may list more. `try` binds as tightly as the other prefix operators: `try f() + 1` is `(try f()) + 1`.

```
// VERIFY: exit 0
func open_it(p *u8, ok bool) *u8 | NotFound {
    if ok { return p }
    return NotFound
}

func check(p *u8, ok bool) *u8 | Denied {
    if ok { return p }
    return Denied
}

func load(p *u8, a bool, b bool) *u8 | NotFound | Denied {
    f = try open_it(p, a)       // on NotFound, load returns
        NotFound
    g = try check(f, b)         // on Denied, load returns Denied
```

```
    return g
}

func main() i32 {
    var r *u8 | NotFound | Denied = load("x", true, false)
    var got = 9
    case r {
        NotFound: got = 1
        Denied: got = 0
        else: got = 2
    }
    if got != 0 { return 1 }
    return 0
}
```

> **Constraint**
>
> The operand of `else` or `try` shall be a union value (`[E066]`).

> **Constraint**
>
> A `try` whose operand has a marker that the enclosing function's return type does not list is *ill-formed* (`[E065]`). So is a `try`, or an `else return`, inside a deferred statement (`[E138]`, §9.9).

```
func open_it(p *u8, ok bool) *u8 | NotFound | Denied {
    if ok { return p }
    return Denied
}

func read_cfg(p *u8) *u8 | NotFound {
    f = try open_it(p, true)      // ERROR [E065]: Denied is not
        a marker of *u8 | NotFound
    return f
}
```

4 **Checked operators.** The checked arithmetic operators `+?`, `-?` and `*?` and the checked cast `as?` (§8.8) compute what their plain forms compute, and on overflow, or on a value the target type cannot hold, they yield a marker instead. That marker exists only to be handled: a checked operator is the left operand of an `else`, in any of its three forms, and no union is ever stored.

> **Constraint**
>
> A checked operator that is not the left operand of an `else` is *ill-formed* (`[E067]`).

```
func next(a i32) i32 {
    return a +? 1                 // ERROR [E067]: what happens
        on overflow?
}
```

```
// VERIFY: exit 0
func main() i32 effects raises {
    var a i32 = 2000000000
    s = a +? a else 0                    // overflows: the fallback
    if s != 0 { return 1 }
    var big i32 = 300
```

```
    n = big as? u8 else 255              // 300 does not fit a u8
    if n != 255 { return 2 }
    var k i32 = 7
    p = k *? k else panic("overflow")    // fits: 49
    if p != 49 { return 3 }
    return 0
}
```

8.10 Move expression (mov)

1 `mov expr` transfers ownership of the value produced by *expr*. The source is placed in the Consumed state; any subsequent use of the source variable is *ill-formed* (`[E001]`). See § 11.

8.11 Borrows at call sites

1 Lain has *no* prefix borrow operator (no `&`, no `ref`, no `mut` expression form). A value is borrowed simply by passing it to a function or procedure; the borrow *mode* is written at the call site, matching the parameter's declared mode: nothing for a shared (read-only) borrow, `var` for a mutable borrow, and `mov` to transfer ownership (§ 8.10). See the call-site mode table in § 8.15 and the ownership rules in § 11.

8.12 Field access

1 The expression `expr.field` accesses the named field of a struct or ADT value, or a property of an array or slice (`.len`, `.data`).

> **Constraint**
>
> The field name shall exist on the type of *expr*.

> **Constraint**
>
> Writing to a field (`expr.field = v`) requires *expr* to be a mutable lvalue.

8.13 Index expression

1 The expression `arr[idx]` accesses the element of array or slice *arr* at integer index *idx*.

> **Constraint**
>
> The value range analysis shall prove $0 \leq idx < arr.len$ at every index expression. An index expression for which this cannot be proved is *ill-formed* (`[E085]`). An `in` guard (§ 8.7) or an `unsafe` block suppresses this requirement.

8.14 Range index expression (sub-slicing)

1 The expression `arr[a..b]` produces a *sub-slice*: a slice covering elements at indices $a, a+1, \ldots, b-1$ of `arr`. The range is **half-open**: the element at index b is excluded. The length of the result is $b - a$.

2 The inferred type of `arr[a..b]` is `T[b-a]`, a sized slice type (§ 7.4.1) whose length constraint records the exact length of the result.

Constraint

The start index `a` shall satisfy $a \geq 0$. A start index for which VRA cannot prove non-negativity is *ill-formed* (`[E085]`).

Constraint

The end index `b` shall satisfy $b \leq arr.len$. An end index for which VRA cannot prove this is *ill-formed* (`[E085]`).

Constraint

When both `a` and `b` are integer literals, the compiler additionally verifies $a \leq b$. If $a > b$, the program is *ill-formed* (`[E087]`).

Example

```
var xs i32[5] = [10, 20, 30, 40, 50]
var s = xs[1..4]       // type i32[3]; covers indices 1, 2, 3
var e = xs[3..3]       // OK: empty sub-slice, length 0
// var bad = xs[5..2] // [E087]: 5 > 2 (literal ordering
    violated)
```

8.15 Function call

1 A function call expression `f(args)` evaluates the arguments and transfers control to the callee. Arguments are evaluated left-to-right.

2 Argument ownership modes shall be given explicitly at the call site:

Syntax	Meaning
`f(x)`	shared borrow (default; parameter declared with no mode keyword)
`f(var x)`	mutable borrow (parameter declared `var`)
`f(mov x)`	ownership transfer (parameter declared `mov`)

Constraint

The mode annotation at the call site shall match the declared parameter mode. A mismatch is *ill-formed*.

Constraint

The number of arguments shall equal the number of parameters (unless the callee is a variadic C function declared with `...`).

3 **Two-phase borrows**: during argument evaluation, borrows of the receiver of a method call are placed in the RESERVED phase and promoted to ACTIVE after all arguments are evaluated (see §11).

8.16 Block expression

1 A block expression `{ stmts... expr }` evaluates each statement in order, then evaluates the final expression, whose value is the value of the block. If the final expression is absent, the block has no value.

> **Note**
>
> **Not yet in force.** This implementation does not parse a block in expression position: `{` cannot begin an expression or a statement (`[E100]`, §9.1). The form is intended, and is described for that reason.

8.17 If expression

1 An `if` expression of the form `if cond { e1 } else { e2 }` evaluates *cond*; if true, evaluates e_1; otherwise, evaluates e_2.

> **Note**
>
> **Not yet in force.** This implementation does not parse an `if` in expression position: `x = if b { 1 } else { 2 }` is refused at the `if` (`[E100]`). The two constraints below therefore describe a form no implementation currently accepts, and nothing checks them. They are retained because the form is intended; they are marked so that a reader does not take them for requirements an implementation is meeting. The `if` *statement* is §9.6, and is unaffected.

> **Constraint**
>
> The types of e_1 and e_2 shall be compatible. If they are not, the expression is *ill-formed*.

> **Constraint**
>
> *cond* shall have type `bool`, or an integer type, or be a union, which is true when it holds a value (§7.9).

8.18 Match expression (`case`)

1 A `case` expression (*match-expression*) selects one of several alternatives based on pattern matching. All arms shall produce values of compatible types; the type of the `case` expression is that common type. See §15 for full exhaustiveness rules.

8.19 Constructor expression

1 A constructor expression builds a value of a struct or ADT type by supplying all field values. See §7.6 and §7.8 for constructor rules.

8.20 Compound assignment

1 Compound assignment `x op= e` is syntactic sugar for `x = x op e`. The left-hand side shall be a mutable lvalue. All compound forms of the arithmetic and bitwise operators are supported (see §5.8).

Chapter 9

Statements

A statement is a unit of execution. Statements are terminated by a line feed or a semicolon (§5.4); the semicolon is used to place several statements on one line. The grammar production for *statement* is in Annex A.

9.1 Blocks

A block `{ stmts... }` is the body of a function, of an `if` or `else`, of a loop, of `unsafe`, or of `comptime if`. It creates a new scope (§6.2) and executes its statements in order. Names declared inside a block are not accessible outside it.

A block is not itself a statement: `{` never begins one. So a braced block cannot stand alone in a body, nor be the statement a `defer` takes, nor be the body of a `case` arm (§15.2).

> **Constraint**
>
> A `{` where a statement begins is *ill-formed* (`[E100]`).

```
func main() i32 {
    {                          // ERROR [E100]: a block is not a
        statement
         x = 1
    }
    return 0
}
```

9.2 Immutable declaration

`name [Type] = expr` declares and initializes an immutable binding. The type may be omitted (inferred from the initializer). The binding cannot be reassigned after initialization.

```
type Point {
    x i32
    y i32
}

func main() i32 {
    x i32 = 42          // explicit type
    y = 10              // type inferred (i32)
    p Point = Point(1, 2)
    return 0
}
```

Constraint

An immutable declaration requires an initializer expression: an immutable binding must have a definite value at its point of declaration.

Constraint

When a type annotation is present, the type of the initializer shall be compatible with the declared type (§ 7.12).

Note

When the type is omitted, the distinction between declaration and reassignment is resolved by scope: `x = 5` where `x` is not yet declared creates a new immutable binding; where `x` is already declared as `var`, it is a reassignment.

9.3 Mutable variable declaration

`var name Type = expr` declares a mutable variable. Mutable variables can be reassigned. The type may be omitted if it can be inferred from the initializer.

```
var counter i32 = 0
counter = counter + 1        // OK: var is mutable

var buf u8[256]               // OK: type present; unassigned
    until written
                              // (reading before a write is [E005])
```

Constraint

A mutable declaration shall include either an explicit type annotation or an initializer expression. A declaration with neither is *ill-formed* (`[E100]`).

Constraint

When both a type annotation and an initializer are present, the type of the initializer shall be compatible with the declared type (§ 7.12).

9.4 Assignment statement

An assignment statement `lvalue = expr` stores the value of *expr* into the storage designated by *lvalue*. An assignment target without a type annotation is always a reassignment, never a declaration.

Constraint

The left-hand side shall be a mutable lvalue (§ 6.6). Assigning to an immutable binding, or to an element or field of one, or to a module constant, is *ill-formed* (`[E009]`). So is lending such a place as `var`, as an argument (`f(var a)`) or a reference binding (`var r = var x`). An array or slice *parameter* is an output reference: its elements may be written, so an immutable array (or a module constant) shall not be passed to one that the callee writes, directly or through a further call; an `extern` is assumed to write it (`[E009]`). Immutability belongs to the *storage*: no write shall reach an immutable binding's storage, or a module constant's, by any path, whether through a slice or reference kept in a `var` binding (`var s = xs[0..2] s[0] = 9`), handed back by a call (`w(id(xs[0..2]))`), or held in a struct

whose callee writes through it, other than the binding's own initialiser ([E009]). A copy (`var c = xs`) is new storage and may be written.

Constraint

Overwriting a linear variable whose current value has not been consumed would leak that value. Such an assignment is *ill-formed* ([E021]).

Note

Assigning to a linear variable after its previous value has been consumed (e.g., loop reassign-and-consume pattern) is permitted. The implementation resets the variable's linear state to Unconsumed after a legal overwrite.

Example

```
import std.fs.{open_file, close_file}

func main() int effects io, raises, alloc {
    var f = open_file("data.txt", "r")
    close_file(mov f)                  // f is now Consumed
    f = open_file("data.txt", "r")  // OK: overwriting Consumed
        state
    close_file(mov f)
    return 0
}
```

9.5 Expression statement

An expression followed by a statement terminator is an expression statement. The value of the expression is discarded.

9.6 If statement

`if cond { body } [else if cond { body }]... [else { body }]` evaluates *cond*; if true, executes the then-body; otherwise, tests the `else if` clauses in order; if none matches, executes the `else` body (if present).

Constraint

cond shall have type `bool`, or an integer type, or be a union, which is true when it holds a value (§ 7.9). An integer in condition position is false when zero and true otherwise.

Range refinement: inside the then-branch of `if x < N { }`, the value range analysis (§ 13) knows that $x \in (-\infty, N-1]$; inside the else-branch, $x \in [N, +\infty)$.

In-guard: when *cond* contains or is an `in` expression (§ 8.7), the in-guard is active inside the then-branch only.

Branch consistency: if one branch consumes a linear variable, all branches shall consume it. If branches disagree on the linear state of a variable from the enclosing scope, the program is *ill-formed* ([E016], § 11.12).

9.7 While statement

1 `while cond { body }` repeatedly evaluates *cond* and, if true, executes *body*.

> **Constraint**
>
> A `while` loop shall be bounded: it shall carry a `decreasing` measure, or the implementation shall be able to infer one from its condition (§ 9.7.2). A loop for which no measure is present and none can be inferred is *ill-formed* (`[E011]`), unless the enclosing function's row names `diverge` (§ 12.2).

> **Constraint**
>
> *cond* shall have type `bool`, or an integer type, or be a union, which is true when it holds a value (§ 7.9). An integer in condition position is false when zero and true otherwise.

2 **In-guard**: when *cond* contains or is an `in` expression, the in-guard is active inside the loop body for each iteration.

9.7.1 Bounded while statement

3 `while cond decreasing M { body }` is a bounded loop. In addition to the while semantics above, the implementation shall statically verify:

1. **Bounded below** (`[E082]`): the implementation shall prove that M cannot decrease without limit.
2. **Analyzable measure** (`[E082]`): M shall be in a form the implementation can analyse. A measure it cannot analyse at all — including a constant, or one over variables the body never assigns — is rejected, and so is one that is merely not provably well-founded: both are `[E082]`, because one termination obligation with one source reports them. (Two earlier codes, `E080` and `E081`, separated these sub-cases when a second termination checker existed in the front end; they are withdrawn, see Annex B.)
3. **Strict decrease** (`[E082]`): the implementation shall prove that every execution of *body* strictly decreases the value of M, *except* along paths that leave the loop via `break` or `return`. A branch that unconditionally exits the loop never iterates again, so it is exempt from the decrease requirement; only paths that loop back need decrease M. (A `continue` re-evaluates *cond* and thus loops back, so it is *not* an exit.)

4 A bounded `while` loop is permitted inside a `func` because the implementation has statically verified termination.

5 The following patterns are accepted by the standard verification algorithm:

Condition	Measure	Why
`i < n`	`n - i`	i increases ⇒ measure decreases
`i <= n`	`n - i`	i increases toward the inclusive bound
`n > 0`	`n`	n decreases directly
`i > k`	`i - k`	i decreases toward k
`a < b`	`b - a`	either a increases or b decreases
`i in 0..arr.len`	`arr.len - i`	the range's upper bound gives $i <$ `arr.len`

6 Beyond the syntactic patterns above, the standard verification algorithm also accepts a measure variable that is set to a constant provably below the loop's guaranteed lower bound (e.g. `j = 0` inside `while j > 0`), and unsigned measures driven by `v = X % v`, `v = v / D`, or `v = v >> C`.

9.7.2 Measure inference

7 The `decreasing` clause *may be omitted*. When it is, and the loop appears in a context that requires termination (a `func` body), the implementation shall attempt to *infer* a measure from *cond* using the correspondences above: a relational condition `i < n`, `i <= n`, `i > k`, or `i >= k` yields the candidate `n - i` (respectively `i - k`); a two-variable bound `lo < hi` yields `hi - lo`; and a range membership `i in 0..n` yields `n - i`.

8 An inferred measure is subject to the *identical* verification as an explicit one (non-negativity, analyzability, strict decrease). If no candidate can be inferred, or the candidate fails verification, the loop is *ill-formed* (`[E011]`): the programmer shall then supply an explicit `decreasing` clause or restructure the loop. Inference is therefore a convenience only; it never accepts a loop that an explicit measure could not.

Example

```
func count(n int) int {
    var i = 0
    while i < n {           // measure `n - i` is inferred
        i = i + 1
    }
    return i
}
```

Rationale

Bounded loops bridge the gap between pure functions (which must terminate) and the common pattern of finite-input state machines (lexers, parsers) that are provably terminating. For the overwhelmingly common counting and scanning loops the measure is mechanical, so it is inferred; the `decreasing` keyword remains available to state a termination argument the compiler cannot infer, which it then verifies.

9.8 Return statement

1 `return [expr]` terminates execution of the current function and transfers the value of *expr* (if present) to the caller.

Constraint

At the point of a `return`, all linear variables in scope shall have been consumed. A live (Unconsumed) linear variable at `return` is *ill-formed* (`[E003]`).

2 A `return` expression may be prefixed with `mov` or `var`: `return mov x` transfers ownership of `x` explicitly, and `return var x` returns a mutable borrow of `x` (the pattern used by a `func` whose return type is `var T`). Ownership transfer is also implicit from the function's declared return type, so a bare `return x` suffices when that type is owned.

Constraint

A `return var x` (or `return mov x`) that would escape a borrow of a *local* variable is *ill-formed* (`[E010]`); returning a `var` borrow tied to a `var` parameter is permitted. Returning

is one of several ways a reference can outlive the storage it borrows, and the rule governing all of them is §19.3.

9.9 Defer statement

1 `defer` *statement* schedules *statement* for execution immediately before the enclosing scope exits, on all exit paths (`return`, `break`, `continue`, or fall-through). Multiple `defer` statements execute in LIFO order. The deferred construct is a single statement, not a braced block: a block is not a statement here, so `defer { ... }` is *ill-formed*.

2 **Interaction with linear types**: if a linear variable is consumed inside a `defer` body, the variable is marked as defer-consumed. It is treated as unconsumed for the purposes of the main body (it may be used before the scope exit) and as consumed at scope exit.

3 A deferred statement is a scope of its own: a `defer` inside it runs when the deferred statement ends, as at the end of any block.

Constraint

Control shall not leave a deferred statement: a `return`, a `break` or `continue` aimed at a loop outside it, and a `try` or `else return` that leaves the function are *ill-formed* (`[E138]`). A loop inside the deferred statement may use `break` and `continue`.

Example

```
import std.fs.{open_file, close_file}

func process() int effects io, raises, alloc {
    var f = open_file("data.txt", "r")
    defer close_file(mov f)        // f consumed at scope exit
    // ... use f ...
    return 0
}
```

9.10 Unsafe block

1 `unsafe { body }` is an unsafe block (see §18). It creates a scope in which certain static checks are relaxed.

9.11 Assert and assume statements

1 `assert P` states a fact the compiler shall prove where it is written: its predicate `P`, an expression of type `bool`, is a proof obligation, and a predicate the analyses cannot discharge makes the program *ill-formed* (`[E012]`). An `assert` costs nothing at run time. `assume P` states a fact the compiler takes without proof, and is allowed only inside an `unsafe` block (`[E129]`). Parentheses around the predicate are optional, as for a module-scope `assert` (§10.8).

Note

The two are duals, and the asymmetry in where they are permitted follows from that. An `assert` *adds* an obligation: the analyses must discharge it, so the worst a wrong one can do is refuse a program. An `assume` *removes* one: it is believed, and every proof downstream of it is conditional on it being true. An assumption nothing pays for is an axiom, and an axiom silently makes a prover able to prove a false statement — which is why it is confined to `unsafe`, where the programmer has already taken the obligation on.

The practical use of `assert` is to choose where imprecision is reported. Without one, a fact the analyses fail to carry surfaces at whatever later line first needs it, often an array access several statements away; with one, it surfaces at the predicate that was actually lost.

Example

```
func last(a u8[]) u8 {
    if a.len > 0 {
        n = a.len - 1
        assert n < a.len
        assert(n + 1 == a.len)
        return a[n]
    }
    return 0
}
```

9.12 Comptime if statement

`comptime if cond { then } [else { else }]` evaluates *cond* at compile time. Only the taken branch is type-checked and emitted; the other branch is discarded. *cond* shall be a compile-time constant expression (§ 14).

Note

`comptime if` is the Lain equivalent of C preprocessor `#if`: it enables platform-specific or type-specific code selection without dead-code warnings.

9.13 For statement

`for i in start..end { body }` iterates with i ranging from *start* (inclusive) to *end* (exclusive), and `for i in start..=end` includes *end*. The loop variable i is immutable; its value range is statically known throughout the body — $[start, end - 1]$, or $[start, end]$ — so an index it proves in bounds needs no further check.

`for x in a { body }` iterates over the elements of an array or slice `a`, binding each in turn to `x`; `for i, x in a` also binds the element's index, which comes first.

`for` loops are permitted whatever a function's row says: the range is the bound.

Constraint

start and *end* shall have integer types (`[E012]`). The loop variable shall not be assigned (`[E009]`).

```
// VERIFY: exit 0
func main() i32 {
    var t u8[3] = [7, 8, 9]
    var n = 0
    for i in 0..=3 { n = n + 1 }          // 0, 1, 2 and 3
    var s u32 = 0
    for x in t { s = s + x }              // 7 + 8 + 9
    var at usize = 9
    for i, x in t {
        if x == 8 { at = i }
    }
    if n != 4 { return 1 }
```

```
13      if s != 24 { return 2 }
14      if at != 1 { return 3 }
15      return 0
16  }
```

Chapter 10

Declarations

1 This clause specifies top-level declarations. Local variable declarations are specified in § 9.3. The grammar productions are in Annex A (*function-declaration*, *struct-declaration*, *enum-declaration*).

10.1 Declaration attributes

1 A declaration may be preceded by zero or more attributes. An attribute has the form **[name]** or **[name(args)]**, where *name* is a recognized attribute identifier from the closed set in Table 10.1, and *args* (if present) is a comma-separated list of expressions. Several attributes on one declaration may share a line or take one each.

```
[hot]
func dot(a f64[], b f64[a.len]) f64 {
    var s f64 = 0.0
    for i in 0..a.len { s = s + a[i] * b[i] }
    return s
}

[private]
func internal_helper() int { return 0 }

[cold]
[private]
func rarely_called() f64 { return 1.0 }
```

2 Recognized attribute names (closed set, version 1.1):

Attribute	Applies to	Purpose
[private]	any declaration	Module-private visibility.
[packed]	a struct	Lay the fields out bit-exactly inside on
[ordered]	a struct, or a sum whose variants carry payloads	Store the fields, or each payload's field
[cold], **[hot]**	any function	Rarely or frequently executed: a hint t
[noreturn]	any function	Never returns. On a function with a b
[allocator]	an **extern** function	The result is non-null and aliases no ot

Constraint

Q-017. An attribute name not listed in Table 10.1, or one written on a declaration it does not apply to, is *ill-formed* (`[E103]`). `[allocator]` applies only to an `extern` function: the claim cannot be checked against a body, and a false one is undefined behaviour in the translation. An attribute that omits the identifier (e.g. `[ ]` or `[(arg)]`) is *ill-formed* (`[E102]`).

Note

Attributes are recognized at declaration position only. At expression position, `[` and `]` continue to delimit array and slice subscripts and array literals; the parser disambiguates by context.

10.2 Struct declarations

1 A struct declaration introduces a named product type with one or more named fields. Fields are listed in declaration order; order is significant for construction (a constructor's arguments are the fields in that order). The order in which they are stored is §7.6's.

```
type Point {
    x i32
    y i32
}
```

2 A field may be annotated with `mov`, making the field (and therefore the containing struct) a linear type:

```
extern type FILE

type File {
    mov handle *FILE
}
```

Note

A struct with zero fields (a *unit* struct, `type Empty {}`) is permitted: it is a zero-sized marker type, constructed as `Empty()`.

10.3 Enum and ADT declarations

1 An enum declaration introduces a type whose values are drawn from a fixed, named set of variants. If every variant has no associated fields, the type is a simple enum (§7.7). If at least one variant has associated fields, the type is an algebraic data type (ADT) (§7.8).

```
type Direction {                // simple enum (one variant per
    line)
    North
    South
    East
    West
}

type Shape {                                    // ADT
    Circle     { radius i32 }
    Rectangle  { width i32, height i32 }
    Point
}
```

10.4 Function declarations

A function declaration introduces a `func`: a pure, terminating, side-effect-free callable. Its grammar production is *function-declaration*.

The signature consists of: the keyword `func`, a name, an optional list of type parameters (§ 14), a parameter list, and a return type annotation.

10.4.1 Parameters

Each parameter has a name, a type, and an ownership mode. The ownership mode determines how the argument is passed:

Syntax	Mode	ABI (struct T)
`name T`	shared (read-only)	`const T*`
`var name T`	mutable	`T*`
`mov name T`	owned (transfer)	`T` (by value)

> **Note**
>
> For primitive types (integer, float, bool), the ABI passes the value directly regardless of ownership mode, as for primitive C arguments.

A parameter whose declared type is the built-in `type` — written `name type`, with no `comptime` prefix — is a compile-time type parameter; it is bound to a concrete type at every call site, by inference from a value argument or explicitly (§ 14).

10.4.2 Return type annotation

The return type is written after the parameter list. It may carry a value range constraint:

```
func abs(x int >= -2147483647) int >= 0 {
    if x < 0 { return -x }
    return x
}
```

The constraint's right-hand side is an integer literal, a parameter, a parameter's `.len`, or a sum or difference of those. The function proves it at every return; every caller then knows it of the result. A search can thereby promise an index into its argument:

```
func position(a u8[>= 1], c u8) usize < a.len {
    var i usize = 0
    while i < a.len {
        if a[i] == c { return i }
        i = i + 1
    }
    return 0
}

func at_first_space(a u8[>= 1]) u8 {
    return a[position(a, 32)]   // proven: the result is below
        a.len
}
```

Constraint

All return paths of a `func` shall produce a value whose type and value range satisfy the declared return type and constraint (`[E086]` where a return cannot be shown to). A right-hand side that names anything but a parameter, or the `.len` of anything but a slice parameter, is unsatisfiable.

10.4.3 Purity constraints

Constraint

A function's `effects` row is an upper bound on what its body does, and its default is ∅. An effect the body has and the row does not name is *ill-formed* (`[E011]` where the bound is ∅, `[E130]` where a non-empty row understates it). The three constraints below are consequences of that one rule, not separate rules.

Constraint

A function shall not contain an unbounded `while` loop — one whose measure is neither given by `decreasing` nor inferable — unless its row names `diverge` (`[E011]`/`[E082]`).

Constraint

A function shall not call one whose row names an effect its own row does not (`[E011]`). The callee's *introducer* is irrelevant: what is refused is the effect, not the keyword that declared it.

Constraint

A function on a recursion cycle shall be shown to terminate, or its row shall name `diverge` (`[E011]`): direct self-recursion by an inferred or given measure, and a cycle through several functions by one measure per function (§ 12.2).

10.5 The entry point

1 `proc` **has been removed.** It was an introducer that granted effects instead of stating them, and every effect it was needed for is now named by an `effects` row on an ordinary `func` — including `diverge`, which admits an unbounded loop. The keyword remains *reserved*, and using it is `[E100]` with the replacement spelled out, so old source fails with an explanation rather than with "expected declaration". The same applies to the function-pointer type `*proc(...)`, replaced by `*func(...)` with a row (§ 12.1).

2 The entry point of every Lain program shall be declared as:

```
func main() int { return 0 }      // with 'effects io' if it
    performs any
```

or, to read its command line, with C's two parameters, `argc` (`i32`, or `int`) and `argv` (`**u8`). Reading `argv` reads through a raw pointer, so it is done inside `unsafe` (§ 18):

```
// VERIFY: exit 1
func main(argc i32, argv **u8) i32 {
    if argc >= 2 {
        var c u8 = 0
        unsafe {
            first = *(argv + 1)        // argv[1], a
                NUL-terminated C string
```

```
            c = *first                  // its first byte
        }
        return c as i32
    }
    return 1                            // run with no arguments,
        argc is 1
}
```

Constraint

`main` takes no parameters, or exactly those two, in that order, neither `var` nor `mov`. Any other parameter list is *ill-formed* (`[E100]`).

```
func main(n i32) i32 {      // ERROR [E100]: not C's two
    parameters
    return 0
}
```

Constraint

A complete *program* shall define exactly one `main` returning `int` as its entry point. An individual module (translation unit) compiled on its own — e.g. a library — need not define `main`: the reference compiler accepts a `main`-less source file, and a program missing an entry point is diagnosed when the translation outputs are linked.

10.6 Extern declarations

An extern declaration introduces a C function into the Lain name space without providing a body. See § 17 for full semantics.

```
extern func printf(fmt *u8, ...) int effects io
extern func strlen(s *u8) usize effects
```

Constraint

Variadic parameters (`...`) shall only appear in `extern` declarations. A variadic function with a body is *ill-formed* (`[E100]`).

10.7 Module-scope constants

A declaration `NAME T = expression` at module scope introduces an immutable constant (`NAME = expression` takes the initialiser's type). There are no mutable module-level variables. A constant array whose elements are integer constants is one read-only object: reading it through an index (`CTYPE[c]`) or a slice (`CTYPE[0..16]`) reads that object, with no copy, and the reference implementation emits it once as a `static const` array. Since the object outlives every function, a slice of it may be returned. A constant may not be written, nor lent as `var` (`[E009]`). The initialiser is converted to `T` by the rules of a declaration: an implicit conversion between integer and floating-point types, or between an integer type and `bool`, is *ill-formed* (`[E012]`), and an array initialiser shall have exactly `T`'s length (`[E012]`).

```
// VERIFY: exit 0
N = 7                           // i32: the initialiser's type
BIG = 5000000000                // i64: too large for an i32

func main() i32 {
```

```
    k = N
    if k != 7 { return 1 }
    if BIG - 1 != 4999999999 { return 2 }
    return 0
}
```

Constraint

When the initialiser is a constant expression (§10.8) of an integer type, its value shall be representable in `T`; for an array of an integer type, each element's shall be representable in the element type. Otherwise the program is *ill-formed* (**[E086]**).

```
MAX_TOKENS usize = 4096
LIMIT u8 = 200
OVER u8 = 300            // ERROR [E086]: 300 does not fit u8
```

2 An *array comprehension* `[e for i in lo..hi]` is the array whose element `k` is `e` with `i` = `lo + k`, for `i` from `lo` up to but excluding `hi` (`..=` includes `hi`). The bounds give the array its length, so `lo` and `hi` shall be constant expressions (**[E137]**). Inside a function a comprehension initialises an array at run time. As a module-scope constant whose element type is an integer type, when `e` is a constant expression too (`e` may name `i`), it *is* the array literal of its values, and is checked as one (§10.7); any other module-scope comprehension is evaluated at compile time, as an initialiser that calls a function is (below):

```
// 1 for a space, 2 for a digit: a lexer's character-class table
CTYPE u8[256] = [(c == 32 or c == 9) as u8 + 2 * (c >= 48 and c
    <= 57) as u8 for c in 0..256]
func is_digit(c u8) bool {
    return (CTYPE[c] & 2) != 0
}
```

3 An initialiser that *calls a function* — a scalar initialiser, an element of an array literal, or the body of a comprehension, evaluated once for each index — is *evaluated at compile time*, by the semantics of this standard, and the constant is the value it computes, as if that value had been written as a literal. The order of the declarations is immaterial: a constant may be computed by a function that reads another constant, declared before it or after it.

Constraint

Every function that such an initialiser can reach shall have the empty effect row (§12.1): it is pure and total. Each shall be free of every diagnostic of this standard, the evaluation shall not exceed the step limit nor recurse deeper than the evaluator can hold (at least 256 calls, §4.4), and the computation of a constant shall not read that constant. Otherwise the program is *ill-formed* (**[E136]**). The value shall be representable in `T`, and for an array each element in the element type (**[E086]**).

```
func cls(c usize) u8 {
    if c == 32 or c == 9 { return 1 }
    if c >= 48 and c <= 57 { return 2 }
    return 0
}
CTYPE u8[256] = [cls(c) for c in 0..256]    // cls runs 256
    times, while compiling
func is_digit(c u8) bool {
    return (CTYPE[c] & 2) != 0
}
```

```
extern func libc_puts(s *u8) i32 effects io
func greet() u32 effects io {
    libc_puts("hello")
    return 1
}
G u32 = greet()          // ERROR [E136]: greet has the effect
    io, so it cannot run while compiling
```

4 The step limit is *implementation-defined*[1] The reference implementation counts the operations of its intermediate representation, ten million for each constant unless the environment variable `LAIN_STATIC_STEPS` says otherwise, so a program that compiles on one machine compiles on every machine. Unsafe code that such a function reaches is evaluated with every check in place: an access outside an object's bounds is refused (**[E136]**), not performed.

10.8 Module-scope assertions

1 A module-scope **assert** states a fact about the program's constants and the layout of its types, where they are declared:

```
type Token {
    kind u8
    len  u16
    pos  u32
}
BUF usize = 64
assert @sizeof(Token) == 8
assert BUF % 16 == 0
```

```
assert (300 as% u8) == 44          // wrapped to the cast's type:
    300 - 256

func main() i32 {
    return 0
}
```

2 Its operand is a *constant expression*: integer and character literals; named immutable module-level constants whose initialisers are constant expressions; `@sizeof(T)` and `@alignof(T)` (§19.8); the arithmetic, bitwise, shift, comparison and logical operators; **as**; and the wrapping operators **+%**, **-%**, ***%** and the wrapping cast **as%**, each computed over the integers and then reduced modulo 2^N to its type. Its value is computed over the integers, not in the width of any one type: `(1 << 32)- 1` is $2^{32} - 1$. The exception is `~`, which has no meaning over the integers for an unsigned operand: `~`x of an unsigned N-bit value is $2^N - 1 - x$, so `~(0 as u64)` is the largest `u64`. The same constant expressions bound or size a struct field's length (§7.6.1).

> **Constraint**
>
> The operand of a module-scope **assert** shall be a constant expression of type `bool`; otherwise the program is *ill-formed* (**[E133]**). The definition above is a closed list, so an element of a constant table and a function call are not constant expressions — not because their values are unknown before run time, since a constant table's elements are fixed at translation time, but because neither is in the list. A fact about a value known only at run time is asserted inside a function, where it is an obligation the value-range analysis must discharge (§13).

[1] Implementation-defined behavior: the number of steps a constant's evaluation may take.

3 Inside a function, the analysis knows an element of a constant table read at a constant index exactly, so a fact about it is proven there:

```
// VERIFY: exit 0
T u8[4] = [1, 2, 3, 4]

func main() i32 {
    assert T[1] == 2                // proven: the element's value
        is fixed at translation time
    if T[3] != 4 { return 1 }
    return 0
}
```

Constraint

A module-scope assertion whose value is `false` makes the program *ill-formed* (`[E134]`). When the operand involves `@sizeof` or `@alignof` its value is the target C compiler's, and the implementation shall have that compiler decide it: the reference implementation emits a C `_Static_assert` carrying `[E134]`. Otherwise the implementation decides it itself.

10.9 Comptime declarations

Note

Top-level `comptime { ... }` blocks — evaluated by the compiler before any runtime code is emitted, to define compile-time constants or trigger `compileError` assertions — are a *planned* feature not yet accepted by the reference implementation (a top-level `comptime` block is presently `[E100]`). The `comptime` keyword is nonetheless reserved, and `comptime if` (§ 9.12) is available.

Chapter 11

Ownership and linearity

1 This clause specifies Lain's linear type system, ownership modes, borrow checker, and non-lexical lifetime rules. All rules in this clause are enforced at compile time with zero runtime overhead.

11.1 Ownership modes

1 Every parameter, return value, and struct field has one of three ownership modes:

Mode	Param syntax	Call-site	Semantics
Shared	`name T`	`f(x)`	read-only; multiple concurrent borrows allowed
Mutable	`var name T`	`f(var x)`	exclusive read-write; no other borrow concurrent
Owned	`mov name T`	`f(mov x)`	transfers ownership; source becomes Consumed

2 Local variables declared with `var name = expr` have mutable storage accessible within their scope; their ownership mode in the type system is Owned. When the value is of a *linear* type, an Owned local must be consumed before its scope exits (`[E003]`); a local of an unrestricted type carries no such obligation.

11.2 Linear states

1 Every variable of a linear type has a *linear state* at each program point. The three states are:

Uninit The variable has been declared but no value has been assigned.

Unconsumed The variable holds a value that has not yet been transferred.

Consumed The variable's value has been transferred via `mov` or a consuming function call.

2 The state machine governing transitions is:

State before	Operation	State after
Uninit	first assignment	Unconsumed
Unconsumed	`mov x` (transfer)	Consumed
Unconsumed	borrow (`var` or shared)	Unconsumed (borrow active)
Consumed	assignment (overwrite)	Unconsumed
Consumed	read or `mov`	*ill-formed*
Uninit	read or `mov`	*ill-formed*

Note

The Consumed → Unconsumed transition via overwrite enables the loop-reassign-then-consume pattern: a linear variable declared outside a loop may be consumed and reassigned on each iteration without leaking.

11.3 Diagnostics

Constraint

Using a variable, or a reference whose owner has been moved, after that value has been transferred (use-after-move; the Consumed state) is *ill-formed* (`[E001]`). Using a variable *before* it is initialized — the Uninit state — is a different diagnostic, `[E005]` (below).

Constraint

Consuming a struct field that has already been consumed (a double-consume at field granularity) is *ill-formed* (`[E002]`).

Constraint

A linear variable that is in the Unconsumed state when its scope exits (resource leak) is *ill-formed* (`[E003]`).

Constraint

An aliasing violation — attempting a mutable borrow when a shared borrow is active, or a shared borrow when a mutable borrow is active, or a second mutable borrow of the same variable — is *ill-formed* (`[E004]`).

Constraint

Reading a variable, struct field, or array element before it has been initialized (use of an uninitialized value; the Uninit state) is *ill-formed* (`[E005]`). Definite-assignment analysis tracks initialization per binding and per field.

Constraint

Consuming a linear variable or field declared outside a loop, from inside a body that may execute more than once, is *ill-formed* (`[E002]`): it *is* consuming the value twice, and `[E002]` names the violation where the retired `E006` named only the syntactic setting it occurred in.

Constraint

Passing a linear value to an owned (`mov`) parameter without the explicit `mov` keyword at the call site — an implicit move — is *ill-formed* (`[E007]`). (Consuming a linear value on some branches but not others is a separate diagnostic, `[E016]`, below.)

Constraint

Moving a value that is currently borrowed (`var` or shared) is *ill-formed*: when the borrow and the move are arguments of one call, `f(p, mov p)`, the diagnostic is `[E008]`; when a borrow held by a binding is still in use, it is `[E004]`. Moving a struct after one of its fields has been moved is moving that field twice (`[E002]`).

Constraint

If a linear value is in state CONSUMED on some control-flow paths through an `if`/`case` statement and in state UNCONSUMED on others, the merged state at the join point is undefined and the program is *ill-formed* (`[E016]`). The same diagnostic applies at field level: if a linear field of a struct is consumed on some paths but not on others, `[E016]` is issued for that field.

11.4 Mutable parameters for primitive types (`var T`)

A parameter declared `var name T`, where `T` is a primitive type (`iN`, `uN`, `f32`, `f64`, `bool`), is passed **by pointer** in the C ABI: the parameter's C type is `T*`.

Write-through semantics: an assignment to such a parameter inside the function body writes through the pointer and is visible to the caller. Reading the parameter's value automatically dereferences the pointer.

Example

```
func increment(var n i32) {
    n = n +% 1    // writes through: caller's variable is updated
}

func main() i32 {
    var x = 0
    increment(var x)
    return x      // x == 1
}
```

Forwarding: when a `var T` primitive parameter is passed as argument to another `var T` parameter, the pointer is forwarded directly without dereferencing and re-taking the address. This behavior is identical to mutable parameters of struct type, where the by-pointer ABI was already in effect. The distinction is that for primitive types the codegen previously did not dereference reads; this was a bug that has been corrected.

Note

The call site shall annotate the argument with `var`: `f(var x)`. Omitting `var` passes a shared (read-only) borrow, which does not permit writes through the callee.

11.5 The read-write lock invariant

Invariant: At every program point, for every variable V, exactly one of the following holds:

1. V has $N \geq 0$ active shared borrows and zero active mutable borrows.
2. V has exactly one active mutable borrow and zero shared borrows.

This invariant is the compile-time analogue of a reader-writer lock. A conforming implementation shall enforce it at every program point.

11.6 Non-lexical lifetimes (NLL)

A borrow's lifetime ends at the borrow's *last use*, not at the end of the variable's lexical scope. The implementation computes last-use points via a use-analysis pre-pass over the AST.

Example

```
type Data {
    v i32
}

func main() i32 {
    var data = Data(42)
    var r = var data.v          // mutable borrow of data.v starts
    r = 7                       // last use of r: the borrow ends HERE (NLL)
    var y = data.v              // OK: the borrow has expired
    return y
}
```

11.7 Reference bindings

1 A declaration `var r = var P`, where `P` is a place (a local, a field path, an element, or a `var` parameter), or `var r = f(...)` where `f` returns `var T`, binds `r` to a **mutable borrow** of that place. Reading `r` yields the value at the place; assigning to `r` (or to a field of it) writes the place; `var r` passes the borrow on. `var r2 = var r` is a *reborrow*: uses of `r2` are uses of `r`'s loan.

2 A slice already carries its data pointer, so `var r = var s` for a slice `s` lends the slice's *elements*: `r[i] = v` writes them. Reassigning `r` as a whole is *ill-formed* [E009], exactly as for a `var` slice parameter — it would rebind the view and leave `s` unchanged.

3 The loan lasts until the binding's last use (§11.6). While it is live, any other access to an overlapping place — a read, a write, or a borrow — violates the read-write lock invariant (§11.5) and shall be diagnosed [E004]. Distinct fields, and elements at distinct constant indices, are distinct places; elements at indices not proven distinct are assumed to overlap.

4 A reference to storage owned by the function shall not be returned, nor stored where the caller can reach it [E010] (§ 19.3). A function whose return type is `var T` shall return `var P` or a call that itself returns `var T` [E017]. A struct field shall not hold a mutable borrow [E126].

5 A borrow — a reference binding, or a `var` or shared parameter — lends a place; it does not own it. A linear value shall not be moved out through one, by `mov` or by a whole-value copy [E020], and a linear place reached through one shall not be assigned [E021]: the resource there is the owner's, and it would be lost. The same assignment to a place the function owns is refused while the place may still hold a resource [E021].

Example

```
type P { x i32, y i32 }

func main() i32 {
    var p = P(1, 2)
    var x = var p.x
    var y = var p.y             // OK: p.x and p.y are disjoint
    x = 7
    y = 8
    var t = p.x                 // OK: x and y are dead here (NLL)

    var q = var p
    p.y = 5                     // error [E004]: q is used below
    q.x = 3
```

```
    return 0
}
```

11.8 Two-phase borrows

During argument evaluation for a function call, borrows of the receiver object are placed in the *RESERVED* phase. A RESERVED borrow allows concurrent shared reads of the same owner but blocks mutable writes and moves. After all arguments have been evaluated, RESERVED borrows are promoted to the *ACTIVE* phase.

This mechanism enables UFCS calls of the form `x.f(x.field)` where the first argument (the receiver) requires a mutable borrow of `x` while the second argument reads a field of `x`.

Phase	Allows on owner	Blocks on owner
RESERVED	shared reads	mutable writes, moves
ACTIVE	(nothing)	all other access

Example

```
type Vec {
    data int
    cap int
}
func push_n(var v Vec, n int) { v.data = v.data +% n }

func main() int {
    var v = Vec(0, 10)
    v.push_n(v.cap)    // desugars to push_n(var v, v.cap)
                       // var v enters RESERVED phase; v.cap
                          read is allowed
    return v.data
}
```

11.9 Move semantics

The `mov` operator transfers ownership. The source variable transitions to Consumed state; any subsequent read or move of the source is *ill-formed* (`[E001]`).

Ownership transfer is the only way to pass a linear value to a consuming parameter. The `mov` keyword shall be present at the call site: `f(mov x)`.

11.10 Borrowed match

The `case &expr { ... }` form (see § 15) performs pattern matching without consuming the scrutinee. A temporary shared borrow of the scrutinee is registered for the duration of the match body and released when the match completes.

Constraint

Inside a `case &expr` body, the scrutinee shall not be mutated or moved. Attempting to do so violates the active shared borrow (`[E004]`).

11.11 Loop and ownership

1 A linear variable declared outside a loop may be consumed inside the loop body if and only if it is also reassigned (overwritten) on every path through the loop body before the next iteration begins.

Example

```
import std.fs.{open_file, close_file}

func main() int effects io, raises, alloc {
    var f = open_file("data.txt", "r")
    close_file(mov f)        // consumed before loop
    var i = 0
    while i < 3 decreasing 3 - i {
        f = open_file("data.txt", "r")  // reassign (Consumed
            -> Unconsumed)
        close_file(mov f)               // consume
        i = i + 1
    }
    return 0
}
```

11.12 Branch consistency

1 After an `if`/`else` chain or `case` match, the linear state of each variable from the enclosing scope is the *intersection* of its states across all branches:

- If consumed in all branches: the variable is Consumed after the chain.
- If consumed in no branches: the variable is Unconsumed after the chain.
- If consumed in some but not all branches: *ill-formed* (`[E016]`).

2 The same rule applies at field level: if a linear field of a struct is consumed on some branches but not on others, `[E016]` is emitted for that field.

Example

```
// ERROR [E016]: h is consumed in the then-branch but not in
    the (implicit) else
type Handle { mov ptr *i32 }

func drop(mov h Handle) { }

func maybe_consume(mov h Handle, flag bool) {
    if flag {
        drop(mov h)    // consumed in then-branch
    }
}
```

Note

`[E007]` (generic ownership mismatch) was the diagnostic emitted before `[E016]` was introduced. `[E016]` is now the dedicated code for branch-inconsistency on linear values.

11.13 Field-level tracking

The borrow checker tracks borrows at the field level. Borrowing non-overlapping fields of the same struct simultaneously is permitted:

```
type Pair {
    a i32
    b i32
}

func main() i32 {
    var p = Pair(1, 2)
    var ra = var p.a        // mutable borrow of p.a
    var rb = var p.b        // mutable borrow of p.b -- OK:
        non-overlapping
    ra = 10
    rb = 20
    return p.a +% p.b
}
```

> **Note**
>
> The reference implementation tracks field borrows at one level of depth. Nested field borrows (e.g., `p.a.x` vs `p.a.y`) are conservatively treated as overlapping.

Chapter 12

Functions and effects

12.1 The effect row

1 A function has one introducer, `func`, and an **effect row**: a set drawn from `io`, `diverge`, `raises` and `alloc`. The row is inferred from the body across the call graph, its default is $\emptyset$ — pure and total — and an `effects` clause widens that default for a function that deviates. The row is a *complete* upper bound: an effect the body has and the row does not name is *ill-formed*, so a caller may rely on what it reads (§ 12.2).

Property	default row ($\emptyset$)	with the effect named
Observable side effects	forbidden	`effects io`
Allocation	forbidden	`effects alloc`
`panic`	forbidden	`effects raises`
Unbounded `while`	forbidden	`effects diverge`
Bounded `while decreasing`	allowed	allowed
`for` loops	allowed	allowed
Direct recursion (with measure)	allowed	allowed
Recursion through several functions	allowed when ranked (§ 12.2)	allowed
Global mutable state	does not exist in the language	—
Termination guarantee	yes	waived by `diverge` only

2 Naming an effect is not a local suppression. It enters this function's row, which is what every caller reads, so the acknowledgement propagates outward until some function is genuinely entitled to it.

> **Note**
>
> `proc` was the form in which the introducer granted the effects instead of the row. It has been removed (§ 10.5); the row is the only mechanism.

3 A `func` is referentially transparent: calling it with the same arguments always produces the same result. Its evaluation has no observable effect on any state outside its own stack frame.

4 Local mutation through a `var T` parameter (exclusive mutable borrow) is permitted in a `func`. The borrow checker guarantees no aliasing, so the mutation is observable only through the parameter binding itself; the function remains deterministic and referentially transparent (same input borrow ⇒ same mutation pattern).

Example

```
type Counter { val i32 }

func increment(var c Counter) {  // legal: mutation is local
    c.val = c.val +% 1
}
```

[W130] advised downgrading an effect-free **proc** to a **func**. It was removed with the form it advised against: a warning whose subject cannot be written is dead weight that still has to be kept correct — and it had just been caught stating the opposite of the truth, because its eligibility test read the callee's *keyword* rather than the callee's row.

12.2 Purity enforcement

Constraint

A function that calls one whose effect row names an effect its own row does not is *ill-formed* (**[E011]**). The callee's introducer plays no part: the effect is what is refused. Effects are transitive, so this reaches a callee three layers down that a rule about direct calls would miss.

Constraint

A function that contains a **while** loop whose termination the implementation can neither verify from an explicit **decreasing** measure nor establish by inferring one (§ 9.7.2) is *ill-formed* (**[E011]**) *unless its row names* `diverge`. Naming it is not a suppression: `diverge` enters this function's row and every caller must acknowledge it in turn. A caller that does not is *ill-formed*: **[E011]** when it states no row, and **[E130]** when its written row omits `diverge`.

```
func spin(x i32) i32 effects diverge {
    var y i32 = x
    while y != 0 { y = y }
    return y
}

func g(x i32) i32 {
    return spin(x)        // ERROR [E011]: g states no row, and
        spin may not return
}
```

Constraint

A **func** that calls itself directly is permitted only if it carries a well-founded **decreasing** measure — supplied explicitly or inferred by the implementation (§ 12.3). A directly-recursive **func** with no measure and none inferable is *ill-formed* (**[E011]**); one whose measure is malformed is reported **[E091]**, and one whose measure is not provably decreasing and non-negative at every self-call is reported **[E082]**.

Constraint

A function whose row does not name `diverge`, and that lies on a recursion cycle with at least one *other* function, owes termination over that cycle. The unit is its *component*: every function it calls, directly or through others, that calls it back. The component is

accepted when one integer parameter of each of its functions — that function's *measure* — can be chosen such that

1. at every call from a function of the component to a function of the component, self-calls included, the argument passed for the callee's measure is at most the caller's measure there, and the caller's measure is at least 0 there (an unsigned type, or a guard that implies it); and
2. every cycle of calls inside the component, of any length and through any of its calls, contains at least one call at which that argument is strictly smaller.

Otherwise each function of the component is *ill-formed*, reported at its first call into the component: **[E011]**, or **[E082]** when that function has a written `decreasing` clause.

Note

The search is bounded and fails closed: a component of more than 16 functions, or with more than 256 calls among them, is refused. One parameter per function is the whole measure, so a cycle that ends only by a sum of two parameters, or by a lexicographic pair, is refused; such a function states `effects diverge`. A self-recursive function on no cycle with another keeps the rule of §12.3. A call through a function pointer is, for this purpose, a call to every function of the program whose address is taken and whose parameter count matches. An `extern` that calls back into the program is outside the proof: its row is believed, as every `extern`'s is.

```
// VERIFY: exit 0
func a(n u8) u8 {
    if n == 0 { return 0 }
    return b(n - 1)                // falls here
}

func b(n u8) u8 {
    return c(n)                    // not here: one fall per way
        round is enough
}

func c(n u8) u8 {
    if n == 0 { return 0 }
    return a(n - 1)
}

func main() i32 {
    if a(9) != 0 { return 1 }
    return 0
}
```

```
func ping(n u8) u8 {
    return pong(n)                 // ERROR [E011]: n never falls,
        so the cycle may not end
}

func pong(n u8) u8 {
    return ping(n)
}
```

> **Note**
>
> Generic functions (`func` with `type` parameters) are excluded from mutual-recursion detection until they are monomorphized.

12.3 Termination analysis

1 A `func` is guaranteed to terminate because:

1. every `while` loop carries a verified termination measure, supplied explicitly or inferred (§9.7.2);
2. `for` loops iterate over finite ranges;
3. every direct self-call strictly decreases a well-founded measure, and a recursion cycle through several functions is ranked by one measure per function (§12.2).

2 **Recursion measure.** The measure for a directly-recursive `func` is a parameter (`decreasing n`) or a difference of two parameters (`decreasing hi - lo`). When omitted, the implementation attempts to infer a single parameter that strictly decreases in *every* self-call (each self-call passes it a syntactically smaller argument, such as `n - 1` or `n / 2`). Each self-call is then verified to make the measure strictly smaller and to keep it non-negative (a base case); a call that does neither is reported `[E082]`. Inference never accepts a recursion an explicit measure could not.

3 No termination analysis is performed for a function whose row names `diverge`.

12.4 Universal Function Call Syntax (UFCS)

1 An expression of the form `x.f(args)` is desugared to `f(x, args)` before name resolution, provided `f` is not a declared field of `x`'s type. This is called Universal Function Call Syntax (UFCS).

2 **UFCS ownership**: the ownership mode of the first argument in the desugared call is determined by the declared mode of the first parameter of `f`:

- If the first parameter is `var`, the receiver is automatically mutably borrowed: `x.f()` desugars to `f(var x)`.
- If the first parameter is `mov`, the receiver is moved: `x.f()` desugars to `f(mov x)`. A *linear* receiver's move shall be written, as every transfer of a linear value is (`[E007]`): call `f(mov x)`.
- If the first parameter is shared (default), the receiver is shared: `x.f()` desugars to `f(x)`.

3 **UFCS resolution algorithm**:

1. Check if `f` is a declared field of `x`'s type. If so, this is a field access, not UFCS.
2. Search for a function named `f` whose first parameter type is compatible with the type of `x`.
3. If found, desugar the call.
4. If no matching function is found, the call is unresolved and the program is *ill-formed* (an unresolved receiver or callee is reported; a bare undeclared identifier is `[E106]`, §6.3).

Example

```
func is_empty(s u8[]) bool { return s.len == 0 }

func main() i32 {
    var buf = "hello"
    var empty = buf.is_empty()    // desugars to: is_empty(buf)
    if empty { return 1 }
    return 0
}
```

12.5 Calling conventions

The calling convention for Lain functions corresponds to the C99 calling convention of the target platform, as determined by the C compiler used for the translation output. The parameter passing rules are:

Lain parameter	C type	Semantics
shared struct `T`	`T * restrict`	pointer to the caller's value, read only
`var` struct `T`	`T * restrict`	pointer to the caller's value
`mov` struct `T`	`T`	by value (bitwise copy)
shared or `mov` primitive	`T`	by value
`var` primitive	`T*`	pointer to the caller's value
slice `T[]`	`size_t, T*`	two parameters, the length first

12.6 Return values

A return statement has the form `return <expr>`. The mode of the returned value is determined by the function's declared return type and follows the same rules as call-site argument passing.

A `return` may be prefixed with `mov` or `var`: `return mov x` makes the ownership transfer explicit, and `return var x` returns a mutable borrow (the pattern for a `func` whose declared return type is `var T`). When the return type is owned, a bare `return expr` already moves `expr`, so `mov` is optional there.

> **Note**
>
> Returning a borrow of a *local* variable is *ill-formed* (`[E010]`); the rule on references outliving their storage is stated in full at §19.3. This supersedes an earlier design (Q-009) in which explicit `mov`/`var` in return position was rejected.

12.7 Function pointers

A function used as a value has a function-pointer type, `*func(P)R` with the function's effect row (§12.1). It converts to a function-pointer type wherever a value converts: an initializer, an assignment, an argument, a return value, a struct field or a sum payload. The function shall have as many parameters as the pointer type, the same parameter and return types apart from their refinements, and an effect row contained in the pointer type's, so that it does no more than the pointer admits; otherwise the program is *ill-formed* (`[E122]`).

The function assumes its parameters' constraints on entry, and a call through the pointer relies on the pointer type's return type. So the function converts only if every argument a

call through the pointer can pass is one the function admits, and every result the function can return is one the pointer type admits; otherwise the program is *ill-formed* (**[E122]**). In particular:

- A constraint written on a parameter itself, a value constraint such as `b i32 > 0` (§13.6) or a length constraint such as `a u8[>= 4]` (§7.4.1), cannot be stated by a function-pointer type, so a function with one never converts.
- A parameter of a refined type alias (§7.2.3) is admitted when the pointer type's parameter admits no value the alias excludes. With `type Small = i32 >= 1`, a function taking `Small` converts to `*func(Small)i32`, and to `*func(Pos)i32` with `type Pos = i32 >= 1 and <= 9`, but not to `*func(i32)i32`.
- A refined return type in the pointer type shall be guaranteed by the function's return type or return constraint: `func two()i32 >= 1` converts to `*func()Small`, and a function returning a plain `i32` does not. A pointer type whose return type is not refined accepts any function, and a call through it gets that type's range.
- A function-pointer value converting to another function-pointer type obeys the same rules. A parameter or return that is itself a function pointer shall have the same type in both, its refinements and effect row included.

```
func safe_div(a i32, b i32 > 0) i32 { return a / b }

func main() i32 {
    var f *func(i32, i32) i32 = safe_div    // ERROR [E122]
    return f(10, 2)
}
```

```
type Small = i32 >= 1

func inv(b Small) i32 { return 100 / b }

func main() i32 {
    var g *func(i32) i32 = inv    // ERROR [E122]
    return g(4)
}
```

```
type Pos = i32 >= 1 and <= 9

func zero() i32 { return 0 }

func main() i32 {
    var g *func() Pos = zero    // ERROR [E122]
    return 100 / g()
}
```

To call a function with a constrained parameter through a pointer, wrap it in a function whose parameters are not constrained, which tests the condition and calls it directly. The example also converts a return constraint and an alias parameter, both of which the pointer types state:

```
// VERIFY: exit 30
type Small = i32 >= 1
type Pos = i32 >= 1 and <= 9

func safe_div(a i32, b i32 > 0) i32 { return a / b }
```

```
func div_or_zero(a i32, b i32) i32 {
    if b > 0 { return safe_div(a, b) }
    return 0
}

func inv(b Small) i32 { return 100 / b }

func two() i32 >= 1 { return 2 }

func main() i32 {
    var f *func(i32, i32) i32 = div_or_zero
    var g *func() Small = two
    var h *func(Pos) i32 = inv
    q = f(10, g())
    r = h(4)
    if q != 5 { return 1 }
    if r != 25 { return 2 }
    return 30
}
```

A function that takes a callback converts when the two callback types are the same:

```
// VERIFY: exit 6
func inc(x i32) i32 { return x +% 1 }

func apply(f *func(i32) i32, x i32) i32 { return f(x) }

func main() i32 {
    var g *func(*func(i32) i32, i32) i32 = apply
    if g(inc, 5) != 6 { return 1 }
    return 6
}
```

12.8 The panic builtin

`panic(msg)` is a builtin expression that terminates program execution immediately. Its type is *Never* (the bottom type: a subtype of every type). Because it does not return, `panic` may appear in any expression position—including as the body of a match arm whose declared type is otherwise unrelated.

A call to `panic` is the `raises` effect: it is permitted in a function whose row names `raises`, and elsewhere *ill-formed* (`[E011]`). Like every effect, it enters the row of each caller in turn.

Constraint

Q-014, S16. When `panic` is executed, the program terminates immediately via the platform abort primitive. No `defer` block is executed. No stack unwinding occurs.

Note

The reference implementation emits, for `panic(msg)`, the equivalent of `fprintf(stderr, "panic: %.*s\n", ...) ; abort()`. The message is written to the standard error stream before abort.

Example

```
func head(s u8[]) u8 effects raises {
    if s.len == 0 {
        panic("empty slice")     // Never; satisfies the return
            type
    }
    return s[0]
}
```

12.9 Function attributes

A function declaration may be prefixed with one or more attributes (§10.1). The attributes applicable to functions in version 1.0 are:

- **`[private]`** — module-private visibility (§16.5).
- **`[cold]`**, **`[hot]`** — a hint to the C compiler that the function runs rarely, or often.
- **`[noreturn]`** — the function never returns, which is checked on a body (**`[E135]`**) and believed on an **`extern`**.

Table 10.1 lists every attribute and where it applies.

Floating-point operations are never contracted. In `a * b + c` the product is rounded before it is added, as two IEEE 754 operations, in every function, so a result does not depend on whether the target has a fused multiply-add.

Constraint

No function opts out of this. The attribute **`[fast_math]`** is *ill-formed* (**`[E103]`**).

```
[fast_math]                          // ERROR [E103]: floating point
    is never contracted
func square(a f64) f64 {
    return a * a
}
```

Note

The reference implementation emits, after the includes of a module that computes in floating point, `#pragma GCC optimize ("fp-contract=off")` for GCC and `#pragma STDC FP_CONTRACT OFF` for clang; a module with no floating point gets neither. Without them, GNU C's default `-ffp-contract=fast` fuses a multiply and an add across statements under `-mfma` or `-march=native`. One case is outside the guarantee: clang given an explicit `-ffp-contract=fast` disregards the pragma, by its design.

Chapter 13

Value range constraints

13.1 Overview

1 Value Range Analysis (VRA) is the static analysis that establishes, at every program point, sound facts about the integer values — and the relationships between values — that may arise at run time. It is the mechanism by which a conforming implementation discharges the language's compile-time proof obligations, and thereby its safety guarantees and check-free code generation. It is used to:

- prove array and slice accesses in bounds (§ 7.3);
- prove the absence of division/modulo by zero (§ 8.3);
- prove the absence of arithmetic overflow at a checked operation (§ 13.10);
- verify parameter and return refinements, and propagate a return refinement to call sites (§ 13.6);
- establish the loop and recursion facts that make termination measures provable (§ 12.3).

2 **This clause specifies the *contract* of the VRA — what a conforming implementation shall guarantee — not the abstract domain or algorithm used to achieve it.** The domain and algorithm are an implementation concern; the reference design (a relational abstract interpreter over the octagon domain, run as a fixpoint over a control-flow graph) is described, informatively, in § 13.13. Any analysis satisfying the contract below conforms, whether it reasons with intervals, difference bounds, octagons, or a stronger relational domain.

> **Constraint**
>
> **Soundness.** The VRA shall compute, at each program point, an *over-approximation* of the set of concrete states reachable there: every state that can occur at run time shall be admitted by the analysis result at that point. Consequently a proof obligation the VRA reports as *discharged* holds on every execution — an accepted access is in bounds, an accepted operation does not overflow or divide by zero, and a verified refinement holds for every result. A proof obligation the implementation cannot discharge shall be reported as an error, never silently assumed (*prove-or-reject*).

> **Constraint**
>
> **Determinism and decidability.** The VRA shall be deterministic and shall terminate on every input, in time polynomial in program size. No general SMT, Presburger, or unbounded search procedure is required or permitted in the analysis core; a conforming implementation reaches its verdict by a decidable, terminating computation.

13.2 What the analysis tracks

1 The analysis maintains, at each program point, a sound over-approximation of each live integer value's possible values and — for a relational implementation — of the relationships between values (for example $i < len$, $lo \le hi$, or $o = i + j$). The per-value component is an interval $[\ell, u]$ (with $-\infty$, $+\infty$ for an absent bound, and *unknown* $= (-\infty, +\infty)$); the relational component, when present, records linear relations among values. The concrete representation is implementation-defined; only the soundness contract above is normative.

13.3 Range propagation rules

1 The following table defines the result range for binary arithmetic operations on intervals $[a, b]$ and $[c, d]$. All arithmetic is saturating: overflow wraps to $\pm\infty$.

Operation	Result range	Notes
$[a,b] + [c,d]$	$[a + c,\ b + d]$	saturating
$[a,b] - [c,d]$	$[a - d,\ b - c]$	saturating
$[a,b] \times [c,d]$	$[\min(ac, ad, bc, bd),\ \max(ac, ad, bc, bd)]$	saturating
$[a,b] \div [c,d]$, $c > 0$	$[a \div d,\ b \div c]$	integer truncation
$[a,b] \div [c,d]$, $0 \in [c,d]$	`[E015]`	division by zero
$[a,b]\ \%\ [c,d]$, $a \ge 0$, $c > 0$	$[0,\ d - 1]$	
$[a,b]\ \%\ [c,d]$, otherwise	unknown	conservative

13.4 Conditional refinement

1 After an `if x < N` test, the VRA refines the range of `x` in the then-branch to $[\ell, N - 1]$ and in the else-branch to $[N, u]$ (where $[\ell, u]$ is the range before the test).

2 At the join point after an `if`/`else`, the range is the union of the ranges from both branches.

3 When conditions are chained with `and`, refinements from the left operand are active when evaluating the right operand (§ 8.7).

4 **Equality and disequality.** A test `x == v` refines `x` to the point range $[v, v]$ in the then-branch. A disequality (the else-branch, or an explicit `x != v`) refines `x` only when v lies *at a boundary* of the range $[\ell, u]$ — to $[v + 1, u]$ when $v = \ell$, or to $[\ell, v - 1]$ when $v = u$ — since removing an interior point cannot be expressed as a single interval. In particular an unsigned `n` guarded by `if n == 0 { return }` is refined to $[1, u]$ on the fall-through, so a subsequent `a[0]` on a length-`n` slice is in bounds.

5 **Unsigned subtraction.** For $r = a - b$ at an *unsigned* result type, the relation $r = a - b$ holds over $\mathbb{Z}$ but not over the type unless $a \ge b$: the difference wraps to a large positive value otherwise. An implementation shall record no relation between r, a and b unless it can show $a \ge b$; where it can — `hi - lo` under a live `lo < hi`, or `n - 1` behind `if n == 0 { return }` — the relation is exact and may be recorded.

> **Note**
>
> Recording the $\mathbb{Z}$ relation unconditionally is unsound and not merely imprecise. Under it, `hi = mid - 1` with `mid` zero appears to give `hi` the value -1 rather than the type's maximum, and every bound derived from that relation inherits the error.

6 **Early-return refinement.** When an `if` whose then-branch exits the enclosing scope (via `return`, and provided no `break` escapes) has condition C, the code that follows the `if` is reached only when C is false; the VRA therefore refines with $\neg C$ for the remainder of the

scope. Likewise, after a `while cond { ... }` loop whose body contains no `break` at that loop's level, the VRA assumes $\neg cond$, since the loop exits only when the condition fails (e.g. after `while j > 0 { ... }`, `j` is refined to 0).

7 **Length guards.** A guard on a slice or array length — `i < a.len`, and also `a.len > 0`, `a.len >= k`, `a.len != 0` — refines the underlying length. For a fixed array `a T[N]` the length is the constant N; for a sized slice `a T[n]` it is the size parameter `n`. Thus `if a.len > 0 { a[0] }` is accepted exactly as `if n > 0 { a[0] }` would be.

13.5 In-guard semantics

1 An in-guard `idx in 0..arr.len` (§8.7) introduces the constraint $0 \leq idx < arr.len$ into the VRA's range environment. Inside the guarded body, `arr[idx]` is accepted without further proof.

2 When `idx in 0..arr.len` is the condition of a `while decreasing M` loop, the VRA infers that $arr.len - idx \geq 0$ (a valid non-negativity proof for the measure $arr.len - idx$).

13.6 Constraint annotations

1 A constraint annotation of the form `T op bound` on a parameter or return type declares an invariant that the VRA shall enforce:

- **Parameter constraint**: at every call site, the VRA shall prove that the argument satisfies the constraint.
- **Return constraint**: at every `return` statement, the VRA shall prove that the return expression satisfies the constraint.

> **Constraint**
>
> A function call where the argument range cannot be proven to satisfy a parameter constraint is *ill-formed* (**[E012]**).

2 A call through a function pointer lands each argument in the pointer type's parameter type, which is checked as every landing is (**[E086]**). A refined type there is proven at the call, as it is at a direct call.

```
type Small = i32 >= 1

func inv(b Small) i32 { return 100 / b }

func apply(f *func(Small) i32, v i32) i32 {
    return f(v)     // ERROR [E086]
}

func main() i32 {
    return apply(inv, 4)
}
```

> **Constraint**
>
> A `return` expression whose range cannot be proven to satisfy the declared return constraint is *ill-formed* (**[E086]**) — the same boundary mechanism (§13.10) that checks assignments and narrowing conversions. (A *parameter*-constraint violation at a call site is the distinct code **[E012]**, above.)

13.7 Interprocedural range propagation

1 When a call expression appears in a context where the VRA needs a range, it uses the callee's declared return constraint (if any) as a conservative approximation.

Example

```
func nonzero() int >= 1 { return 42 }

func divide(a int) int {
    return a / nonzero()    // the divisor is known >= 1, so no
        divisor diagnostic
}
```

2 A call to a function defined in another module (§ 16.1) contributes only that function's declared return type and constraint. When the callee declares no return constraint, the call expression has an unknown range. An instance of a generic function belongs to the module that defines the generic function.

3 A call to a function defined in the same module may also take a range derived from the callee's body, including one derived for the call's constant arguments. This is implementation quality and not part of this contract, so a program that depends on such a range may be refused by another conforming implementation.

Note

The boundary is the module so that changing a function's body cannot change whether code in another module is accepted. To make a range part of a function's contract, declare it, as `nonzero` does above.

Example

```
func five() i32 { return 5 }

// five is in this module, so the reference
// implementation reads its body and accepts.
// Imported from another module, it is refused.
func divide(a i32) i32 {
    return a / five()
}
```

13.8 Limits and known gaps

1 The following limitations are known and accepted:

1. A struct field is tracked like a variable from the point the function itself stores it. Its value on ENTRY — through a `var` parameter or a pointer — is unknown beyond what its declared invariant (§ 7.6) states.
2. VRA does not track ranges across loop-carried dependences with non-linear measures.
3. The analysis reasons over $\mathbb{Z}$. This is consistent with the language, because an operation that could overflow is *ill-formed* (§ 5.10) and the operators that *do* wrap or saturate are written explicitly and carry no obligation. Such an operator's result is known exactly when the analysis can show it does not wrap, and not at all otherwise.
4. Floating-point values are not tracked by VRA.

13.9 Loop body affine recap

1 For `for V in start..end` loops whose body contains assignments of the form `x = x + c` or `x = x - c` with constant step c, the implementation computes the post-loop range of x precisely:

$$x_{\text{post}} \in [x_{\text{init}} + c \cdot n_{\min},\ x_{\text{init}} + c \cdot n_{\max}]$$

where $n_{\min}$, $n_{\max}$ is the iteration count derived from the loop's range. This avoids the conservative widening that would otherwise mark x as unknown after the loop.

Example

```
var x = 0
for i in 0..10 {
    x = x + 1
}
// Post-loop: x is exactly 10 (range [10, 10]).
```

2 Patterns not matching the affine form (mixed updates in branches, non-constant step, multiple variables updated together) fall back to the conservative widening (unknown post-loop range).

13.10 Overflow-at-boundary check

1 The Phase 5 boundary check applies at every integer-type boundary where a value crosses into a narrower or different-domain target. The check sites are:

- Variable declaration (`var x T = expr`).
- Assignment to identifier, field, or indexed element (`x = expr`, `p.f = expr`, `arr[i] = expr`).
- Return value vs the function's declared return type.
- Call argument vs the parameter's declared type.
- Struct constructor field initialization.

2 At each site, if the source value's VRA range exceeds the target type's natural range, the program is *ill-formed* (**[E086]**). The diagnostic offers these resolutions: widen the target; choose an explicit overflow policy via a cast tier or operator — checked (`as?`, `+?`), wrapping (`as%`, `+%`), or saturating (`as|`, `+|`); or tighten input constraints so VRA can prove safety (§ 7.2.6).

3 The check is skipped:

- Inside an `unsafe` block;
- When the source range is unbounded (the analysis cannot rule out overflow without further information; the user must annotate);
- For integer-literal sources (literals are polymorphic across `iN`/`uN` and trusted to fit when written explicitly).

4 The wrapping/saturating operators (`+%`, `*%`, `+|`, `*|`) produce a result whose range is automatically clamped to the natural range of the operation's one type (§ 7.2.6), so they pass the boundary check unconditionally.

Example

```
var a u8 = 200
var b u8 = 100
// var bad u8 = a + b       // [E086]: range [300, 300] exceeds u8
var w u8 = a +% b            // OK: wrapping clamps to [0, 255]
var s u8 = a +| b            // OK: saturating clamps to [0, 255]
```

13.11 Sub-slice bounds

A sub-slice expression `arr[a..b]` produces a slice covering the half-open range $[a, b)$ of `arr`. The inferred type of the result is `T[b-a]`, a sized slice (§ 7.4.1) whose length is statically known. When both endpoints are compile-time integer literals, the compiler verifies $a \leq b$; if the check fails the program is *ill-formed* (**[E087]**).

When the endpoints are variables, each is range-checked against the container length (**[E085]**, the standard bounds check), and the length $b - a$ is an unsigned subtraction carrying the obligation of § 13.10: unless $a \leq b$ is established — by a guard, a refinement, or literal bounds — the sub-slice is *ill-formed* (**[E086]**). No check is emitted at run time.

```
// VERIFY: exit 0
func mid(a u8[], i usize, j usize) usize {
    if i <= j and j <= a.len {
        s = a[i..j]                // i <= j: the length j - i cannot wrap
        return s.len
    }
    return 0
}

func main() i32 {
    var t u8[4] = [1, 2, 3, 4]
    if mid(t, 1, 3) != 2 { return 1 }
    return 0
}
```

```
func bad(a u8[], i usize, j usize) usize {
    if i <= a.len and j <= a.len {
        s = a[i..j]                // ERROR [E086]: nothing says i <= j
        return s.len
    }
    return 0
}
```

Example

```
var arr i32[10] = [0,1,2,3,4,5,6,7,8,9]
var ok  = arr[2..5]     // type i32[3], half-open, length 3
// var bad = arr[5..2] // [E087]: start 5 > end 2
var empty = arr[3..3]   // OK: empty slice of length 0
```

13.12 Sized slice call-site verification

1 When a function parameter is declared with a sized slice type (§ 7.4.1), the callee assumes the parameter's length constraint at its entry, so every call site shall establish it: the argument's length shall be *proven* to satisfy the constraint, and an argument for which it is not proven is *ill-formed* (**[E087]**). The same holds for a scalar parameter's refinement (`n u32 < K`), whose call sites owe **[E012]**.

2 The length, or the refinement's right-hand side, may be an integer literal, a constant (`T[K]` with `K` a module constant), another parameter (`T[n]`), a parameter's length (`T[ref.len]`), or a sum or difference of those. One the call site cannot evaluate, because it calls a function or casts, is *ill-formed* at the call (**[E137]**): the callee would assume a fact nobody established.

Example

```
func pair_sum(arr i32[>= 2]) i32 { return arr[0] +% arr[1] }

func main() i32 {
    var one i32[1] = [42]
    // pair_sum(one)        // [E087]: len 1 < required 2
    var two i32[2] = [1, 2]
    return pair_sum(two)    // OK: len 2 satisfies >= 2
}
```

3 **Sized slice VRA entry.** At function entry, for each parameter `name T[expr]` the compiler seeds the range table with the constraint derived from the size expression:

- `T[k]` (literal): `__len_name` $\in [k, k]$.
- `T[>= k]` (literal): `__len_name` $\in [k, +\infty)$.
- `T[> k]` (literal): `__len_name` $\in [k{+}1, +\infty)$.
- `T[a.len]` (member reference): difference constraint `__len_name` = `__len_a`.
- `T[a.len + b.len]` (arithmetic): constraints `__len_a` $\leq$ `__len_name` and `__len_b` $\leq$ `__len_name`.

These constraints are available to the VRA throughout the function body (§ 13.13 shows two patterns they prove).

13.13 The reference implementation (informative)

1 This section is informative. It says how the reference compiler meets the contract above, so that the precision a program may rely on is concrete. Every example in it is compiled by the specification's gate.

2 **Domain.** An abstract interpreter over the *octagon* domain. Every tracked integer quantity — an SSA value, a local variable's cell, a field of a local struct, the length of a slice — is a dimension, and a state is a conjunction of constraints $\pm x \pm y \leq c$. That includes intervals ($x \leq c$, $-x \leq c$) and differences ($x - y \leq c$). Unary bounds are kept exactly within $\pm 2^{60}$ and only weakened beyond it, never strengthened. A table of compile-time constants gives exact values alongside.

3 **Closure.** The constraints are kept under integer *tight closure* (shortest paths, then unary bounds tightened to integers, then strengthening), so a fact implied by others is as available as one stated directly. After a step adds a few constraints to a closed state, the closure is updated through the new constraints only.

4 **Control flow.** The analysis runs to a fixpoint over the control-flow graph of the IR, refining the state along each branch by its condition, and widening at loop headers with thresholds taken from the loop's own constants. A branch the state makes impossible is dead: its obligations hold vacuously and it takes no part in a termination argument.

5 **Arithmetic.** A checked operation is modelled over $\mathbb{Z}$, which is exact because its overflow is an obligation. A wrapping or saturating operation is modelled exactly when the state shows it cannot wrap: `x +% 1` with `x` known to be 6 is 7. When it may wrap, its relation to the operands is dropped.

6 **Beyond the domain.** Facts the octagon cannot hold are computed alongside and handed back as bounds: a running total's range as the loop's trip count times its step, the trip count bounded by a length's refinement or by a measure that falls by at least $k \geq 1$ on every iteration (at most $\lceil (D - b)/k \rceil$ iterations of `while d > b` from $d = D$); a divisor's non-zero-ness, and that of the argument of `@ctz` and `@clz`, from a `!= 0` refinement or guard; a remainder's and a logical shift's bound by their operand; the range of `@movemask`.

7 **Examples** the reference implementation proves. `reverse` proves as written. `concat` needs its inputs' lengths bounded: without a bound, the parameter type's own `a.len + b.len` may overflow a `usize`, and that is refused (`[E086]`).

Example

```
func reverse(arr i32[], out i32[arr.len]) {
    var n = arr.len
    for i in 0..n {
        out[n - i - 1] = arr[i]    // n - i - 1 is in [0, n - 1]
            since i < n
    }
}

func concat(a i32[<= 1000000], b i32[<= 1000000], out i32[a.len
    + b.len]) {
    for j in 0..b.len {
        out[j + a.len] = b[j]      // j + a.len < b.len + a.len
            == out.len
    }
}
```

Chapter 14

Generics and compile-time evaluation

14.1 Overview

1 Lain implements genericity through compile-time function evaluation (CTFE). Generic types and functions accept *type parameters* (declared as `name type`), and a generic type also accepts *compile-time value parameters* (declared as `name T` with `T` an integer type). The compiler evaluates them at compile time and monomorphizes the result. There is no type erasure and no runtime polymorphism.

> **Note**
>
> **Implementation status.** The reference compiler implements the *core* of this chapter and monomorphizes it: generic `func`s with `type` parameters (bound by inference from a value argument or by an explicit type argument), and generic types written `type Name(T type){ ... }`, including compile-time value parameters of a type (const generics, `type Buf(N usize)`). The remaining constructs below are *planned* and not yet accepted — `comptime { ... }` expression blocks and the `func`-returning-`type` spelling; each is flagged at its section, and is presently reported as `[E100]`. (`comptime if`, § 14.7, *is* available.)

14.2 Type parameters

1 A parameter declared `name type` is a type parameter. Types are implicitly compile-time — the `comptime` keyword shall *not* precede a type parameter. At every call site, the argument shall be a type expression known at compile time.

```
func identity(T type, x T) T {
    return x
}

func main() i32 {
    var n = identity(int, 42)        // T = int, given; monomorphized
    var s = identity("hi")           // T = u8[:0], inferred; a separate monomorphization
    return 0
}
```

2 Type parameters are evaluated entirely at compile time; they produce no runtime code.

> **Constraint**

> The `comptime` keyword shall not appear before a parameter whose type is `type`. `func f(comptime T type)` is ill-formed; write `func f(T type)` instead.

3 The number of type parameters is a translation limit (§4.4). The reference implementation accepts at most eight in a generic type or function, and at most 64 value parameters in a generic function, and refuses a declaration with more where it is declared (**[E124]**), before any instance is made.

```
// ERROR [E124]: nine type parameters
func nine(A type, B type, C type, D type, E type, F type, G type,
    H type, I type, x I) I {
    return x
}
```

14.3 Compile-time value parameters

1 A type's parameter list may declare a *value* parameter, `N usize`: a parameter whose type is an integer type rather than `type`. An instance names its value, which shall be a constant expression (§10.8): a literal, `Buf(4)`, or a module constant, `Buf(SIZE)`. Each value is its own instance, generated once. Inside the type the parameter sizes an array field, `data u8[N]`, or bounds a slice field's length, `src u8[<= N]`, exactly as the literal would. A constructor infers the value from the length of the array it is given, or takes it first:

```
SIZE usize = 16
type Buf(N usize) {
    data u8[N]
}
assert @sizeof(Buf(4)) == 4
assert @sizeof(Buf(SIZE)) == 16
func last(b Buf(4)) u8 {
    return b.data[3]                          // b.data.len is 4:
        proven
}
func main() i32 {
    b = Buf([1, 2, 3, 4])                     // N = 4, from the
        array
    c = Buf(8, [0, 0, 0, 0, 0, 0, 0, 9])      // N = 8, given
    return 0
}
```

> **Constraint**
>
> The argument for a value parameter shall be a non-negative constant expression; otherwise the program is *ill-formed* (**[E124]**). An array given for a field the parameter sizes shall have that length (**[E012]**).

> **Constraint**
>
> A compile-time evaluation context (CTFE) shall not call a function whose effect row is non-empty; only calls with an empty row are admissible. A violation is *ill-formed* (**[E101]**).

14.4 Type as value

1 A type declared with type parameters is a type constructor: `Option(int)` names the instance for `int`, which is generated once (monomorphized) and may be used in type annotations and

variable declarations. A variant of an instance is constructed as `Option(int).Some(42)` where the declaration states the instance. A generic struct takes its type argument first, as a value parameter does: `Box(i32, 7)` for `type Box(T type){ v T }`, or `Box(x)`, which infers `T` from the field.

```
type Option(T type) {
    Some { value T }
    None
}

func main() i32 {
    var x Option(int) = Option(int).Some(42)    // the variable states the instance
    return 0
}
```

Type constructors may also be written as `func` declarations returning `type`:

> **Note**
>
> **Planned (not yet implemented).** The `func`-returning-`type` spelling below is presently `[E100]`; use the `type Name(T type){ ... }` form above, which is accepted. (The spelling `type Name(T type)= type { ... }` declares an alias, not a generic type: its instances are refused, `[E124]`.)

```
// ERROR: E100 -- the planned spelling, not yet accepted
func Option(T type) type {
    return type {
        Some { value T }
        None
    }
}
```

> **Note**
>
> **Not yet in force.** This implementation does not parse a function whose return type is `type`: `func Mk(T type)type` is refused at the return type (`[E100]`). A generic type is declared directly, as `type Option(T type){ ... }`, which is the form the standard library uses. The constraint below is retained because the form is intended, and marked because nothing currently checks it.

> **Constraint**
>
> A function returning `type` shall have an empty effect row: a type-level value cannot depend on an effect.

14.5 Monomorphization

For each unique combination of comptime arguments at call sites, the implementation produces a separate instantiation of the function. The name of each instantiation in the translation output is *implementation-defined*[1].

Monomorphization is a compile-time-only operation; at runtime, each call site calls a specific, non-generic function.

[1] Implementation-defined behavior: determined by the implementation

14.6 Comptime expression blocks

1 A `comptime { expr }` expression evaluates *expr* at compile time. The result is a compile-time constant that may be used as a type annotation, array size, or parameter constraint value.

> **Note**
>
> **Planned (not yet implemented).** `comptime { ... }` expression blocks are not accepted by the reference compiler (presently `[E100]`). The statement form `comptime if` (§14.7) *is* available.

> **Constraint**
>
> A `comptime` expression shall be pure and terminating (i.e., it may only use `func` calls and the constructs permitted in `func` bodies).

14.7 Comptime if

1 `comptime if cond { ... } else { ... }` is a compile-time conditional. Only the taken branch is type-checked and emitted; the other branch is entirely discarded. *cond* shall evaluate to a compile-time boolean constant.

> **Note**
>
> `comptime if` enables platform-specific code selection without dead-code warnings. It is the preferred way to write code that varies based on compile-time parameters.

14.8 Comptime recursion depth

1 The planned compile-time evaluation engine (§14.3, §14.6) will bound the recursion depth of comptime function evaluation *implementation-defined*[2] and reject an evaluation that exceeds it. Since that engine is not yet implemented, no depth diagnostic is currently emitted.

14.9 Mode preservation in generics

1 When a type parameter `T` is substituted with a concrete type, the ownership mode of the parameter is preserved. If a parameter was declared `var T`, and `T` is substituted with a type that has mode `shared` in its declaration, the substituted parameter retains the `var` mode.

> **Note**
>
> This rule ensures that mode annotations in generic functions behave consistently regardless of the concrete type argument.

[2] Implementation-defined behavior: by an implementation-defined limit

Chapter 15

Pattern matching

15.1 Overview

The `case` expression matches a value against a sequence of patterns and evaluates the body of the first matching arm. Every `case` expression shall be exhaustive: the patterns shall collectively cover all possible values of the scrutinee type. The grammar production is *match-expression* (Annex A).

15.2 Syntax

```
// SYNOPSIS: the form of a case expression
case expr {
    Pattern1: body1
    Pattern2: body2
    else:     default_body
}
```

Each arm consists of a pattern (or comma-separated list of patterns) and a body. The arms are tested in declaration order; the first matching arm is taken.

In a `case` *statement*, an arm's body is one or more statements, each ending at a newline, running from the `:` to the next arm's pattern or to the closing brace. The body is not enclosed in braces: a `{` after the `:` is not the start of a block, and does not parse. In a `case` *expression* (§15.6), an arm's body is a single expression.

```
// VERIFY: exit 0
func classify(x i32) i32 {
    var y = 0
    case x {
        1, 2:
            y = 10            // a body of two statements, one per
                line
            y = y + 1
        else: y = 2
    }
    return y
}

func main() i32 {
    if classify(2) != 11 { return 1 }
    if classify(7) != 2 { return 2 }
    return 0
}
```

15.3 Patterns

1 The following pattern forms are supported:

Wildcard (`else`) Matches any value. Must be the last arm. Required for integer and character scrutinees.

Literal pattern Matches a specific integer or character value: `42:`, `'a':`.

Enum/ADT variant pattern Matches a specific variant. If the variant has fields, they are bound: `Circle(r): ...` binds the radius to `r`.

Range pattern Matches a range of integer or character values. Consistent with expression ranges and sub-slices (§5.8, §8.14), `a..b` is *exclusive* of the upper bound and `a..=b` is *inclusive*: `1..10:` matches 1 through 9, and `1..=10:` matches 1 through 10.

Multi-pattern (OR) A comma-separated list of patterns on a single arm: `1, 2, 3: ...`.

> **Constraint**
>
> The scrutinee shall be an integer (a `u8` character included), a `bool`, an enum or ADT, or a string (`u8[]`, §15.7), and every pattern shall have its scrutinee's type: a string scrutinee takes string literals and nothing else does. Any other scrutinee or pattern is *ill-formed* (`[E012]`).

> **Constraint**
>
> A variant pattern shall name a variant of the scrutinee's own enum or ADT, bare (`Red`) or qualified by that type (`Color.Red`); any other name is *ill-formed* (`[E106]`).

15.4 Exhaustiveness

1 A `case` expression is exhaustive if the set of patterns covers all values of the scrutinee type. The rules by type are:

Scrutinee type	Exhaustiveness requirement
Enum	All variants covered, or `else` arm present
ADT	All variants covered, or `else` arm present
Integer	`else` arm always required
`u8` (char)	`else` arm always required
`bool`	`true` and `false` covered, or `else`
String (`u8[]`)	`else` arm always required

> **Constraint**
>
> A non-exhaustive `case` expression is *ill-formed* (`[E014]`).

> **Example**

```
type Color {
    Red
    Green
    Blue
}

```

```
func code(c Color) i32 {
    case c {
        Red:   return 1
        Green: return 2
        Blue:  return 3
    }  // exhaustive: all three variants covered
}
```

15.5 Ownership and match

By default, `case expr` may consume the scrutinee if it is a linear type. Bindings introduced by variant patterns in consuming arms hold ownership of the bound values.

`case &expr` performs a non-consuming match: the scrutinee is not consumed but instead temporarily borrowed for the duration of the match body (see § 11.10).

> **Constraint**
>
> Inside a `case &expr` body, the scrutinee shall not be mutated or moved (**[E004]**).

15.6 Case expression result type

When `case` is used as an expression, all arms shall produce values of compatible types (§ 7.12). The type of the `case` expression is the common type.

> **Constraint**
>
> Arms of a `case` expression that produce incompatible types are *ill-formed* (**[E012]**).

15.7 String pattern matching

String literals may appear as patterns. A string pattern matches if the scrutinee has the same byte sequence and length as the pattern literal. The comparison is performed byte-by-byte (analogous to `memcmp`).

```
// VERIFY: exit 0
func kind(s u8[]) i32 {
    return case s {
        "red": 1
        "blue": 2
        else: 0
    }
}

func main() i32 {
    if kind("blue") != 2 { return 1 }
    if kind("blu") != 0 { return 2 }      // a prefix is not a
        match
    if kind("blues") != 0 { return 3 }    // nor is a longer string
    return 0
}
```

> **Note**

String pattern matching in the translation output is implemented as a sequence of byte comparisons or `memcmp` calls, depending on the implementation. Its asymptotic cost is O(n) in the pattern length.

Chapter 16

Module system

16.1 Overview

1 Each Lain source file defines one module. Modules are the unit of code organization and compilation. All names declared at the top level of a source file are in that module's scope and are accessible from other modules that import the module. A module is also the boundary for the value ranges that cross a call (§ 13.7).

16.2 Module identity

1 A module's identity is *implementation-defined*[1]; there is *no* `module` declaration keyword (a leading `module ...` line is `[E100]`). The reference implementation names a module by its path relative to the working directory, with directory separators mapped to dots — so `std/io.ln` is the module `std.io`.

16.3 Import declarations

1 `import module.path` makes the declarations of the named module available in the current source file through the module's qualifier (below): `import std.math` makes `math.max` available, not `max`. The grammar production is *import-declaration* (Annex A).

2 **Selective import.** `import module.path.{name1, name2, ...}` also brings the listed names into scope unqualified; no other name of the module is available unqualified. This is the form used throughout the standard library, e.g. `import std.mem.{mem_alloc, mem_free}` or `import std.fs.{File, open_file, close_file}`.

> **Constraint**
>
> Each name in an import list shall be defined by the imported module or by a module it imports, directly or through further imports; any other name is *ill-formed* (`[E106]`). Neither the order of the import declarations nor an import cycle changes which names a module reaches.

3 **Qualified access.** Every import declaration, with or without an import list, also declares a *qualifier* for the imported module in the importing source file: the last segment of the module path, or the identifier after `as` (`import std.io as o` makes `o.println` available, and not `io.println`). `q.name` names the declaration `name` of the module that `q` qualifies.

> **Constraint**
>
> In a qualified name `q.name`, `q` shall be a qualifier declared by an import in the same source file, and `name` shall be defined by the module it qualifies or by a module that module

[1] Implementation-defined behavior: derived from its file path by the implementation

imports, directly or through further imports; otherwise the program is *ill-formed* (`[E106]`). A qualified name never names a local.

4 The `use` statement (§ 5.5) provides a *local-scope* import/alias inside a function body. *implementation-defined*[2]; the standard library uses top-level `import` rather than `use`.

Note

What this implementation does. It refuses the statement at the `use` keyword, before resolving the target, so the reader learns that the construct is unavailable rather than that their module path is wrong (`[E100]`): "this construct is not supported by the code generator yet (a `use` statement in a function body)". Both the local-alias and the module-alias form give that same refusal.

Refusing before resolution is deliberate. Until I.91 the module form was refused *during* resolution, by `[E106]` complaining that `std` was not a value, which told a reader to fix a path that was not the problem. An unmodelled construct should be reported as unmodelled, wherever it appears.

`use` is accepted by the parser only inside a function body; at top level it is not a declaration form.

Refusing is itself the fix for a defect worth recording, because it shows what an unmodelled construct costs when it is merely skipped. Lowering previously ignored the statement and marked the containing function *incomplete*, and the function was then emitted with every check suppressed — so an out-of-bounds access in the same body compiled and ran. An implementation that does not model a construct must reject the program, not emit the rest of the function without its obligations.

5 **Path resolution**: a module path is converted to a file system path by replacing dots with directory separators and appending `.ln`. *implementation-defined*[3]: the reference implementation searches from the working directory when the source file is named by a relative path, and from the source file's own directory when it is named by an absolute one.

Module path	File path
`std.c`	`std/c.ln`
`std.io`	`std/io.ln`
`std.fs`	`std/fs.ln`
`std.math`	`std/math.ln`
`mylib.utils`	`mylib/utils.ln`

16.4 Module loading

1 When the compiler encounters an `import` declaration:

1. The module path is converted to a file path.
2. If the module has already been loaded, the import is a no-op (deduplication).
3. Otherwise, the file is read, lexed, parsed, and recursively resolved.

[2]Implementation-defined behavior: Its precise semantics are implementation-defined in this revision

[3]Implementation-defined behavior: The exact search algorithm, including support for search paths, is implementation-defined

4. The loaded module's declarations are merged into the importing module's declaration list.

2 Each module is loaded at most once. Circular imports are therefore implicitly handled: if module A imports module B which imports module A, the second import of A is a no-op because A is already loaded.

> **Note**
>
> The reference implementation maintains a linked list of loaded modules keyed by their dot-separated name.

16.5 Name visibility

1 By default, every name declared at module scope is *public*: visible to any module that imports the declaring module. This default matches the typical usage where module declarations form an API surface.

2 A declaration prefixed with the attribute `[private]` (§10) is *module-private*: it is visible only within the defining module and may not be referenced from any importing module.

```
// In module foo.bar:
func public_api() int { return internal_helper() }   // public by default

[private]
func internal_helper() int { return 0 }   // only foo.bar may reference this
```

> **Constraint**
>
> **Q-018.** A reference to a `[private]` declaration from a module other than the one where it is defined is *ill-formed* (`[E084]`). The reference may appear in any expression position—identifier lookup, member access, type instantiation, etc.

> **Note**
>
> The reference implementation emits a whole program as one C translation unit, so `[private]` has no C counterpart: the compiler enforces it (`[E084]`), and private and public declarations alike are emitted with external linkage.

16.6 Standard library module paths

1 The following module paths are defined by this specification for the standard library:

Module path	Description
`std.c`	C standard library bindings
`std.io`	Console I/O
`std.fs`	File system operations
`std.math`	Mathematical functions

16.7 Name mangling

1 The name of a Lain declaration in the translation output is *implementation-defined*[4]. The reference implementation uses the function name prefixed with the module path, with dots replaced by underscores. Name collisions between modules are *implementation-defined*[5].

[4]Implementation-defined behavior: determined by the implementation

[5]Implementation-defined behavior: detected by the implementation; the reference implementation appends a numeric suffix to resolve conflicts

Chapter 17

C interoperability

17.1 Overview

1 Lain compiles to C99 and provides direct, zero-overhead interoperability with C libraries through two mechanisms:

1. `extern` declarations — declare C functions and opaque C types for use in Lain;
2. type mapping — Lain types map to their C99 counterparts.

17.2 C headers

1 A Lain program includes no C header. Each C function it calls is declared by an `extern` declaration (§ 17.3), and the translation output declares that function with the prototype the `extern` states, in Lain's types. A header would declare the same function again, in C's types, and a C compiler refuses the pair when they differ: an `extern` taking `*u8` declares `uint8_t *` where `<stdio.h>` declares `const char *`. The rest of a header, its macros and constants, cannot be named from Lain.

> **Constraint**
>
> `c_include` is a reserved word. A `c_include` declaration is *ill-formed* (`[E100]`).

```
c_include "<stdio.h>"       // ERROR [E100]

func main() i32 {
    return 0
}
```

17.3 Extern function declarations

1 `extern func name(params)return` declares a C function. Because an extern has no body, its effect row cannot be inferred; the row is therefore *believed*, and its default is the set of *all* effects (`io`, `diverge`, `raises`, `alloc`). An `effects` clause on an extern *narrows* that default.
2 This is the inverse of a declaration with a body, whose default row is ∅ and whose clause *widens* it (§ 10.4). The direction differs for one stated reason: inference is available in the one case and impossible in the other. A caller of an extern acknowledges whatever the extern's row names, by the ordinary rule.

> **Constraint**

An `extern` with no `effects` clause has the row {`io`, `diverge`, `raises`, `alloc`}. A caller whose own bound does not cover it is *ill-formed* (`[E011]`).

Note

Trust boundary. An extern's `effects` clause is a *programmer assertion* that the referenced C function does no more than the row names. The compiler cannot verify it and believes it, so a wrong narrowing—declaring `libc_rand` with an empty row, say—may invalidate the guarantees that depend on purity (notably P4 determinism for any function that transitively calls it). What has changed is the cost of *silence*: omitting the clause once meant "trusted pure" (the rule once labelled I-016), so forgetting it on a C function that performs I/O was a silent fail-open. Omitting it now claims everything, so forgetting costs precision instead. The empty row, `effects` with no names, is how purity is asserted deliberately.

3 Ownership annotations on extern parameters work identically to those on native Lain functions; the borrow checker tracks ownership of resources obtained via extern calls.

17.4 Type mapping

1 The following table defines the normative mapping between Lain types and their C99 equivalents. The mapping is the ABI used by the translation output.

Lain type	C99 type	Notes
`u8`	`uint8_t`	
`u16`	`uint16_t`	
`u32`	`uint32_t`	
`u64`	`uint64_t`	
`usize`	`uint64_t`	the pointer width in this specification (§ 7.2.1); 64 bits in the reference implementation
`i8`	`int8_t`	
`i16`	`int16_t`	
`i32`	`int32_t`	
`i64`	`int64_t`	
`isize`	`int64_t`	as `usize`
`int`	`int32_t`	the alias of `i32`
uN, iN (other N)	the narrowest of the above that holds it	e.g. `u3` is `uint8_t`
`f32`	`float`	IEEE 754
`f64`	`double`	IEEE 754
`bool`	`_Bool`	C99 bool (1 byte); *implementation-defined*[1]
shared struct `T`	`T * restrict`	a parameter
`var` struct `T`	`T * restrict`	a parameter

[1] Implementation-defined behavior: 0 or 1

Lain type	C99 type	Notes
`mov` struct `T`	`T`	by value
`T[]`	`struct { T* data; size_t len; }`	passed as two parameters, the length first; not in an `extern` (`[E131]`)
`*T`	`T*`	read-only: the compiler refuses a write (`[E009]`)
`*var T`	`T*`	writable

17.5 ABI for function parameters

1 This table is normative and applies to all Lain function calls, including those to `extern` functions.

Lain param	Type	C param	Direction
`name T`	struct	`T * restrict`	in
`var name T`	struct	`T * restrict`	in/out
`mov name T`	struct	`T`	in (by value)
`name T`, `mov name T`	primitive	`T`	in
`var name T`	primitive	`T*`	in/out
`name T[]`	slice	`size_t`, `T*`	in (length, then data)

17.6 Pointer operations

1 Reading or writing through a raw pointer (`*p`, and a field access through one) requires an `unsafe` block (see § 18). Taking an address, `&x`, does not (§ 7.10).

> **Constraint**
>
> A cast to or from a raw pointer type — pointer to pointer, integer to pointer, or pointer to integer — shall occur only inside an `unsafe` block (`[E012]`).

17.7 String interop

1 Lain strings (`u8[]`) are byte slices, not null-terminated C strings. To pass a Lain string to a C function expecting `char*`, the `.data` pointer must be extracted explicitly:

```
extern func puts(s *u8) int effects io

func main() int effects io {
    var s = "hello"
    puts(s.data)    // OK: .data is *u8 = const char*
    return 0
}
```

2 String literals are null-terminated (§ 5.7.5), so `literal.data` is a valid `const char*` for C functions that expect a C string.

17.8 Opaque C types

1 An opaque C type is declared with `extern type Name`. The type's layout is unknown to Lain; it may only be used behind a pointer.

```
extern type FILE
extern func fopen(path *u8, mode *u8) mov *FILE effects io, alloc
```

Constraint

An opaque type shall not be used as a value type (only as a pointer target): a variable, a parameter, a field, a module constant or a return value of an opaque type is *ill-formed* (`[E012]`), as is `var f FILE`.

Chapter 18

Unsafe code

18.1 Purpose

1 The `unsafe` block provides a controlled escape hatch for operations that a conforming implementation cannot statically verify. Unsafe blocks are explicitly delimited in the source text and are therefore auditable. Unsafe code cannot accidentally spread outside the enclosing block.

18.2 Syntax

1 An unsafe block is written as `unsafe { body }`. It may appear wherever a statement is valid (§ 9.10).

18.3 What unsafe enables

1 Inside an `unsafe` block, the following operations are permitted:

1. **Raw pointer dereference**: `*ptr` reads the value pointed to by a raw pointer; `*ptr = v` writes through a writable raw pointer `*var T`. A `*T` is read-only: a write through it — `*p = v`, `p.f = v`, `@store(p, i, v)` — is *ill-formed* here too (`[E009]`), and a `*T` does not convert implicitly to a `*var T` (`[E012]`). A `*T` becomes writable only by an explicit cast inside `unsafe`, `p as *var T`, which asserts that writing through it is valid.

2. **Indexing**: a raw pointer has no length and cannot be indexed (`[E012]`); read through it with `*p`, or `@load` a vector.

3. **Pointer and integer casts**: `p as *T`, `n as *T`, `p as usize`, etc., except a cast that views the bytes of a struct, or of a sum's payload, whose storage order the compiler chooses, which stays *ill-formed* (`[E012]`; § 7.6): mark it `[ordered]`.

4. **Direct ADT field access**: reading a variant's field with the qualified form `value.Variant.field` without going through a `case` arm. Outside `unsafe` this is *ill-formed* (`[E125]`); the safe idiom is to destructure with `case`.

5. **Bounds-check suppression**: array indexing inside an `unsafe` block does not require a VRA proof (the programmer asserts the access is in bounds).

6. **Numeric-obligation suppression**: an integer operation inside an `unsafe` block does not require a proof that it cannot overflow or that a narrowing fits (`[E086]`), that a divisor is not zero (`[E015]`), or that a shift amount is below the width. A narrowing that does not fit, whether an explicit `as` or the implicit one where a value is stored, passed, returned or placed in a constructor, yields the value of the target type congruent

to it modulo 2^N, where N is the width of the target type and not of its storage: for an unsigned target the value modulo 2^N, for a signed one that value read in two's complement. For an unsigned target of 8, 16, 32 or 64 bits this is C17's conversion; `x as u12` keeps 12 bits, which no C type does. The analysis of the code after the block reasons from that value: with `var y u16 = 260`, `unsafe { x = y as u8 }` leaves `x` equal to 4.

2 The index below is proven from the wrapped value, 4, in the safe code after the block:

```
// VERIFY: exit 0
func main() i32 {
    var a i32[8] = [1, 2, 3, 4, 5, 6, 7, 8]
    var y u16 = 260
    var x u8 = 0
    unsafe {
        x = y as u8
    }
    if a[x] != 5 { return 1 }    // a[4], and the index needed no
        bounds check
    return 0
}
```

3 The width is the type's, so the value never leaves the type: 70000 is 368 modulo 2^{12}, and a `u12` holds 368, which safe code after the block may use as an index into a 4096-element array.

```
// VERIFY: exit 0
func main() i32 {
    var y u32 = 70000
    var x u12 = 0
    unsafe {
        x = y as u12
    }
    if x != 368 { return 1 }
    return 0
}
```

4 Taking an address is not among them: `&x` is permitted in safe code (§ 7.10), since making a pointer is harmless and only reading or writing through it is the danger. It yields a `*var T` when the program may write `x` (a `var` binding or parameter, an element or field of one, or what a `*var` pointer points to), and a `*T` otherwise.

18.4 What unsafe does NOT disable

1 The following safety features remain active inside `unsafe` blocks:

Feature	Disabled by `unsafe`?
Pointer dereference (`[E060]`)	**Yes**
Pointer/integer casts (`[E012]`)	**Yes**
Direct ADT field access (`[E125]`)	**Yes**
Bounds check requirement (`[E085]`)	**Yes**
Ownership tracking (`[E001]`–`[E007]`)	No
Borrow checking (`[E004]`)	No
Move semantics	No

Feature	**Disabled by `unsafe`?**
Type checking	No
Overflow and narrowing check (`[E086]`)	**Yes**
Division-by-zero check (`[E015]`)	**Yes**
Exhaustiveness checking (`[E014]`)	No
Struct field invariants (`[E121]`)	No
Narrowing into a refined type (`[E086]`)	No
Purity enforcement (`[E011]`)	No

2 The two halves of that table are separated by one principle rather than by a list of exceptions, and the principle is worth stating because the rows that look alike are not. A numeric obligation is *local*: `unsafe` accepts responsibility for one operation, that operation then has a defined result — the value modulo 2^N, as § 18.3 gives it — and the analysis of everything after it proceeds from that result. Nothing elsewhere in the program was relying on the obligation having been discharged, so the consequence of waiving it reaches no further than the value it produced.

3 A struct field's invariant (§ 7.6.1) and a refined type's constraint (§ 7.2.3) are not obligations of that kind; they are *premises*. The check exists so that every later read may assume the property without proving it, and an implementation is permitted to compile a read that contradicts one into unreachable code. Suspending such a check inside `unsafe` would therefore not confine its consequence to the block: it would make deductions false in *safe* code elsewhere, which the programmer asserted nothing about and could not have, since no assertion available at the `unsafe` block speaks about a later function's reasoning. An assignment that would break an invariant, or that would land outside the refinement of the type it is stored into, is therefore *ill-formed* inside `unsafe` exactly as it is outside it (`[E121]`, `[E086]`).

4 So: `unsafe` waives a *proof about an operation*; it never waives a *fact the rest of the program is entitled to assume*. A reader deciding whether some future check belongs above or below the midrule of that table should ask which of those two it is.

> **Constraint**
>
> Ownership and linearity rules (§ 11) are not suspended inside `unsafe` blocks. Use-after-move, double-move, and borrow conflicts are still constraint violations inside `unsafe` (`[E001]`–`[E007]`).

18.5 Undefined behavior in unsafe

1 The following operations inside an `unsafe` block may produce **undefined behavior**[1]:

- Dereferencing a null pointer.
- Dereferencing a pointer to freed or uninitialized memory.
- Writing to memory through a pointer derived from a read-only (shared) source.
- Accessing an array element with an out-of-range index.
- Dividing or taking a remainder by zero, shifting by an amount outside $[0, N)$ for an N-bit operand, or a signed arithmetic result that does not fit its type (operands narrower than 64 bits are computed wider, so only a 64-bit one can). That includes a 64-bit

[1] Undefined behavior: undefined behavior if the programmer's safety assertion is incorrect

remainder `x % -1` with `x` the type's minimum: its result, 0, fits, but the quotient it is computed with does not.

- Accessing a field of an ADT through a tag that does not match the active variant.
- Writing into an enumeration's storage a value that is none of its variants (`*(&k as *var u8)= 3` over `type K u8 { A, B, C }`): a sum that stores its payload-less variants in the enumeration's spare values reads it as one of those (§ 19.9).
- Writing into a tagged sum's tag a value that is none of its variants: no `case` arm matches it.
- Writing, through a raw pointer, a value that does not satisfy a refined type's predicate or a struct field's invariant. This is the same kind of violation as the two items above, for the same reason: the predicate and the invariant are premises (§ 18.4), and every later read is entitled to assume them without proof. With `type D { v i32 >= 0 and <= 3 }`, the write `*(&d.v as *var i32)= 100` makes a subsequent `t[d.v]` read outside `t i32[4]` although the index was proven in bounds.

2 The rule is about the *value*, not about the pointer. Writing a value that does satisfy the predicate is well defined, and the reads that follow remain correct, so a raw pointer into refined storage is not in itself an error:

```
// VERIFY: exit 0
type D { v i32 >= 0 and <= 3 }

func main() i32 {
    var t i32[4] = [10, 20, 30, 40]
    var d = D(1)
    unsafe {
        var q = &d.v as *var i32
        *q = 2                    // satisfies the predicate: well
            defined
    }
    if t[d.v] != 30 { return 1 }
    return 0
}
```

3 A conforming implementation that generates C99 translation output inherits the undefined behavior rules of ISO/IEC 9899 (C17) for operations within `unsafe` blocks.

18.6 Borrow scope for unsafe blocks

1 An `unsafe` block is a regular block scope for the purposes of borrow checking. Borrows registered within the block are released on block exit, and the borrow checker's invariants hold at the block boundaries.

18.7 Nesting

1 `unsafe` blocks may be nested. The inner block does not extend the permissions of the outer block beyond what is already enabled. *implementation-defined*[2].

[2]Implementation-defined behavior: Whether the implementation issues a diagnostic for redundant nested unsafe blocks is implementation-defined

Chapter 19

Memory model

19.1 Overview

1 Lain's memory model is founded on three invariants:

1. **Deterministic allocation** — every allocation is explicit and predictable; the implementation never implicitly allocates heap memory.
2. **Zero runtime overhead** — there is no garbage collector and no reference-counting mechanism.
3. **C99 layout compatibility** — the physical layout of all objects follows ISO/IEC 9899 (C17) rules exactly.

2 This clause specifies the abstract notions of *object*, *storage duration*, *object representation*, and *address*. The abstract machine described here is realised by any conforming translation output (§ 4).

19.2 Objects

1 An *object* is a contiguous region of storage that holds a value of a declared type. Every variable declaration introduces exactly one object. Fields of a struct introduce sub-objects. Elements of a fixed-size array introduce element sub-objects.

2 Every object has:

- a *declared type* (§ 7);
- an *ownership mode* — shared, `var`, or `mov` (§ 11.1);
- a *linear state* — Uninit, Unconsumed, or Consumed (§ 11.2);
- a *storage duration* (§ 19.3);
- an *address* — the byte address of its first storage unit, aligned as required by the target platform.

19.3 Storage duration

1 Three storage durations are defined:

Automatic duration Objects declared inside a function or block body (including function parameters) have automatic storage duration. Storage is allocated when the enclosing scope is entered and released when that scope exits.

Static duration Objects declared at module scope have static storage duration. Their storage persists for the entire program execution. Every such object is a constant whose value is computed at compile time (§ 10.7); there are no module-scope variables.

Heap (explicit) duration Objects allocated through C interop (`malloc` or equivalent) have programmer-controlled duration. The ownership system (§ 11) enforces that `mov` pointers to heap memory are consumed exactly once (`[E003]`).

> **Constraint**
>
> A reference whose referent has automatic storage duration shall not outlive that storage (`[E010]`). It shall not be returned, whether directly or held by a struct or a sum, through any binding, assignment or call that can hand it back; it shall not be stored into a place the caller owns (what a `var` parameter refers to, or a place reached through it); and it shall not be passed to a function that keeps it where such a place can reach it. An array passed to an array field or payload is copied into it and lends nothing. A function declared `extern` is believed to keep no reference past the call.

19.4 Stack allocation

1 All local variables, struct values, and fixed-size arrays are allocated on the call stack. No implicit heap allocation is ever performed by the compiler.

```
type Point { x i32, y i32 }

func main() i32 {
    var x int = 42       // stack: sizeof(int32_t) bytes
    var arr int[100]     // stack: 100 * sizeof(int32_t) bytes
    var p Point          // stack: sizeof(Point) bytes
    return 0
}
```

2 Each function call produces a *stack frame* containing:

- all local variables declared in the function;
- all function parameters (by value or by pointer, per the calling convention of § 12.5);
- compiler-generated temporaries required for expression evaluation.

3 The stack frame is automatically reclaimed when the function returns. *implementation-defined*[1].

19.5 Heap allocation

1 Heap allocation is performed exclusively through C interop (§ 17). The idiomatic pattern declares the C `malloc`/`free` pair as an `extern func` with its row:

```
extern func malloc(size usize) mov *void effects io, alloc
extern func free(ptr mov *void) effects io
```

2 Because `malloc` returns `mov *void`, the caller receives sole ownership of the allocated block. Because `free` consumes `mov *void`, calling `free` transfers that ownership to the allocator. The ownership system therefore enforces statically:

[1] Implementation-defined behavior: The implementation's behaviour when the call stack is exhausted is implementation-defined; the reference implementation inherits the platform's stack overflow signal

- **No use-after-free**: using the pointer after `free` is a use-after-move (`[E001]`).
- **No double-free**: a second call to `free` on the same pointer is a double-consume (`[E002]`).
- **No leak**: failing to call `free` on a `mov` pointer is an unconsumed linear value (`[E003]`).

19.6 Value semantics

1 The behaviour of assignment and function argument passing is determined by the type's linearity and the ownership mode:

Type	Assignment `b = a`	Notes
Primitives (`int`, `bool`, ...)	Bitwise copy	Always allowed
Non-linear struct	Field-by-field copy	Allowed
Linear struct (has `mov` field)	Forbidden	Requires `mov`
Non-linear array	Element-wise copy	Allowed
Linear array	Forbidden	Requires `mov`
Slice (`T[]`)	Copy `{data, len}` pair	Always allowed
Raw pointer (`*T`)	Copy address	Always allowed

Constraint

A linear value cannot be implicitly copied: transferring it (by assignment, argument, or return) without an explicit `mov` is an implicit move and is *ill-formed* (`[E007]`, § 11.3).

19.7 Object representation

1 The *object representation* of a Lain type is its C99 equivalent as defined in § 17.4. Every object is a sequence of `u8` bytes whose count equals the C99 `sizeof` of the corresponding C type.

2 *Equality.* `==` and `!=` compare integers and `bool`s by value, raw pointers by address, and a slice against a string literal byte by byte, lengths included (§ 8.4). No other type has them — a struct, an array, another slice, an enum, an ADT or a union is `[E012]`: compare its fields, or match it with `case`.

19.8 Alignment and layout

1 Lain inherits ISO/IEC 9899 (C17) alignment and padding rules:

- Struct fields are laid out in the storage order of § 7.6: by decreasing alignment, or in declaration order for a struct that is `[ordered]`, `[packed]` or reachable from an `extern` signature. A tagged sum's payloads follow the same rule, decided by the sum (§ 19.9).
- Each field is aligned to its natural alignment (the alignment of its C99 equivalent type).
- The compiler inserts padding between fields as necessary.
- The total size of a struct is rounded up to a multiple of its largest field alignment.

2 Lain provides one layout control annotation: `[packed]`, which lays a struct of integer fields out bit-exactly inside one integer (§ 7.2.4). There is no `#pragma pack` equivalent and no `alignas`; every other struct has the platform-default C99 layout.

implementation-defined[2].

3 `@sizeof(T)` and `@alignof(T)` are expressions of type `usize` whose values are the size and alignment of `T` as laid out for the target: the C99 `sizeof` and `_Alignof` of the type the implementation emits for `T`. A layout claim written as a comment is not checked; one written as a module-scope `assert @sizeof(Token)== 8` (§ 10.8) is, every time the program is built.

19.9 ADT and enum layout

1 A *plain enumeration* — an enum type none of whose variants carries a payload — is emitted as the smallest unsigned integer type that holds its variants: `uint8_t` for up to 256 variants, `uint16_t` for up to 65 536, `uint32_t` otherwise. A backing width (§ 7.7) replaces that choice: `type K u16 { A, B }` is a `uint16_t` however few variants it has. The variants are the values $0, 1, \ldots, n-1$ in declaration order. Each enum type is still nominally distinct in the language; the C translation names it with its own `typedef`:

```
typedef uint8_t Color;            // Red = 0, Green = 1, Blue = 2
```

2 Because a struct's fields are stored by decreasing alignment (§ 19.8), a narrow enumeration's byte goes after the wider fields whatever the order they are written in: `{kind Kind, pos u32, len u16}` is 8 bytes, as `{pos u32, len u16, kind Kind}` is. Declared `[ordered]`, the first is 12.

3 An ADT type uses a niche-based encoding whenever a zero-cost representation is derivable. The compiler performs *aggressive multi-slot niche optimization* (D-Niche): it explores the full sentinel space of the payload and packs as many empty variants as fit, without a tag. Otherwise the sum is a tagged union, a tag beside a union of the payloads, and the compiler emits `[W120]` (a warning, not an error) with the reason.

```
// Tagged union (no niche is derivable: two variants carry a payload):
typedef struct Shape Shape;
struct Shape { uint8_t tag; union { struct { float r; } Circle;
                                    struct { float w; float h; } Rect; } data; };
```

4 The tag is the variant's number, $0, 1, \ldots, n-1$ in declaration order, stored as a plain enumeration is: the smallest unsigned integer type that holds it, `uint8_t` for up to 256 variants, `uint16_t` for up to 65 536, `uint32_t` otherwise. So a sum of narrow payloads stays narrow: `{ A { x u8 }, B { y u8 } }` is 2 bytes, aligned to 1. The width is a rule of the declaration, `[ordered]` or not. A sum that the signature of an `extern` function reaches (as for a struct, § 7.6) keeps an `int32_t` tag, the layout the foreign code was written against. A tag that is none of the variants is undefined (§ 18.5).

5 Each payload of a tagged union is stored as a struct is (§ 19.8): its fields by decreasing alignment, unless the sum is declared `[ordered]` or reachable from an `extern` signature, which keeps every payload in declaration order. An array field is stored inline, as a struct's is: a sum holds its payload by value, so constructing it copies the array, and the sum outlives the array it was built from. Construction, `case` and field access name each field, so the order is not observable but through `@sizeof`, and a cast that would view the bytes is refused (`[E012]`). A niche-packed sum has no payload struct (its one payload field is the value), so it is unaffected.

```
type Sh {
    Wide { a u8, b i64, c u8 }   // stored {b, a, c}: 16 bytes,
       not 24
    Flat
```

[2]Implementation-defined behavior: The exact size and alignment of each type is implementation-defined, following the C99 ABI of the target platform

```
}
assert @sizeof(Sh) == 24          // 32 in declaration order
```

Niche encoding (zero-cost). When the payload of one variant contains a bit pattern that cannot occur in valid values—for instance, `0x0` for a pointer field, or a value outside a VRA-proven range for an integer field—that pattern is reserved to encode an alternative variant. No explicit tag is emitted.

```
// Niche encoding for Option(*File):  size = sizeof(*File)
typedef File *Option_Ptr_File;
//  Some(p)   -> non-null pointer p
//  None      -> 0x0
```

The sentinel pool size depends on the target. For pointer payloads, the pool is the zero-page-unmappable region of the target's address space, stepped by the pointer alignment (e.g. 16384 slots for x86_64-linux with alignment 8). For integer payloads, the pool is the type's full range minus any VRA-narrowed sub-range: `type Pct = i32 >= 0 and <= 100` as a payload yields a pool of $\approx 4.3 \times 10^9$ slots (every integer outside $[0, 100]$). For `bool`, the pool is the 254 byte patterns outside $\{0, 1\}$. For a plain enum that declares its width, the pool is the values above its last variant: with `type K u8 { A, B, C }`, an `Opt { Some { k K }, None }` is one byte, `None` the value 3, and `K | none` likewise. Without a declared width the enum is the smallest integer that holds its variants, so its spare values would move with every variant added, and it is not a niche source. Nor is one that an `extern` function's signature reaches (as a parameter, a result, or through a pointer, a struct or a function pointer): C may hand it any value of its width, and a 3 would read as `None`.

```
type K u8 {
    A
    B
    C
}
type Opt {
    Some { k K }
    None
}
assert @sizeof(Opt) == 1          // no tag: None is K's spare
    value 3
```

Niche allocation cascades through nested enum payloads. If `Option(*T)` reserves the value `0` for `None`, a surrounding `Result(Option(*T), Err4)` sees the remaining slots $(8, 16, 24, \ldots)$ and packs its own empty variants into them. Layout of the entire result still occupies one pointer-sized word.

More than one payload. A sum with more than one payload-carrying variant is a tagged union whatever its payloads: `Result(*T, FileErr)` carries a `uint8_t` tag beside a union of the pointer and the `FileErr`. So does a payload of more than one field. A sum whose only variant carries one scalar field is that field, with no tag, since there is nothing to tell apart.

Outside a `T | markers` union, the compiler never refuses a program to satisfy a layout: a sum that gets a tag draws **[W120]** with the reason, and the programmer decides whether to accept the tag. A union whose markers carry no payload is the exception: its zero cost is required, and a union that would need a tag is *ill-formed* (**[E064]**).

The CLI flag `-target=<triple>` selects the target's sentinel pool size (host auto-detection is the default). Supported triples include `x86_64-linux-gnu`, `aarch64-linux-gnu`, `x86_64-windows-msvc`, and `cortex-m4-bare` (the last has zero pool: no MMU means address 0 may be a valid vector table entry).

19.10 No garbage collector

1 Lain has no garbage collector. Consequently:

- execution is never interrupted for memory management;
- deallocation occurs at statically deterministic program points;
- no reference counting or heap tracing is performed at runtime.

2 This makes Lain suitable for real-time, embedded, and safety-critical applications where garbage-collection pauses are unacceptable.

19.11 Memory safety in the safe fragment

1 In the *safe fragment* (code outside `unsafe` blocks), the following memory safety properties hold by construction:

Property	Enforcement mechanism
No use-after-free	`[E001]` (use-after-move)
No double-free	`[E002]` (double-consume)
No leak of linear values	`[E003]` (unconsumed at scope exit)
No dangling references	`[E010]` (ref to automatic-duration object)
No out-of-bounds access	`[E085]` (VRA proof failure)
No null-pointer deref	No raw pointers in safe fragment
No data races	No concurrency in current version

Constraint

Inside `unsafe` blocks, pointer operations may violate the above properties if the programmer's safety assertion is incorrect (§18.5).

19.12 Compiler-internal arena (informative)

Note

The reference implementation uses a bump-pointer arena internally for all AST nodes, symbol table entries, and diagnostic objects. This arena is an implementation detail; it is never exposed to user code. User programs may implement arena allocators of their own through C interop.

Chapter 20

Standard library

20.1 Overview

1 The Lain standard library is a collection of modules under the `std` namespace (§ 16.6). All standard library modules are written in Lain, using C interop (§ 17) for system-level operations. They are subject to the same ownership, borrow-checking, and purity rules as user code.

2 This clause is normative for the modules listed in Table 20.1. All other modules described in informative documentation are not part of this specification.

Table 20.1: Standard library module index

Module path	Description	Status
`std.c`	C standard library bindings	Normative
`std.io`	Console I/O	Normative
`std.fs`	File system operations	Normative
`std.math`	Pure mathematical functions	Normative

> **Constraint**
>
> A conforming implementation shall provide all modules listed in Table 20.1 at the specified module paths.

20.2 `std.c` — C standard library bindings

20.2.1 Purpose

1 `std.c` provides low-level bindings to selected C standard library functions. It is the foundation on which all other standard library modules that require system calls are built.

20.2.2 C headers

2 `std.c` includes no C header. Each binding is an `extern` declaration, and the translation output declares it with the prototype that declaration states (§ 17.2).

20.2.3 Opaque types

3 `std.c` declares the following opaque C type:

```
extern type FILE
```

`FILE` is an opaque type representing a C standard I/O stream. It may only be used through pointers and passed to the `extern` functions declared below.

> **Constraint**
>
> `FILE` shall not be used as a value type. A declaration such as `var f FILE` is *ill-formed* (§17.8).

20.2.4 Extern declarations

4 The following C functions are declared in `std.c`:

Declaration	Ownership notes
`extern func libc_printf(fmt *u8, ...)int effects io`	Shared `fmt`; variadic
`extern func libc_puts(s *u8)int effects io`	Shared `s`
`extern func fopen(filename *u8, mode *u8)mov *FILE effects io, alloc`	Returns owned handle
`extern func fclose(stream mov *FILE)int effects io`	Consumes handle
`extern func fputs(s *u8, stream *FILE)int effects io`	Shared args
`extern func fgets(s *var u8, n int, stream *FILE)*var u8 effects io`	Mutable buffer

> **Note**
>
> `libc_printf` and `libc_puts` are renamed to avoid conflicts with user-defined functions named `printf` and `puts`.

20.3 `std.io` — Console I/O

20.3.1 Dependencies

1 `std.io` imports `std.c`.

20.3.2 `print`

---

print *(module `std.io`)*

Synopsis: `func print(s u8[:0]) effects io`
Effect: Writes the bytes of `s` to standard output. No trailing newline is appended, and `s` is never read as a format string.
Parameter `s`: A string, `u8[:0]` (a string literal has this type).

20.3.3 `println`

---

println *(module `std.io`)*

Synopsis: `func println(s u8[:0]) effects io`
Effect: Writes the bytes of `s` to standard output, followed by a newline character (`0x0A`).
Parameter `s`: A string, `u8[:0]` (a string literal has this type).

20.4 std.fs — File system

20.4.1 Dependencies

std.fs imports std.c.

20.4.2 Type: File

File is a linear struct that wraps a C FILE* handle:

```
import std.c.{FILE}

type File {
    mov handle *FILE
}
```

Because handle is annotated mov, File is a linear type. It must be consumed exactly once, typically by calling close_file.

20.4.3 open_file

---

open_file *(module std.fs)*

Synopsis: `func open_file(path u8[:0], mode u8[:0]) mov File effects io, raises, alloc`
Effect: Opens the file at path with the access mode mode (C fopen semantics). Returns an owned File value. The caller is responsible for closing the file by consuming the File.
Parameter path: Shared borrow of a null-terminated path string.
Parameter mode: Shared borrow of a mode string ("r", "w", "a", etc.).

> **Note**
> If fopen fails, open_file panics: hence raises in its row. A File therefore never holds a null handle, and no later write_file or close_file can reach one.

20.4.4 close_file

---

close_file *(module std.fs)*

Synopsis: `func close_file(mov {handle} File) effects io`
Effect: Closes the file by calling C fclose on the extracted handle pointer. Consumes the File value.
Parameter: The File value is consumed by the call; it shall not be used after this point.

20.4.5 write_file

---

write_file *(module std.fs)*

Synopsis: `func write_file(f File, s u8[:0]) effects io`

Effect: Writes the byte sequence `s` to the file `f` using C `fputs`.
Parameter `f`: Shared borrow of the file handle.
Parameter `s`: Shared borrow of the string to write.

Example

```
import std.fs.{open_file, close_file, write_file}

func main() int effects io, raises, alloc {
    var f = open_file("log.txt", "w")
    defer close_file(mov f)
    write_file(f, "Hello, world!\n")
    return 0
}
```

20.5 `std.math` — Mathematical functions

20.5.1 Overview

All functions in `std.math` are declared `func` (pure and terminating). They may be called from any context, including other `func` bodies. `std.math` has no dependencies.

20.5.2 `min`

---

min *(module `std.math`)*

Synopsis: `func min(a i32, b i32) i32`
Returns: The smaller of `a` and `b`. If `a == b`, either may be returned (they are equal).

20.5.3 `max`

---

max *(module `std.math`)*

Synopsis: `func max(a i32, b i32) i32`
Returns: The larger of `a` and `b`.

20.5.4 `abs`

---

abs *(module `std.math`)*

Synopsis: `func abs(x i32) i32 >= 0`
Returns: The absolute value of `x`, i.e., `x` if `x >= 0`, `-x` otherwise.

> **Note**
> $|-2^{31}|$ is not representable as an `i32`, so `abs` negates with the saturating `-|`: `abs` of the smallest `i32` is the largest, and the return refinement `>= 0` holds for every input.

20.5.5 `clamp`

---

clamp *(module `std.math`)*

Synopsis: `func clamp(x i32, lo i32, hi i32) i32`
Returns: `x` clamped to the inclusive range `[lo, hi]`, i.e., `max(lo, min(x, hi))`.

> **Constraint**
> Behaviour is **undefined behavior**[a] if `lo > hi`.
>
> [a] Undefined behavior: undefined

20.6 Generic types (informative)

1 This section describes generic type constructors that are part of the standard library but are not normative in the current version of this specification.

20.6.1 `Option(T)`

2 `Option(T)` is a type constructor for optional values:

```
type Option(T type) {           // std/option.ln
    Some { val T }
    None
}
```

3 Usage:

```
import std.c.{libc_printf}
import std.option.{Option}

type OptionInt = Option(int)

func main() int effects io {
    var x = OptionInt.Some(42)
    case x {
        Some(v): libc_printf("value: %d\n", v)
        None:    libc_printf("no value\n")
    }
    return 0
}
```

20.6.2 `Result(T, E)`

4 `Result(T, E)` is a type constructor for fallible computations:

```
type Result(T type, E type) {  // std/result.ln
    Ok  { val T }
    Err { err E }
}
```

Usage:

```
import std.result.{Result}

type Digit = Result(i32, int)

func digit(c u8) Digit {
    if c >= 48 and c <= 57 { return Digit.Ok((c - 48) as i32) }
    return Digit.Err(1)
}
```

`Result`'s fields are not linear, so a linear value (a `File`) cannot be its payload: `Result(File, int)` is refused (**[E083]**). A fallible open declares its own sum type whose payload field is `mov`.

Appendix A

Grammar summary

1 This annex collects all grammar productions defined in this specification. It is normative. Productions are listed in the order they are introduced in the body of the specification. Each section heading identifies the originating clause.

2 The grammar uses an extended BNF notation. `::=` introduces a production. Quoted strings are terminals. Unquoted names are non-terminals. `{ X }` denotes zero or more repetitions. `[ X ]` denotes an optional item. `A | B` denotes alternatives.

§5 Lexical grammar

§5.3 Tokens

```
token ::=
    keyword
  | identifier
  | integer-literal
  | float-literal
  | char-literal
  | string-literal
  | bool-literal
  | operator
  | punctuator

end ::= newline | ";"          % ends a statement or a declaration (chapter 5)

comment ::=
    "//" { any character but a line feed }
  | "/*" { any character | comment } "*/"          % block comments nest
```

§5.5 Keywords

```
keyword ::=
    "and"       | "as"       | "assert"   | "assume"   | "break"
  | "c_include" | "case"     | "comptime" | "continue" | "decreasing"
  | "defer"     | "effects"  | "else"     | "extern"   | "false"
  | "for"       | "func"     | "if"       | "import"   | "in"
  | "mov"       | "nil"      | "or"       | "proc"     | "return"
  | "true"      | "try"      | "type"     | "unsafe"   | "use"
  | "var"       | "while"

% c_include, nil and proc are reserved and appear in no production: each was removed,
% and stays a keyword so that using it is diagnosed with the reason.
```

§5.6 Identifiers

```
identifier           ::= identifier-start { identifier-continue }    % not a keyword
identifier-start     ::= letter | "_"
identifier-continue  ::= letter | decimal-digit | "_"
letter               ::= "A" .. "Z" | "a" .. "z"
qualified-identifier ::= identifier { "." identifier }
```

§5.7 Literals

```
integer-literal  ::=
    decimal-literal
  | hex-literal
  | binary-literal
  | octal-literal

decimal-literal  ::= nonzero-digit { decimal-digit | "_" }
                   | "0"                          % no leading zero: 010 is ill-formed
hex-literal      ::= ( "0x" | "0X" ) hex-digit { hex-digit | "_" }
binary-literal   ::= ( "0b" | "0B" ) binary-digit { binary-digit | "_" }
octal-literal    ::= ( "0o" | "0O" ) octal-digit { octal-digit | "_" }

decimal-digit    ::= "0" .. "9"
nonzero-digit    ::= "1" .. "9"
hex-digit        ::= decimal-digit | "a" .. "f" | "A" .. "F"
binary-digit     ::= "0" | "1"
octal-digit      ::= "0" .. "7"

float-literal    ::= digit-sequence "." digit-sequence [ float-exponent ]
                   | digit-sequence float-exponent
digit-sequence   ::= decimal-digit { decimal-digit | "_" }
float-exponent   ::= ( "e" | "E" ) [ "+" | "-" ] digit-sequence

char-literal     ::= "'" ( char-element | escape-sequence ) "'"
char-element     ::= any character but "'", "\" or a line feed
string-literal   ::= '"' { string-element | escape-sequence } '"'
string-element   ::= any character but '"' or "\"
escape-sequence  ::= "\" ( "n" | "t" | "r" | "0" | "\" | "'" | '"' )
                   | "\" "x" hex-digit hex-digit

bool-literal     ::= "true" | "false"

literal          ::= integer-literal | float-literal | char-literal
                   | string-literal | bool-literal
```

§5.8 Operators and punctuators

```
operator         ::=
    "+"   | "-"   | "*"   | "/"   | "%"
  | "+%"  | "-%"  | "*%"  | "/%"                    % wrapping
  | "+|"  | "-|"  | "*|"  | "/|"                   % saturating
  | "+?"  | "-?"  | "*?"                           % checked
  | "<<"  | "<<%" | ">>"
  | "&"   | "|"   | "^"   | "~"   | "!"
  | "=="  | "!="  | "<"   | ">"   | "<="  | ">="
  | "="   | "+="  | "-="  | "*="  | "/="  | "%="
  | "&="  | "|="  | "^="  | "<<=" | ">>="
  | "."   | ".."  | "..="
  | "and" | "or"  | "in"  | "as"

punctuator       ::=
```

```
    "("   | ")"   | "{"   | "}"   | "["   | "]"
  | ","   | ":"   | ";"   | "@"   | "?"   | "..."

% && and || are not operators; each is diagnosed with its spelling, and / or.
```

§6 Program structure grammar

```
program               ::= { top-level-declaration | end }

top-level-declaration ::=
    import-declaration
  | { attribute } type-declaration
  | { attribute } function-declaration
  | { attribute } extern-declaration
  | { attribute } constant-declaration
  | module-assertion

import-declaration    ::= "import" qualified-identifier
                          [ "." "{" identifier { "," identifier } "}" ]
                          [ "as" identifier ]
constant-declaration  ::= identifier [ type ] "=" expression     % compile-time constant
module-assertion      ::= "assert" expression             % checked at compile time

attribute             ::= "[" attribute-name [ "(" [ argument-list ] ")" ] "]" { end }
attribute-name        ::= "private" | "packed" | "ordered"
                        | "cold" | "hot" | "allocator" | "noreturn"
                          % identifiers; any other name is E103 (‘fast_math‘
                          % included: floating point is never contracted)
```

§7 Type grammar

```
type              ::= core-type { "|" marker }       % a union: T | none, T | A | B
marker            ::= identifier
                      [ "(" marker-field { ( "," | newline ) marker-field } ")" ]
marker-field      ::= identifier type

core-type         ::=
    named-type { array-suffix }
  | "Vec" "(" integer-literal "," core-type ")"     % a SIMD vector of N lanes
  | "*" [ "var" ] core-type                         % raw pointer; *var T is writable
  | "*" "func" "(" [ core-type { "," core-type } ] ")" [ core-type ]
    [ effects-clause ]                               % function pointer
  | "mov" core-type                                 % owned
  | "var" core-type                                 % mutable borrow
  | "type"                                          % the type of a type parameter

% The pointee of * is not an array or slice type (E100): an array or a slice is
% already passed by reference.

named-type        ::= qualified-identifier
                      [ "(" type-argument { "," type-argument } ")" ]
type-argument     ::= core-type | integer-literal

% The primitive types are identifiers (chapter 7): int, i1 to i64, u1 to u64, isize,
% usize, float, f32, f64, bool, and void. A vector type may also be written as one
% identifier, element and lane count joined by x: u8x16 is Vec(16, u8).

array-suffix      ::=
```

```
    "[" integer-literal "]"                          % fixed-length array
  | "[" "]"                                          % slice
  | "[" [ refinement-op ] expression "]"             % slice, its length constrained
  | "[" ":" "0" "]"                                 % a slice followed by a 0; any
                                                     % other sentinel is E100

refinement       ::= range-refinement [ "and" comparison-list ]
                   | comparison-list
range-refinement ::= "in" bound-term ( ".." | "..=" ) bound-term
comparison-list  ::= refinement-op bound { "and" refinement-op bound }
refinement-op    ::= "<" | "<=" | ">" | ">=" | "==" | "!="
bound            ::= bound-term { ( "+" | "-" ) bound-term }
bound-term       ::= [ "-" ] integer-literal | identifier [ "." identifier ]

% Where each form is allowed (chapter 7, constrained types): a parameter takes all of
% them, a name being an integer module constant or another parameter. A struct field's
% bound is one bound-term, a name being an integer module constant or another field; a
% variant field's bound is a number. A return type's comparison-list has no negative
% number and no range, a name being an integer module constant or a parameter. An
% alias's bound is a number or an integer module constant.
```

§8 Expression grammar

```
expression            ::= or-expression { "else" [ "return" ] or-expression }
                          % the error handler; it binds most loosely of all

or-expression         ::= and-expression { "or" and-expression }
and-expression        ::= equality-expression { "and" equality-expression }
equality-expression   ::= relational-expression
                          { ( "==" | "!=" ) relational-expression }
relational-expression ::= bitor-expression
                          { ( "<" | ">" | "<=" | ">=" ) bitor-expression
                          | "in" membership-set }
membership-set        ::= shift-expression ( ".." | "..=" ) shift-expression
                        | "[" expression { "," expression } "]"
bitor-expression      ::= bitxor-expression { "|" bitxor-expression }
bitxor-expression     ::= bitand-expression { "^" bitand-expression }
bitand-expression     ::= shift-expression { "&" shift-expression }
shift-expression      ::= additive-expression
                          { ( "<<" | "<<%" | ">>" ) additive-expression }
additive-expression   ::= multiplicative-expression
                          { additive-operator multiplicative-expression }
additive-operator     ::= "+" | "-" | "+%" | "-%" | "+|" | "-|" | "+?" | "-?"
multiplicative-expression ::= cast-expression
                          { multiplicative-operator cast-expression }
multiplicative-operator   ::= "*" | "/" | "%" | "*%" | "/%" | "*|" | "/|" | "*?"
cast-expression       ::= unary-expression { "as" [ "?" | "%" | "|" ] core-type }
unary-expression      ::= unary-operator unary-expression
                        | postfix-expression
unary-operator        ::= "-" | "!" | "~" | "&" | "*" | "mov" | "var" | "try"

postfix-expression    ::= ( identifier | "(" expression ")" ) { selector }
                        | primary-expression
selector              ::= "." identifier
                        | "(" [ argument-list ] ")"
                        | "[" index "]"
index                 ::= expression
                        | expression ".." [ expression ]
```

```
                        | ".." expression
argument-list         ::= expression { "," expression }

% A call, the construction of a struct (Point(1, 2)), of a variant (Shape.Circle(3))
% or of a marker with fields (Bad(3)), and an explicit generic argument
% (max(i32, a, b)) are one form: a name followed by "(" argument-list ")".

primary-expression    ::=
    literal
  | array-literal
  | array-comprehension
  | match-expression
  | builtin-expression

array-literal         ::= "[" [ expression { "," expression } [ "," ] ] "]"
array-comprehension   ::= "[" expression "for" identifier "in"
                          expression ( ".." | "..=" ) expression "]"

builtin-expression    ::=
    "@" ( "os" | "arch" )
  | "@" ( "likely" | "unlikely" ) "(" expression ")"
  | "@" ( "ctz" | "clz" | "popcount" | "movemask" ) "(" expression ")"
  | "@" ( "sizeof" | "alignof" ) "(" type ")"
  | "@" "splat" "(" type "," expression ")"
  | "@" "load" "(" type "," expression "," expression ")"
  | "@" "store" "(" expression "," expression "," expression ")"
  | "@" "shuffle" "(" expression "," expression ")"
  | "@" "assume_aligned" "(" expression "," integer-literal ")"

match-expression      ::= "case" [ "&" ] expression "{" { expression-arm } "}"
expression-arm        ::= arm-head expression [ "," ]
arm-head              ::= ( pattern-list | "else" ) ":"
pattern-list          ::= pattern { "," pattern }
pattern               ::=
    pattern-value [ ( ".." | "..=" ) pattern-value ]   % a value, or a range of them
  | variant-name "(" [ identifier { "," identifier } ] ")"
                                                       % a variant, binding its fields
pattern-value         ::= [ "-" ] integer-literal | char-literal
                        | string-literal | bool-literal
                        | variant-name                 % a unit variant, or a constant
variant-name          ::= qualified-identifier
                        | identifier "(" type-argument { "," type-argument } ")"
                          "." identifier

% Arms are separated by newlines, or by ",". The body of an expression-arm is ONE
% expression; a "{" after the ":" is not a block. The statement form, whose arms hold
% statements, is match-statement (chapter 9).
```

§9 Statement grammar

```
statement             ::=
    binding-declaration
  | variable-declaration
  | assignment-statement
  | expression-statement
  | if-statement
  | for-statement
  | while-statement
```

```
  | match-statement
  | "break"
  | "continue"
  | return-statement
  | defer-statement
  | unsafe-statement
  | assertion-statement
  | comptime-statement
  | use-statement

block                ::= "{" { end } [ statement { end { end } statement } ] { end } "}"

% There is no block STATEMENT: "{" never begins a statement. A block is the body of
% a function, an if or else, a loop, unsafe, or comptime if.

binding-declaration  ::= identifier [ type ] "=" expression             % immutable
variable-declaration ::= ( "var" | "mov" ) identifier [ type ]
                         [ "in" expression ] [ "=" expression ]
                         % var: mutable; mov: owned. Without "=", the type is required.

assignment-statement ::= assignable assignment-operator expression
assignable           ::= postfix-expression | "*" unary-expression
assignment-operator  ::= "=" | "+=" | "-=" | "*=" | "/=" | "%="
                       | "&=" | "|=" | "^=" | "<<=" | ">>="

expression-statement ::= expression

if-statement         ::= "if" expression block [ "else" ( if-statement | block ) ]
                         % else may begin the line after the "}"
for-statement        ::= "for" identifier [ "," identifier ] "in"
                         expression [ ( ".." | "..=" ) expression ] block
                         % for x in a, for i, x in a (index first), for i in lo..hi
while-statement      ::= "while" expression [ "decreasing" expression ] block

match-statement      ::= "case" [ "&" ] expression "{" { arm-head arm-body } "}"
arm-body             ::= { end } statement end { { end } statement end }
                         % statements, each ending at a newline, up to the next
                         % arm-head or the "}"

return-statement     ::= "return" [ expression ]
defer-statement      ::= "defer" statement               % one statement, not a block
unsafe-statement     ::= "unsafe" block
assertion-statement  ::= ( "assert" | "assume" ) expression
comptime-statement   ::= "comptime" "if" expression block [ "else" block ]
                       | "comptime" identifier [ type ] "=" expression
use-statement        ::= "use" qualified-identifier "as" identifier
                         % semantics implementation-defined (chapter 16); refused, E100
```

§10 Declaration grammar

```
function-declaration ::= "func" identifier "(" [ parameter { "," parameter } ] ")"
                         [ return-type ] [ effects-clause ]
                         [ "decreasing" expression ] block

parameter            ::= [ "var" | "mov" ] identifier type [ refinement ]
                       | [ "var" | "mov" ] "{" identifier { "," identifier } "}" type
                         % the second form destructures the argument into its fields
```

```
return-type          ::= type [ "in" identifier ] [ comparison-list ]
                         % in p: the returned reference borrows parameter p

effects-clause       ::= "effects" [ effect-name { "," effect-name } ]
effect-name          ::= "io" | "diverge" | "raises" | "alloc"          % identifiers

extern-declaration   ::= "extern" "func" identifier "(" [ extern-parameters ] ")"
                         [ type [ "in" identifier ] ] [ effects-clause ]
                       | "extern" "type" identifier              % an opaque C type
extern-parameters    ::= identifier type { "," identifier type } [ "," "..." ]
                       | "..."

type-declaration     ::= alias-declaration | struct-declaration | enum-declaration

alias-declaration    ::= "type" identifier "=" alias-target
alias-target         ::=
    identifier comparison-list          % a refined alias: type Small = u8 < 200
  | named-type { "[" expression "]" }    % a constant length: type Grid = u8[3][3]
  | "type" "{" ( field-list | variant-list ) "}"   % an anonymous struct or enum

struct-declaration   ::= "type" identifier [ type-parameters ] "{" [ field-list ] "}"
field-list           ::= { end } field { ( "," | end ) { end } field }
                         [ "," ] { end }
field                ::= [ "mov" ] identifier type [ refinement ]

enum-declaration     ::= "type" identifier [ type-parameters ] [ backing-width ]
                         "{" [ variant-list ] "}"
variant-list         ::= { end } variant { ( "," | end ) { end } variant }
                         [ "," ] { end }
variant              ::= identifier
                         [ "{" variant-field { ( "," | end ) variant-field } "}" ]
variant-field        ::= identifier type [ comparison-list ]
                         % a linear field is h mov H: here mov belongs to the type
backing-width        ::= "u8" | "u16" | "u32" | "u64"          % a plain enum only

type-parameters      ::= "(" identifier type { "," identifier type } ")"
                         % T type is a type parameter; N usize a compile-time value
```

Appendix B

Diagnostic codes

1 This annex is normative. It lists every diagnostic code defined by this specification, the clause that requires the diagnostic, the category (constraint violation, warning, or informational), and a representative message text. A conforming implementation shall issue a diagnostic for every constraint violation in the table below.

2 Diagnostic codes are divided into the following ranges:

Range	Domain
`[E001]`–`[E010]`, `[E016]`–`[E021]`	Ownership and lifetime
`[E011]`–`[E015]`, `[E060]`	Purity, safety, and verification
`[E082]`	Bounded while verification
`[E083]`–`[E087]`	Array/slice safety and integer boundaries
`[E084]`, `[E101]`–`[E138]`	Modules, attributes, generics, packing, ADT access, effects, verification
`[E100]`	Miscellaneous
`[VRA]`	Value range analysis

Ownership and lifetime diagnostics (`[E001]`–`[E010]`, `[E016]`, `[E020]`–`[E021]`)

Code	Clause	Message and trigger
`[E001]`	§11.3	*use of moved value 'x'* — Issued when an expression reads or passes a variable whose linear state is CONSUMED.
`[E002]`	§11.3	*value 'x' consumed twice* — Issued when a second `mov` expression is applied to a variable already in state CONSUMED. This subsumes the retired `E006` (*consumed inside a loop*): consuming a value declared outside a loop from inside its body *is* consuming it twice, and `[E002]` names the violation where `E006` named only the syntactic setting it occurred in.
`[E003]`	§11.3	*linear value 'x' not consumed before end of scope* — Issued when a `mov`-mode variable exits scope in state UNCONSUMED.

Code	Clause	Message and trigger
`[E004]`	§11.3	*cannot borrow 'x' as mutable; it is already borrowed* — Issued when a mutable borrow conflicts with an existing borrow (reads or write).
`[E005]`	§11.3	*use of uninitialized variable 'x'* — Issued when a variable, struct field, or array element is read before it has been initialized (definite-assignment analysis).
`[E007]`	§11.3	*moving linear value 'x' requires an explicit* `mov` — Issued when a linear value is passed to an owned (`mov`) parameter (or otherwise transferred) without the explicit `mov` keyword: an implicit move.
`[E010]`	§19.3	*this reference would outlive the value it borrows* — Issued when a reference whose referent has automatic storage duration would outlive it: returned (directly, or held by a struct or a sum, through any binding, assignment or call), stored into a place the caller owns, or passed to a function that keeps it there.
`[E016]`	§11.3	*linear variable 'x' is used inconsistently in the branches of* stmt — Issued when a linear value is consumed in some control-flow paths of an `if`/`case` but not in others, leaving an ambiguous state at the join point. Field-level inconsistency on linear struct fields is reported with the same code.
`[E020]`	§11.7	*cannot move out of a borrow* — Issued when a `mov`, or a whole-value copy of a linear value, takes its operand from a place the function does not own: a `var` or shared parameter, a reference binding, a returned borrow, or a field or element of one. The owner still holds the resource; moving it out would leave two owners.
`[E021]`	§11.3	*assignment overwrites a linear value that still holds a resource* — Issued when a linear place is assigned while it may still hold a resource, which would be lost. Consume it first. Through a borrow the old value can never have been consumed (`[E020]`), so any such assignment is refused.

Purity, safety, and verification (`[E011]`–`[E015]`)

Code	Clause	Message and trigger
`[E011]`	§12.2	*unacknowledged effect in 'f'* — Issued when a function's body has an effect its row does not name, while its bound is ∅ — that is, while no `effects` clause was written, or an empty one was. A bound of ∅ is a function's default *guarantee*, so violating it is this code; understating a *non-empty* row is `[E130]`. The effects are `io`, `diverge`, `raises` and `alloc`. Also issued for a `while` loop whose measure is neither given nor inferable, and for a direct self-recursion for which no measure can be inferred — both sources of `diverge`. (A direct recursion *with* a well-founded measure is permitted; a present-but-failing measure is `[E082]`/`[E091]`.)
`[E012]`	§17.6	*type or constraint violation* — Issued when a value does not have the type its context requires (an argument, a constructor field, a return, an operand, a `case` scrutinee or pattern, §15.3), when a value would have an opaque type or `void`, when a refinement at a call site cannot be proven, when an array, slice or pointer would convert to one whose element type is refined differently (§7.2.3), and when a pointer-to-pointer or integer-to-pointer cast occurs outside an `unsafe` block, and when a cast, inside `unsafe` too, would view the bytes of a struct, or of a sum's payload, whose storage order the compiler chooses (§7.6; mark it `[ordered]`). The message names which; a test that expects `[E012]` should say which it means, since the code alone does not.
`[E013]`	§6.2	*redeclaration or shadowing of 'x' is forbidden* — Issued for a duplicate parameter name, a local that repeats or shadows an outer binding, a module-scope name defined twice (two `extern` declarations of one function must agree), or a binding (variable, parameter, or loop variable) whose name is a builtin type (`i32`, `u8`, `bool`, ...): a type name denotes a first-class type value and cannot also name a binding.
`[E014]`	§15.4	*non-exhaustive `case`* — Issued when the arms of a `case` do not cover all values of the scrutinee type. (An out-of-bounds *index* is a different code, `[E085]`.)

Code	Clause	Message and trigger
[E015]	§8.3	*a value that must be non-zero is not provably so* — The divisor of a division or remainder, or the argument of `@ctz` or `@clz` (§7.5), when neither its type, its range nor a guard shows it is not zero.

Bounded while verification ([E082])

Code	Clause	Message and trigger
[E082]	§9.7.1, §12.3	*termination measure not proven well-founded* — Issued when the implementation cannot verify that every looping-back path through the loop body strictly decreases the measure by at least 1 (paths that `break`/`return` out are exempt), or that every self-call of a directly-recursive `func` makes its measure strictly smaller, or that the measure is bounded below. A measure the implementation cannot analyse at all is this code as well. **Note** E080 (*measure not proven non-negative*) and E081 (*cannot extract variables from measure*) were separate codes for two subcases of this one, issued by a second termination checker in the front end. That checker is removed and the obligation has a single source, so the two codes are withdrawn: an absent or uninferable measure is [E011], a present one that cannot be shown well-founded is [E082], and nothing else is reported. A conforming implementation shall not issue E080 or E081.
[E091]	§12.3	*malformed recursion measure* — Issued when a directly-recursive `func`'s `decreasing` measure is not a parameter or a difference of two parameters.

Miscellaneous ([E100])

Code	Clause	Message and trigger
`[E100]`	§5, §7	*ill-formed token / semantic constraint violation* — A catch-all code used for lexical and syntactic errors without a dedicated code, including: an unexpected token, using `==` on enum types (§7.7), an empty or multi-character character literal, a string literal, character literal or block comment that the source file ends inside, a keyword used as an identifier, and a zero-field array size. Also a construct this implementation does not model, refused where it appears rather than translated without it, and a function beyond the capacity of one of its analyses (an implementation limit), refused rather than judged on a truncated view.

Array/slice safety and integer boundaries (`[E083]`–`[E087]`)

Code	Clause	Message and trigger
`[E083]`	§7.6	*linear field 'f' missing `mov` annotation* — Issued when a struct or enum variant field has a linear type but lacks the `mov` keyword. (Q-008 transitivity rule.)
`[E085]`	§8.13, §8.14	*index or slice endpoint not proven in bounds* — Issued when VRA cannot prove a non-negativity or upper-bound requirement for an array/slice index or sub-slice endpoint.
`[E086]`	§13.10	*integer overflow at boundary: range exceeds target type* — Issued when the VRA range of a source value provably exceeds the natural range of the target type at a type boundary (declaration, assignment, call argument, return, struct constructor), when a left shift may lose bits (§8.6; `<<%` discards them on purpose), or when a module-scope constant's value, or an element of one, is not representable in its declared type (§10.7).
`[E087]`	§8.14, §13.12	*sized slice constraint violated* — Issued (1) when both endpoints of a sub-slice `arr[a..b]` are integer literals and $a > b$; (2) at a call site where VRA can prove that the argument does not satisfy the formal parameter's sized slice constraint (`T[k]`, `T[>= k]`, `T[> k]`, or `T[ref.len]`).

Value range analysis (`[VRA]`)

Code	Clause	Message and trigger
`[VRA]`	§13	*value range proof required* — Issued when VRA cannot prove that an operation satisfies its required constraint and more specific code is not available. Appears as a diagnostic prefix in messages such as: "`[VRA] cannot prove index in bounds`" and "`[VRA] return value may not satisfy constraint`".

Additional diagnostics (`[E008]`, `[E009]`, `[E017]`–`[E125]`)

The following diagnostics are emitted by the reference implementation and are normative. Each was verified against the compiler's diagnostic strings during the §B reconciliation pass.

Code	Message and trigger
`[E008]`	cannot move a value that is currently borrowed
`[E009]`	cannot mutate a field through a raw pointer in safe code — use a `var` borrow; or a write through a r
`[E017]`	a mutable borrow is spelled `var`: passing to a `var` parameter requires it at the call site, and a functio
`[E018]`	a function can reach its end without returning a value
`[E019]`	use of a partially-initialized value
`[E060]`	pointer dereference outside an `unsafe` block, including a field access whose base is a raw pointer (`p.v`
`[E063]`	a possibly-marker union value used without testing it first
`[E064]`	a `|`-union cannot be niche-optimized (no spare bit pattern)
`[E084]`	access to a `[private]` declaration from another module
`[E101]`	a `comptime` context calling a function whose `effects` row is non-empty: compile-time evaluation has
`[E102]`	expected an attribute name after the opening bracket
`[E103]`	unknown attribute, or an attribute on a declaration it does not apply to (`[ordered]` off a struct or
`[E104]`	a type contains itself by value (infinite size)
`[E105]`	identifier is defined in another module — add an `import`
`[E106]`	use of an undeclared identifier, a `case` pattern that names no variant of the scrutinee's enum (§15.3)
`[E121]`	a struct field invariant — `pos usize in 0..text.len` (a valid index, always below the length; `in 0..`
`[E122]`	a function does not match the pointer type it converts to: arity, types, row or refinements (§12.7)
`[E123]`	wrong number of arguments in a call through a function pointer
`[E124]`	generic instantiation: non-generic type, missing or uninferable type argument, a value argument tha
`[E125]`	direct ADT field access outside `unsafe` — destructure with `case`
`[E065]`	a `try` may propagate a marker the enclosing return union does not list
`[E066]`	`try`/`else` requires a `T | markers` union value
`[E067]`	a checked operator (`+?`, `-?`, `*?`, `as?`) must be handled by `else`

Code	Message and trigger
[E088]	a function returns a fixed-size array by value — return a slice or write through a `var` parameter
[E089]	a non-terminated value used where a sentinel-terminated type is required
[E090]	assignment between fixed-length strings of different lengths
[E126]	a mutable borrow (`var`) stored in a struct field — a field would outlive any loan the borrow check
[E127]	call to an undeclared function
[E128]	the receiver has no such member, and no function of that name is in scope for UFCS
[E129]	`assume` outside an `unsafe` block — use `assert` to have the fact proven
[E130]	a declared, non-empty `effects` row does not cover what the body does (an empty or absent row i
[E131]	an `extern` declares a slice parameter — a slice has no C representation; pass a pointer and a leng
[E132]	a struct field's refinement is not one that is enforced — a relation other than `f CMP g` (g an integ
[E133]	a module-scope `assert` whose operand is not a `bool` constant expression (§10.8) — a run-time fa
[E134]	a module-scope assertion is false — reported by the implementation, or, for one that measures a
[E135]	a `[noreturn]` function can return: a `return`, or the end of its body, is reachable (§10.1)
[E136]	a module constant computed at compile time cannot be: its initialiser reaches a function whose e
[E137]	a compile-time constant is required and the expression is not one: a parameter refinement or size
[E138]	control would leave a deferred statement: a `return`, a `break` or `continue` aimed at a loop outside i

Appendix C

Language design rationale

1 This annex is informative. It explains the reasoning behind key design decisions in the Lain language. Understanding the rationale helps readers interpret edge cases in the normative text and guides implementors when the specification is ambiguous.

C.1 Declaring effects: the `effects` row

1 **Decision.** State what a function may do at the declaration level, in an `effects` row on the one introducer `func`, where silence means the empty row — pure and total.

2 **Rationale.** Every reason below was a reason to declare effects at all, and each still holds; only the spelling changed.

- *Termination guarantees*: a function whose row does not name `diverge` is guaranteed to terminate (no unbounded `while`, no unchecked recursion). This lets the implementation reason statically about completion, and lets such a body be used inside `comptime` expressions.
- *Effect propagation*: the constraints propagate — a function may not call one whose row names an effect its own row does not (`[E011]`) — so pure code stays pure by construction.
- *Optimisation*: the implementation may freely reorder, memoize or eliminate calls to a function with the empty row, because it has no observable side effects.
- *Readability*: the row tells the reader what a callable may do without inspecting its body, and because it is a complete upper bound rather than a hint, the *absence* of a row is itself a statement.

3 **The original spelling, and why it was replaced.** The decision was first realised as two introducers: `func` for the pure and total case and `proc` for anything else. That form was removed on 2026-09-25, in the effect-row work (plan Part 7B). Three reasons, each about what a keyword *cannot* say:

- The bound had only two points, `func` and `proc` — a single bit. Four effects therefore had to be spelled a second way regardless, and were: by keyword, by attribute (`@diverges`) and by clause, all at once. One concern with three spellings is what the language's own law of a single mechanism refuses.
- A row makes assignment between function-pointer types *row containment*, which subsumes the one relation the keywords could express (a `func` being usable where a `proc` was expected) and gives every other one for free.
- A row lets an arrow say *"may print, but terminates"*. A bit cannot: it must either grant divergence along with `io` or deny both.

Alternatives considered. A single keyword with a purity annotation (`func(pure)`) was rejected as less visible than a row that must name every effect. A Haskell-style IO monad was rejected as too complex for a systems language.

C.2 Explicit `mov` at call sites

Decision. Require explicit `mov` at call sites when transferring ownership.

Rationale.

- *Readability*: `close_file(mov f)` makes ownership transfer visible at the call site.
- *Intentionality*: accidental ownership transfer is a common source of bugs. Requiring `mov` forces the programmer to acknowledge the transfer.
- *Symmetry*: the annotation appears in both declarations (`mov x T`) and at call sites (`mov x`), creating a uniform language.

Alternatives considered. Rust-style implicit moves were rejected because they are invisible at call sites. A library function `std::move` (C++) was rejected because ownership transfer should be a language primitive, not a library function.

C.3 Implicit NLL lifetimes (no annotations)

Decision. Borrows have implicit lifetimes determined by non-lexical lifetime analysis. No explicit lifetime annotations are required.

Rationale.

- *Simplicity*: Rust's lifetime annotations are widely cited as its steepest learning curve. Lain targets programmers who want safety without that complexity.
- *Sufficient expressiveness*: NLL handles the vast majority of real-world borrow patterns.
- *Predictability*: borrows expire at last use — a single, intuitive rule.

Trade-offs. Some programs that Rust can verify with explicit lifetime annotations may be rejected by Lain's more conservative analysis. Higher-order borrowing patterns are constrained.

C.4 C99 translation output

Decision. Compile to C99 source code rather than LLVM IR, machine code, or bytecode.

Rationale.

- *Portability*: C99 compilers exist for every platform, from server-class x86 to microcontrollers. Lain inherits this portability for free.
- *Toolchain reuse*: users get GCC/Clang optimisation passes, debuggers, profilers, and sanitisers without custom tooling.
- *Interoperability*: C interop is zero-cost because the output *is* C.
- *Simplicity*: a C emitter is orders of magnitude simpler than an LLVM backend or a native code generator.
- *Auditability*: the generated C code is human-readable.

Trade-offs. Compilation is two-phase (Lain → C → binary). Some optimisations are impossible without direct IR control. The C type system constrains what Lain can express (e.g., no guaranteed tail calls).

C.5 No exception mechanism

Decision. Lain has no exception mechanism. All error handling uses return values.
Rationale.

- *Zero overhead*: no unwinding tables, catch tables, or RTTI.
- *Determinism*: error handling follows the same control flow as normal code; no invisible jumps.
- *Embedded compatibility*: stack unwinding is unavailable or unreliable on many embedded platforms.
- *Explicit error paths*: every error is visible in the function signature, as a marker of its union return type (`T | NotFound`, § 7.9).
- *Compatibility with* `defer`: the `defer` mechanism provides deterministic cleanup without exceptions.

C.6 Explicit `unsafe` blocks

Decision. Pointer dereference and direct ADT field access require an explicit `unsafe` block.
Rationale.

- *Auditability*: safety-critical codebases can grep for `unsafe {` to enumerate all potentially dangerous operations.
- *Minimal scope*: Lain's `unsafe` is narrow — it enables only pointer dereference, address-of, casts, direct ADT access, and bounds suppression. Unlike C (where everything is unsafe by default) or some languages (where `unsafe` enables a much larger capability set), Lain's `unsafe` is a surgical escape hatch.
- *Social contract*: the keyword communicates to code reviewers that extra scrutiny is warranted.

C.7 Value Range Analysis instead of SMT

Decision. Use a value range analysis — intervals, and the relations between pairs of variables an octagon keeps — for static verification instead of an SMT solver.
Rationale.

- *Decidability*: VRA always terminates in polynomial time. SMT solving is NP-hard in the general case and may time out.
- *Predictability*: programmers can reason about what the compiler will accept by thinking about integer ranges. SMT solver reasoning is opaque.
- *Self-containment*: the analysis is part of the compiler, with no external dependency, and its every verdict can be replayed from a certificate.

Trade-offs. VRA is incomplete — it may reject valid programs that an SMT solver could prove correct. Non-linear constraints produce conservatively wide ranges.

C.8 Type parameters instead of <T> syntax

Decision. A type parameter is an ordinary parameter whose type is `type`, `func max(T type, a T, b T)T`, and every instance is monomorphized (§ 14).

Rationale.

- *Unification*: types and compile-time values are passed the same way; `type Buf(N usize)` takes a length as `type Box(T type)` takes a type.
- *Simplicity*: no angle brackets, no trait bounds, no where clauses. The body is checked at each instance with the types it is given.
- *Zero cost*: an instance is an ordinary function or type, with no dictionary passing and no boxing.
- *Precedent*: inspired by Zig, whose types are compile-time values.

C.9 Universal Function Call Syntax (UFCS)

Decision. Support UFCS: `x.f(args)` desugars to `f(x, args)`.

Rationale.

- *No classes needed*: UFCS provides method-like syntax without class hierarchies, virtual dispatch, or inheritance.
- *Chainability*: enables fluent APIs.
- *Discoverability*: IDE auto-completion works naturally — typing a dot after a value shows available operations.
- *Simplicity*: syntactic sugar only; no new concepts, no method tables, no implicit `this` pointer.

C.10 Bounded `while` in `func`

Decision. Allow `while` loops in a function with the empty row when their termination can be shown: by a measure the compiler infers from the loop, or by one the programmer writes with `decreasing`.

Rationale. A *measure* — a non-negative integer expression that strictly decreases on every iteration — is a well-established technique from program verification (*ranking functions*). For the common loop, `while i < n` with `i` rising, the compiler finds it itself (§ 9.7.2); where it cannot, the programmer states one, and the compiler checks it rather than believing it. Neither needs an external solver.

This allows algorithms such as binary search and GCD computation to be expressed with the empty effect row, rather than having to admit an effect they do not have.

Appendix D

Comparison with related languages

1 This annex is informative. It places Lain in the context of other systems-programming languages to help readers understand its design choices. Comparisons are accurate as of the date of this specification; language features evolve and readers should verify current status.

D.1 Feature matrix

Feature	Lain	Rust	C	Zig	Go
Memory safety	Compile-time	Compile-time	Manual	Runtime + CT	GC
Null safety	`T | none`	Option type	None	Optional	Nil
Exceptions	None (unions)	None (Result)	errno	Error union	panic/recover
Generics	Type params	Trait-based	Macros	Comptime	Type params
Lifetime annots.	None (NLL)	Required	N/A	None	N/A
Purity tracking	`effects` row	None	None	None	None
Pattern matching	Exhaustive	Exhaustive	switch	switch	switch
UFCS	Yes	No	No	No	No
Backend	C99	LLVM	Native	LLVM	Go compiler
GC / runtime	None	Minimal	None	Minimal	GC

D.2 Lain versus Rust

Similarities

- Ownership-based memory safety with no GC.
- Move semantics with explicit transfer.
- No null pointers: an absent value is a union marker (`T | none`, where Rust has `Option`), tested before use.
- Pattern matching with exhaustiveness checking.

- No exceptions: an error is a union marker passed on with `try`, as Rust passes a `Result` with `?`.
- Zero-cost abstractions.

Differences

Aspect	Lain	Rust
Lifetime annotations	Implicit (NLL only)	Explicit (`'a`, `'static`)
Mutable borrow syntax	`var x` at call site	`&mut x`
Move syntax	Explicit `mov x`	Implicit (default)
Generics	Type parameters, monomorphized	Trait-based + monomorphization
Purity	`effects` row, enforced	Convention only
Async	Not supported	`async`/`await`
Traits / Interfaces	Not supported	Full trait system
Macros	Not supported	`macro_rules!` + proc macros
Backend	C99	LLVM
Learning curve	Lower	Higher

1 **Key insight.** Lain trades Rust's expressiveness (explicit lifetime annotations, trait system, async/await) for simplicity. The result is a language that is easier to learn but less expressive for complex borrowing patterns.

D.3 Lain versus C

Similarities

- Compiles to native code; no GC.
- Explicit memory management.
- No runtime overhead.
- Systems-level programming: raw pointers accessible in `unsafe`.

Differences

Aspect	Lain	C
Memory safety	Compile-time guarantees	Manual (error-prone)
Type safety	Strong, narrow widening	Weak, implicit conversion
Ownership	Tracked by compiler	Programmer discipline
Bounds checking	Static (VRA)	None
Division by zero	Static detection (E015)	Undefined behavior
Pattern matching	Exhaustive `case`	`switch` (fall-through)
Null safety	`T | none`, tested before use	Null everywhere

Aspect	Lain	C
Strings	Slices with length	Null-terminated arrays
Generics	Type parameters	Macros / `void*`
Header files	Module system	`#include`

Key insight. Lain provides C-level performance with compile-time safety guarantees. The C99 backend enables seamless interop with existing C libraries while eliminating the most common classes of C undefined behavior from the safe fragment.

D.4 Lain versus Zig

Similarities

- Comptime-based generics.
- No hidden control flow; no hidden allocations.
- C interoperability as a first-class concern.
- Explicit error handling with no exceptions.

Differences

Aspect	Lain	Zig
Memory safety	Compile-time ownership	Runtime (debug)
Ownership system	Borrow checker	Manual
Backend	C99	LLVM + self-hosted
Comptime scope	Constants, `comptime if`, type and value parameters	Full expression evaluation
Error handling	Error unions (`T | M`) + `try`, `else`	Error unions + `try`
Allocator model	C interop (malloc/free)	Custom allocators everywhere

Key insight. Both Lain and Zig use comptime for generics. Zig's comptime is more powerful (arbitrary expression evaluation); Lain's ownership system provides stronger compile-time memory safety guarantees than Zig's debug-mode runtime checks.

D.5 Lain versus Go

Similarities

- Simple, readable syntax.
- Fast compilation.
- No class hierarchies.

Differences

Aspect	Lain	Go
Memory management	Ownership (no GC)	Garbage collected
Runtime	None	GC + goroutine scheduler
Generics	Type parameters	Type parameters
Error handling	Error unions (`T | M`)	Multiple return values
Null safety	`T | none`	Nil (null)
Concurrency	Not supported	Goroutines + channels
Determinism	Yes (no GC pauses)	GC pauses possible

Key insight. Go achieves simplicity through garbage collection. Lain achieves similar simplicity goals through ownership-based safety, without a runtime — making Lain suitable for embedded and real-time applications where Go's GC is problematic.

D.6 Features unique to Lain

The following features distinguish Lain from all languages in the comparison:

Feature	Description
The `effects` row	Compiler-enforced purity with termination guarantees. No other listed language enforces purity at the declaration level.
Caller-site ownership annotations	`mov` and `var` are visible at every call site, making ownership transfer explicit and auditable.
Relational range analysis	Static verification of array bounds and arithmetic, carrying relations such as `i < n` between variables (an octagon), without an SMT solver.
`in` operator for range membership	`if idx in 0..arr.len` proves $0 \leq$ `idx` $<$ `arr.len` inside its branch; on a parameter or a field it is a refinement the writer establishes.
Bounded `while` with `decreasing`	Loops in functions with the empty row, by a ranking function the compiler infers or the programmer states, verified either way.
Non-consuming match (`case &expr`)	Pattern matching that does not consume the scrutinee; the scrutinee is borrowed for the duration of the match body.

Index

ABI, 117
abs, 132
ADT declaration, 72
ADT layout, 126
affine loop body, 99
algebraic data type, 7, 45
alignment, 27, 125
arena, 8
    compiler-internal, 128
arithmetic operator, 52
array type, 36
as operator, 55
assert
    module scope, 77
assert statement, 68
assignment statement, 64
attribute, 71, 94

basic concepts, 25
bitfield, 32
bitwise operator, 53
block, 63
block expression, 61
boolean literal, 19
boolean type, 36
borrow, 8
    active, 8
    phase, 8
borrow expression, 59
borrowed match, 83
bounded while
    rationale, 154
bounded while loop, 66
bounds check, 9
branch consistency, 84

C comparison, 156
C interoperability, 115
C99 backend
    rationale, 152
c_include, 115
calling convention, 91
case expression, 61
cast expression, 55
character literal, 19
clamp, 133
close_file, 131
comment, 15
comparison operator, 52
compilation model, 2
compile-time evaluation, 76, 103
compound assignment, 61
comptime, 9, 103
comptime block, 106
comptime declaration, 78
comptime if, 69, 106
comptime parameter, 104
comptime recursion depth, 106
conditional refinement, 96
conformance, 11
conforming implementation, 11
conforming program, 11
constant
    module scope, 75
constrained type, 7, 48
constraint annotation, 97
constructor expression, 61

declaration, 25, 71
decreasing measure, 9, 66
defer statement, 68
definition, 25
diagnostic codes, 143
diagnostic message, 6
    requirements, 12

E087, 101
effect row, 9, 87
effects row
    rationale, 151
`else` (handler), 56
entry point, 74

enum declaration, 72
enum layout, 126
enum type, 44
escape sequence, 20
exceptions
  absence of, rationale, 153
exhaustiveness, 9, 108
expression, 51
expression statement, 65
extern declaration, 75
extern function, 115

field access, 59
field invariant, 42
field-level borrow tracking, 85
File type, 131
floating-point literal, 19
floating-point type, 35
for statement, 69
func, 9
func purity constraints, 74
function, 87
function call, 60
function declaration, 73
function pointer, 91

garbage collector
  absence of, 128
generics, 103
  rationale, 154
Go comparison, 157
grammar summary, 135

heap allocation, 124

identifier, 17
if expression, 61
if statement, 65
immutable declaration, 63
implementation limits, 12
implementation-defined behavior, 6
import, 10
import declaration, 111
in guard, 10, 54
  VRA semantics, 97
in operator, 54
index expression, 59
integer literal, 17
integer overflow, 23, 34
integer rank, 7, 33
integer type, 29
  hardware-aligned, 30
  logical, 30
integer widening, 7, 33
interprocedural VRA, 98
interval, 10

keyword, 16

language comparison, 155
lexical conventions, 15
lifetime annotations
  rationale, 152
linear state, 8, 79
linear type, 7
linearity, 79
literal, 17
logical operator, 53
lvalue, 27

marker, 45
max, 132
measure inference, 67
memory model, 123
memory safety, 128
min, 132
mode preservation, 106
module, 10
module declaration, 111
module loading, 112
module system, 111
monomorphization, 10, 105
mov
  rationale, 152
move, 8
move expression, 59
move semantics, 83

name mangling, 114
name resolution, 26
name visibility, 113
non-lexical lifetime, 8, 81
`none`, 45
normative references, 3

object, 27, 123
object representation, 125
opaque type, 117
open_file, 131
operator, 21
operator precedence, 51
Option type, 133
overflow check, 99
ownership, 79
ownership diagnostics, 80
ownership in loops, 84

ownership in match, 109
ownership mode, 7, 79

packed struct, 32
panic, 93
parameter, 73
pattern matching, 107
pointer operations, 117
pointer type, 47
primary expression, 51
primitive type, 7, 29
print, 130
println, 130
proc, 9
purity, 9
purity enforcement, 88

qualified name, 10

range index, 59
range propagation, 96
range representation, 96
rationale, 151
read-write lock invariant, 8, 81
reference binding, 82
refinement type, 31
resource, 8
Result type, 133
return constraint, 10
return statement, 67
return type, 73
return value, 91
Rust comparison, 155
rvalue, 27

safe fragment, 13
scope, 25
scope (of document), 1
SIMD, 39
sized slice
    call-site check, 101
sized slice type, 38
slice type, 37
    sized, 38
slicing, 100
source file, 6
    encoding, 15
stack allocation, 124
standard library, 129
    module paths, 113
statement, 63
static assertion, 77
std.c, 129
std.fs, 131
std.io, 130
std.math, 132
storage duration, 26, 123
string interop, 117
string literal, 20
string pattern matching, 109
struct declaration, 72
struct invariant, 42
struct layout, 125
struct type, 41
sub-slice, 59, 100

termination, 9
termination analysis, 90
terms and definitions, 5
token, 15
token disambiguation, 23
`try`, 56
two-phase borrow, 8, 83
type, 29
    taxonomy, 29
    vector, 39
type alias
    refinement, 31
type as value, 104
type compatibility, 49
type equivalence, 49
type mapping, 116
type parameter, 103

UFCS, 10, 90
    rationale, 154
undefined behavior, 6
    in unsafe, 121
union type, 45
Universal Function Call Syntax, 10
unrestricted type, 7
unsafe
    rationale, 153
    what it does not disable, 120
    what it enables, 119
unsafe block, 68
unsafe code, 119
unsafe fragment, 13
unspecified behavior, 6

value range analysis (VRA), 10, 95
value semantics, 125
var binding
    of a borrow, 82
var parameter

primitive type, 81
variable
consumed, 8
variable declaration, 64
void type, 36
VRA, 95
rationale, 153
reference implementation, 101

while statement, 66
whitespace, 16
write-through, 81
write_file, 131

Zig comparison, 157